

SISTEMI DI REALTÀ VIRTUALE E AUMENTATA

Introduction to VR

VIRTUAL REALITY

Cosa è la Realtà Virtuale?

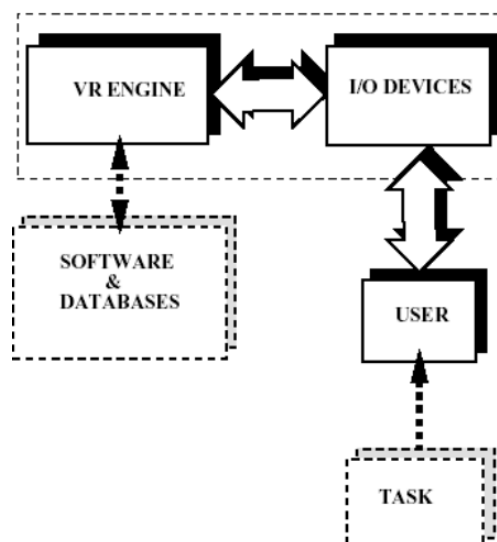
- "Un'interfaccia avanzata tra utente e computer che coinvolge simulazione e interazione in tempo reale attraverso diversi canali sensoriali." (vista, suono, tatto, olfatto, gusto)
- "Un ambiente artificiale creato con software e presentato all'utente in modo tale che venga percepito come un ambiente reale."
- "La Realtà Virtuale è l'uso della tecnologia informatica per creare l'effetto di un mondo tridimensionale interattivo in cui gli oggetti danno una sensazione di presenza spaziale."

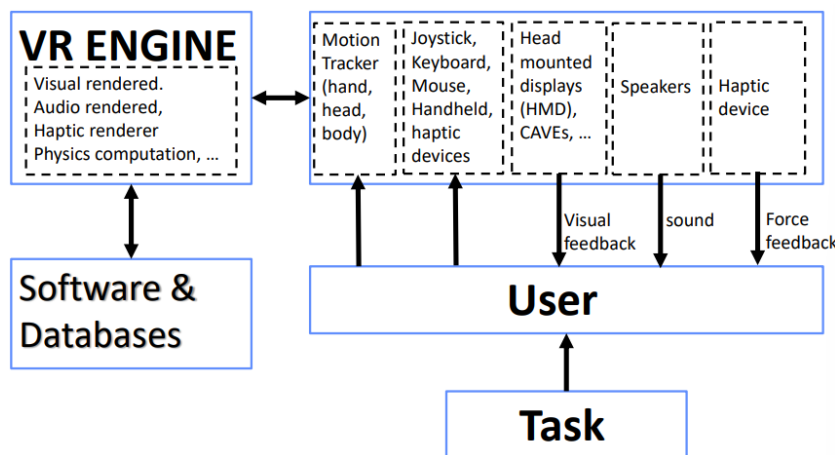


Aree di applicazione: industriale, visualizzazione (scientifica, medica, ...), intrattenimento, formazione, riabilitazione e terapia, modellazione e design.

- **Collaborative Virtual Reality:** gli utenti condividono lo stesso spazio virtuale (comunicazione naturale, gesti, cooperazione).
- **Distributed Virtual Reality:** collaborazione distribuita su lunghe distanze.

Gli elementi chiave di un sistema di realtà virtuale:





AUGMENTED REALITY

Combina mondo reale e virtuale, registra in 3D (gli oggetti virtuali e reali sono in relazione 3D tra loro), dati del mondo reale utilizzando telecamere.

AUGMENTED VIRTUALITY

Migliorare il mondo virtuale con immagini/modelli del mondo reale.

PROPRIOCEPTION

La *propriocezione* è la capacità di percepire la posizione relativa delle parti del nostro corpo e la quantità di *sforzo muscolare impiegata nel muoverle*. La propriocezione coinvolge tipicamente *molteplici input sensoriali* oltre all'esperienza. La percezione diventa "anormale" quando un input è presente senza gli altri. Ciò può causare affaticamento, mal di testa, nausea.

CYBER-SICKNESS

Una forma di *mal di movimento con sintomi* che includono *nausea, vomito, affaticamento* degli occhi, disorientamento, atassia e vertigini (Kennedy, Berbaum, & Drexler, 1994). Si ritiene che la cyber-sickness sia legata all'incongruenza dei segnali sensoriali, che *si verifica quando c'è un conflitto tra le percezioni nei diversi sensi* (uditivo, visivo, vestibolare, propriocettivo) o *quando le informazioni dei segnali sensoriali nell'ambiente VR sono incongruenti con ciò che il corpo percepisce* o con quanto ci si aspetta in base all'esperienza sensorimotoria "del mondo reale" dell'utente.

Esempi:

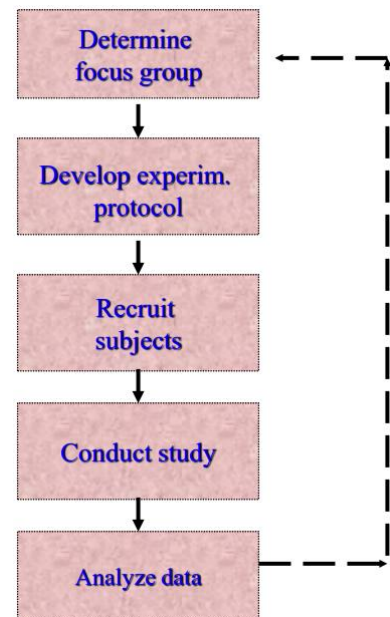
- ❖ *Errori del tracker che causano una discrepanza tra il movimento dell'utente e il movimento dell'avatar nella VR*
- ❖ *Ritardi di sistema* che producono grandi ritardi temporali tra il movimento dell'utente e la risposta della simulazione (grafica). Il ritardo è a sua volta influenzato dalla velocità di campionamento del tracking, dalla potenza del *computer*, dalla *velocità di comunicazione* e dall'*ottimizzazione del software*
- ❖ *Risoluzione dell'immagine dell'HMD e campo visivo (FOV)*. Una risoluzione scadente e un FOV ridotto non sono accettabili. FOV molto ampi possono essere problematici.

Le caratteristiche dell'utente possono *giocare un ruolo importante nella cyber-sickness: età, stato di salute* (gli utenti malati, inclusi quelli che assumono farmaci o droghe, sono più

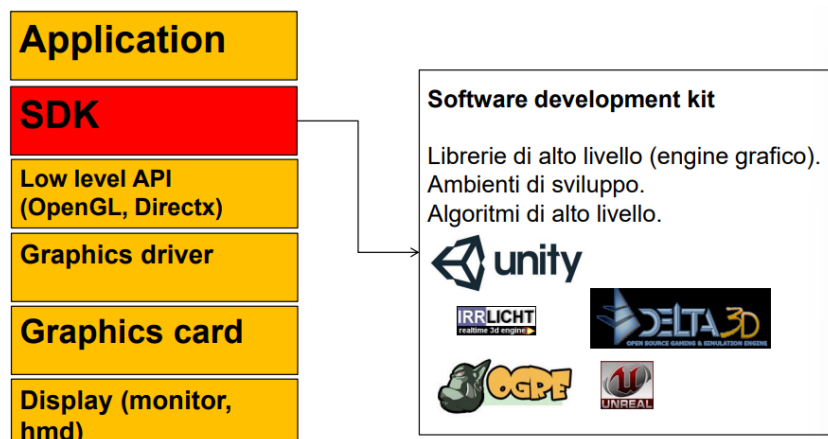
inclinati alla cyber-sickness), suscettibilità al mal di movimento (alcune persone sono più sensibili al mal di movimento rispetto ad altre).

THE STAGES OF HUMAN FACTORS STUDIES

- *Determinare focus group*: qual è il problema? determina l'ipotesi (es. Grafica più veloce è migliore); stabilisce il tipo di studio (usabilità, sociologico, ecc.); valutazione obiettiva, valutazione soggettiva o entrambe?
- *Protocollo sperimentale*: quali compiti vengono svolti durante un'analisi? quante prove vengono ripetute per sessione? quante sessioni al giorno e quanti giorni per lo studio? quanti soggetti in ciascun gruppo sperimentale? quali misurazioni pre e post-prova vengono effettuate? quali domande vengono incluse nel modulo di valutazione soggettiva?
- *Reclutamento dei soggetti*: è necessario arruolare un numero sufficiente di soggetti nello studio per avere significatività statistica; i soggetti vengono selezionati per idoneità allo studio; i soggetti firmano il modulo di consenso
- *Condurre lo studio*: le misurazioni devono essere sensibili (per distinguere tra utenti novizi ed esperti), affidabili (ripetibili e coerenti) e valide (veritieri)
- *Analisi dei dati*: gli esperimenti memorizzano variabili diverse a seconda del tipo di test:
 - tempo di completamento del compito → tempo necessario per finire il compito (tempo del sistema, sequenza delle azioni)
 - tasso di errore del compito → numero o percentuale di errori commessi durante una prova
 - analisi della Varianza (ANOVA) → pacchetto statistico utilizzato per analizzare i dati e determinare se esistono differenze statistiche tra prove, condizioni, parametri



LEVELS OF A COMPUTER GRAPHICS APPLICATION



MODELING A VIRTUAL WORLD

Software come Unity ti permette di creare un modello geometrico di un mondo 3D. È simile a disegnare su carta a mano, ma in formato digitale. La maggior parte dei modelli sono superfici composte da un insieme di triangoli.

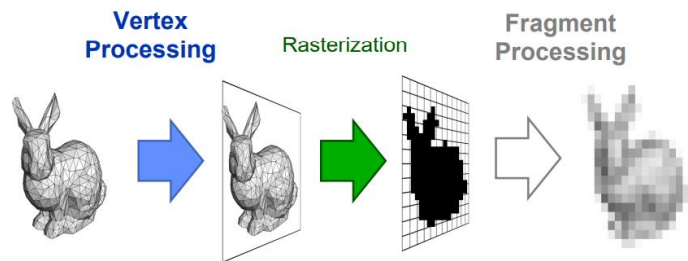
I modelli 3D sono spesso generati utilizzando uno strumento di modellazione come Maya o Blender. Esporta il tuo modello come file, importalo nel tuo motore di gioco (ad esempio Unity) o nella tua applicazione (ad esempio un programma OpenGL).

3D GRAPHICS PIPELINE

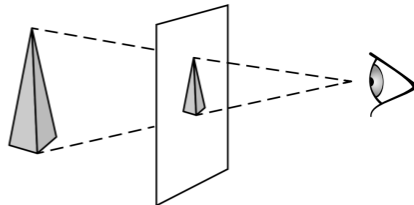
RGB Color

Rosso, Verde e Blu sono i colori primari. Ogni colore primario può assumere un valore di intensità che va da 0 (minimo) a 1 (massimo). Mescolare questi tre colori primari a diversi livelli di intensità produce una varietà di colori. La raccolta di tutti i colori ottenuti mediante tale combinazione lineare di rosso, verde e blu forma lo spazio colore RGB a forma di cubo. I colori sono vettori nello spazio 3D. Qualsiasi colore è ottenuto come una combinazione lineare dei colori primari. Esempio: (1, 0.5, 0) arancione (1, 1, 0) giallo.

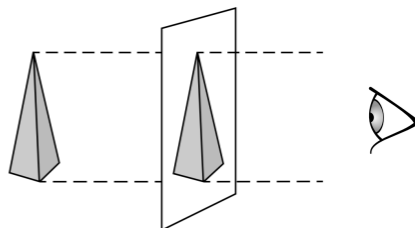
I frammenti sono simili ai pixel, ma non sono necessariamente i pixel finalizzati che vedi in un'immagine. Ogni frammento ha una posizione 3D e un colore. Il valore finale del pixel si ottiene tipicamente applicando la rimozione delle superfici nascoste e, possibilmente, il compositing a un insieme di frammenti.



Perspective Projection



Orthographic Projection



Quanti FPS sono sufficienti? Studi hanno dimostrato che, in pratica, qualsiasi configurazione VR che generi frame rate inferiori a 90 fotogrammi al secondo (FPS) è probabilmente destinata a causare disorientamento, nausea e altri effetti negativi per l'utente. Più basso è il frame rate, peggiori sono gli effetti. Pertanto, l'obiettivo per gli sviluppatori di VR è quello di puntare a 90 FPS in ogni momento nel loro software.

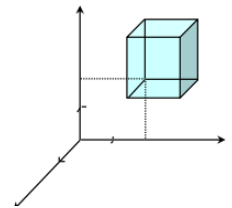
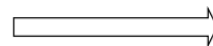
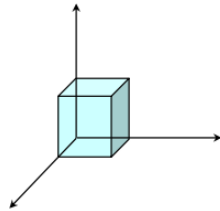
3D GEOMETRICAL TRANSFORMATIONS

Un punto 3D P è rappresentato nelle coordinate omogenee da un vettore a 4 dimensioni.

$$P = \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

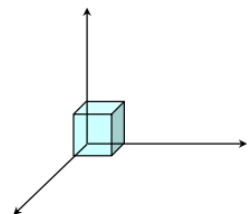
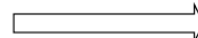
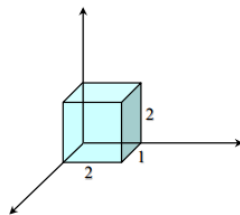
Traslazione: $T(d_x, d_y, d_z) = \begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$

$$\begin{bmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x + d_x \\ y + d_y \\ z + d_z \\ 1 \end{bmatrix}$$

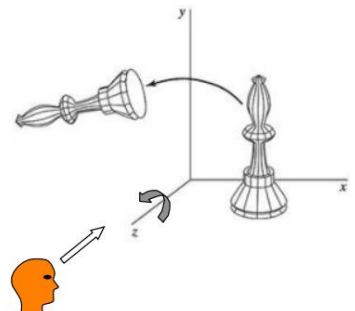


Scala: $S(s_x, s_y, s_z) = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

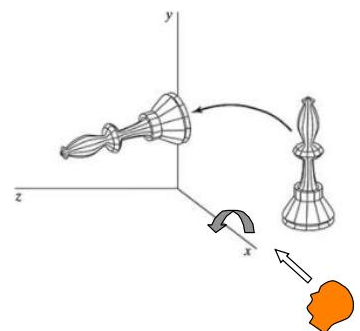
$$\begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} s_x \cdot x \\ s_y \cdot y \\ s_z \cdot z \\ 1 \end{bmatrix}$$



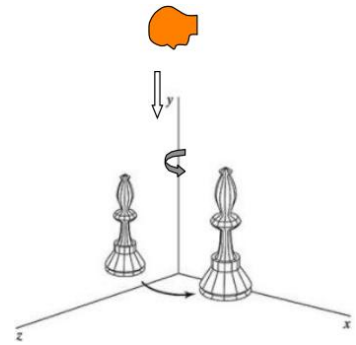
Rotazione attraverso l'asse z: $R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$



Rotazione attraverso l'asse x: $R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$



Rotazione attraverso l'asse y: $R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$



COMPOSITION OF 3D TRANSFORMATIONS

La moltiplicazione delle matrici rappresenta la composizione; la composizione si legge da destra a sinistra. Rotazione di θ attorno a un asse parallelo a z, passando per il punto p:

1. Trasla l'oggetto all'origine
2. Ruota attorno a z di θ
3. Trasla di nuovo indietro

$$M = T^{-1}RT$$

$$T^{-1}RT = \begin{pmatrix} 1 & 0 & 0 & p_x \\ 0 & 1 & 0 & p_y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -p_x \\ 0 & 1 & 0 & -p_y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$= \begin{pmatrix} \cos \theta & -\sin \theta & 0 & (-p_x \cos \theta + p_y \sin \theta + p_x) \\ \sin \theta & \cos \theta & 0 & (-p_x \sin \theta - p_y \cos \theta + p_y) \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

ANIMATION AND INTERPOLATION

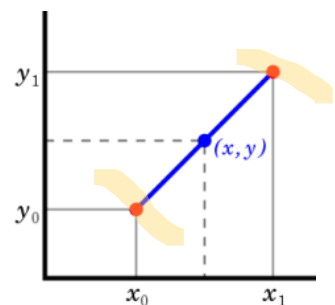
L'animazione utilizza spesso una tecnica chiamata **keyframing** (o interpolazione a fotogrammi chiave). L'animatore specifica lo stato di un modello nei **fotogrammi chiave**. L'animazione riempie i fotogrammi tra i fotogrammi chiave utilizzando l'interpolazione.

LINEAR INTERPOLATION

L'interpolazione lineare è il modo più semplice per riempire i valori intermedi. Basta creare una linea retta tra **due punti e campionare a intervalli regolari**. Per interpolare linearmente tra un punto P e un punto Q, costruisci un'equazione **parametrica per la linea PQ**.

$$L(t) = (1 - t)P + tQ = P + t(Q - P)$$

Puoi quindi utilizzare il parametro t per campionare la linea. Ad esempio, al tempo $t=0.5$, come nell'immagine.



QUATERNIONS

Alternativa agli **angoli di Eulero** e alle matrici di rotazione per rappresentare una rotazione 3D. Sviluppata da Sir William Rowan Hamilton [1843]. I **quaternioni sono numeri complessi a 4 dimensioni con un asse reale e tre assi immaginari: i, j, k.**

$$q = q_0 + iq_1 + jq_2 + kq_3$$

Si scopre che i **quaternioni unitari sono in effetti una rappresentazione angolo-asse** di un'orientazione, basta pensarli come un modo per codificare queste informazioni.

Quaternioni unitari: $|q| = \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} = 1$

Data un'angolazione θ e un asse unitario $a = (a_x, a_y, a_z)$, è facile convertire da e verso un quaternione unitario.

Data una matrice di rotazione, è "facile" convertire da e verso un quaternione unitario:

$$q = \left[\cos \frac{\theta}{2} \quad a_x \sin \frac{\theta}{2} \quad a_y \sin \frac{\theta}{2} \quad a_z \sin \frac{\theta}{2} \right] \text{ o } q = \left\langle \cos \frac{\theta}{2}, a \sin \frac{\theta}{2} \right\rangle$$

I quaternioni consentono un'interpolazione stabile e costante delle orientazioni, cosa che non può essere fatta facilmente con gli angoli di Eulero.

A ROTATION QUATERNION IS UNIT-LENGTH

$$\begin{aligned} |q| &= \sqrt{q_0^2 + q_1^2 + q_2^2 + q_3^2} = \sqrt{\cos^2 \frac{\theta}{2} + a_x^2 \sin^2 \frac{\theta}{2} + a_y^2 \sin^2 \frac{\theta}{2} + a_z^2 \sin^2 \frac{\theta}{2}} \\ &= \sqrt{\cos^2 \frac{\theta}{2} + \sin^2 \frac{\theta}{2} (a_x^2 + a_y^2 + a_z^2)} = \sqrt{\cos^2 \frac{\theta}{2} + \sin^2 \frac{\theta}{2} |a|^2} = \sqrt{\cos^2 \frac{\theta}{2} + \sin^2 \frac{\theta}{2}} \\ &= \sqrt{1} = 1 \end{aligned}$$

QUATERNION TO ROTATION MATRIX

Per convertire un quaternione in una matrice di rotazione:

$$\begin{bmatrix} 1 - 2q_2^2 - 2q_3^2 & 2q_1q_2 - 2q_0q_3 & 2q_1q_3 + 2q_0q_2 & 0 \\ 2q_1q_2 + 2q_0q_3 & 1 - 2q_1^2 - 2q_3^2 & 2q_2q_3 - 2q_0q_1 & 0 \\ 2q_1q_3 - 2q_0q_2 & 2q_2q_3 + 2q_0q_1 & 1 - 2q_1^2 - 2q_2^2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

ROTATION MATRIX TO QUATERNION

$$\begin{bmatrix} m_{0,0} & m_{0,1} & m_{0,2} & m_{0,3} \\ m_{1,0} & m_{1,1} & m_{1,2} & m_{1,3} \\ m_{2,0} & m_{2,1} & m_{2,2} & m_{2,3} \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad \begin{aligned} q_0 &= \frac{1}{2} \sqrt{\text{tr}(M)} \\ q_1 &= \frac{m_{21} - m_{12}}{4q_0} \\ q_2 &= \frac{m_{02} - m_{20}}{4q_0} \\ q_3 &= \frac{m_{10} - m_{01}}{4q_0} \end{aligned}$$

$\text{tr}(M)$ è la traccia, somma degli elementi diagonali $\rightarrow \text{tr}(M) = m_{00} + m_{11} + m_{22} + 1$

QUATERNION MULTIPLICATION

Possiamo eseguire la moltiplicazione sui quaternioni.

$$q_1 = w_1 + x_1i + y_1j + z_1k \quad \text{e} \quad q_2 = w_2 + x_2i + y_2j + z_2k$$

$$q_1 = [w_1, v_1] \quad \text{e} \quad q_2 = [w_2, v_2] \quad \text{dove} \quad v_1 = (x_1, y_1, z_1) \quad \text{e} \quad v_2 = (x_2, y_2, z_2)$$

$$q_1 q_2 = [w_1 w_2 - v_1 \cdot v_2, w_1 v_2 + w_2 v_1 + v_1 \times v_2]$$

QUATERNION DOT PRODUCTS

Il prodotto scalare di due quaternioni: $p \cdot q = p_0 q_0 + p_1 q_1 + p_2 q_2 + p_3 q_3 = |p| |q| \cos \varphi$

ROTATION USING QUATERNIONS

Ogni quaternione unitario q corrisponde a una rotazione 3D. Dato un punto 3D che vogliamo ruotare $r=(r_x, r_y, r_z, 1)$; sia $[0, r]$ un quaternione. La rotazione può essere espressa come: $R(q) = q[0, r]q^{-1} = [0, r']$, dove $r'=(r'_x, r'_y, r'_z, 1)$ è il vettore ruotato. Il prodotto di due quaternioni unitari p e q corrisponde alla composizione delle due rotazioni: prima q , poi p

$$pq[0, r](pq)^{-1} = pq[0, r]q^{-1}p^{-1}$$

L'inverso di un quaternione unitario $q = q_0 + iq_1 + jq_2 + kq_3$ è $q^{-1} = q_0 - iq_1 - jq_2 - kq_3$

SPHERICAL LINEAR INTERPOLATION

L'interpolazione lineare sferica di due quaternioni unitari a e b è:

$$\text{Slerp}(t, a, b) = \frac{\sin((1-t)\theta)}{\sin \theta} a + \frac{\sin(t\theta)}{\sin \theta} b$$

Utile per animare oggetti tra due rotazioni arbitrarie.

Input Devices

Hardware che consente all'utente di comunicare con il sistema VR. Classificazione:

- ❖ **Gradi di Libertà (DOF):** numero di valori/risultati indipendenti, comunemente utilizzati:
 - i. 2DOF → 2D, ad es. mouse
 - ii. 3DOF → posizione
 - iii. 3DOF → orientamento (rotazione rispetto agli assi del sistema di coordinate)
 - iv. 6DOF → posizione + orientamento
- ❖ **Discreto – Continuo:**
 - Discreto/basato su eventi → genera un evento alla volta in base all'input dell'utente (esempi: tastiera, pulsanti del mouse, DataGlove (gesti), pulsanti del joystick)
 - Continuo/campionato → genera eventi continuamente in risposta all'azione dell'utente (esempi: tracker di movimento, movimento del mouse, DataGloves/movimento del joystick)
- ❖ **Dispositivi di Input Desktop/Non-Desktop:**
 - Dispositivi di input desktop usati nella VR da desktop: tastiere, mouse 2D e trackball, joystick, tablet con penna, guanti, dispositivi a 6 gradi di libertà per il desktop (mouse 3D, Leap Motion), Feedback aptico
 - Dispositivi di input non desktop (gli utenti sono in piedi/movimento fisico): dispositivi wireless portatili (guanto wireless, pad wireless) → possono essere tracciati in 6D (mappatura diretta del movimento della mano) o non tracciati, input vocale (riconoscimento vocale)
- ❖ **Principio Tecnologico** (meccanico, magnetico, ottico, inerziale, visivo, elettrico)

Cosa rende difficile l'interazione 3D? Input spaziale 3D, mancanza di standard, affaticamento, i dispositivi di fascia alta sono costosi

Principali compiti di interazione nella VR:

- I. **Selezione**: scegliere oggetto(i) da un insieme
- II. **Manipolazione**: modificare le proprietà degli oggetti (in particolare posizione/orientamento, forma, colore)
- III. **Navigazione**: spostamento – muovere l'avatar
- IV. **Cambiare lo stato o la modalità del sistema**

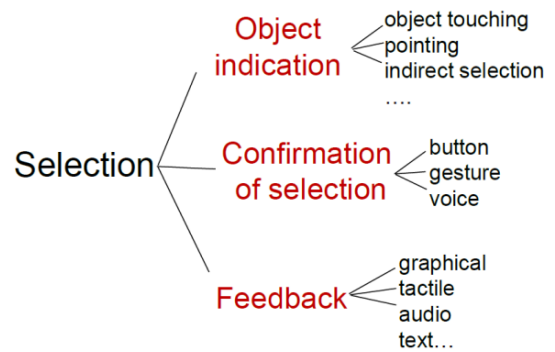
SELECTION

Gli **obiettivi** della selezione sono **attivare un oggetto**, indicare un'azione sull'oggetto, **raggiungere la posizione dell'oggetto**, preparare **la manipolazione**. I principali **fattori che influenzano** la selezione sono la **distanza** dell'oggetto dall'utente, **dimensione** dell'oggetto, **densità** degli oggetti nell'area. La **selezione può essere**:

- **Isomorfica**: **corrispondenza** rigorosa e geometrica **1:1** tra **mondo fisico e virtuale** (la più naturale, es: usare una mano virtuale)
- **Non isomorfica**: **strumenti** artificiali che **estendono** il volume di lavoro o la **lunghezza del braccio**

Tipi comuni di **selezione** di oggetti 3D:

- **Tocco**: toccare un oggetto 3D con una mano virtuale/puntatore
- **Puntamento** (ray casting)
- **Tecnica go-go** (esempio di selezione non isomorfica): estensione artificiale del braccio con una mappatura non lineare tra la posizione della mano fisica e quella virtuale
- **Nominazione** (riconoscimento vocale, selezione indiretta)



MANIPULATION

Gli **obiettivi** della manipolazione di oggetti 3D sono **afferrare** oggetti, **posizionare** oggetti, **usare strumenti**. Tipi comuni di manipolazione di oggetti 3D:

- **Dispositivi portatili** (3 DOF, 6 DOF)
- **Mano virtuale**: naturale, complessa da implementare, **portata limitata**, **richiede un dispositivo appropriato** (guanto, Leap Motion)

NAVIGATION

L'obiettivo della navigazione dell'utente è **impostare la posizione (e l'orientamento) del punto di vista dell'utente**. Si tratta della tecnica di interazione di base e più comune negli ambienti virtuali. Metafore di spostamento:

- Metafore di **spostamento "naturali"** (camminata reale, tapis roulant)
- Guida: specifica continua/discreta della direzione del movimento:
 - **Direzione basata sullo sguardo**
 - **Dispositivo fisico** (volante, joystick)
- **Basato su target**: specifica discreta della **posizione dell'obiettivo (teletrasporto)**

CHANGING SYSTEM STATE OR MODE

Consente l'emissione di **comandi** e/o di **cambiare modalità**. Tipi comuni di tecniche di controllo del sistema: sistemi di **menu**, **comandi vocali**, **gesti/posture**.

CYBERGLOVE

Utilizza **18-22 sensori resistivi**, gli angoli sono ottenuti misurando le tensioni, 150 Hz, **richiede calibrazione quando viene indossato sulla mano**, risoluzione di 0,5 gradi.

HAPTIC INTERFACES

HAPTICS: "*La scienza di applicare il tatto all'interazione umana con i computer*"

Interfaccia aptica: un dispositivo di output che **fornisce feedback tattile e/o feedback cinestetico** all'utente.

Feedback tattile: agisce sui **recettori della pelle** e subito sotto di essa. Permette al cervello di percepire **vibrazioni**, **pressione**, **tocco**, texture, ecc.

Feedback cinestetico: feedback di forza che **agisce su muscoli**, **articolazioni** e **tendini** del corpo umano.

Classificazione basata sul numero di gradi di libertà (DOF)

- DOF bassi: fino a **3 DOF (forze)**
- DOF alti: fino a **6 DOF (forze e coppie)**
- DOF molto alti: **più di 6 DOF** (quando è necessario applicare forze in più punti di **contatto**)

Classificazione basata sulla cinematica:

- Seriale
- Parallelo

Classificazione basata sul numero di punti di contatto con il corpo:

- **Punto Singolo**: il più comune, solo un **punto di contatto** (chiamato "punto di contatto")
- **Punti Multipli**: almeno **due punti di contatto** (es: due dita che toccano un oggetto)
- **Replicare il Compito**: il dispositivo aptico ha una forma che assomiglia a uno **strumento reale** (es: **dispositivo chirurgico**)

Classificazione basata sulla strategia di controllo:

- **Impedenza** (la più comune): l'**input** è la **posizione o la velocità** del punto di contatto, e l'**output** è un **feedback di forza applicato al punto di contatto che l'utente percepisce**
- **Admittanza**: l'**input** è la **forza applicata dall'utente al punto di contatto**, e l'**output** è la **velocità o la posizione del punto di contatto**

Parametri:

- I. **Spazio di lavoro/Intervallo di movimento**: il volume **raggiungibile** dal punto di contatto nello spazio reale (deve essere scalato **per adattarsi** alle dimensioni dell'ambiente virtuale)
- II. **Forza di picco**: l'intensità **massima** del feedback di forza percepito dall'utente
- III. **Risoluzione** (dispositivi di impedenza): **variazione minima di posizione rilevabile**
- IV. **Precisione** (dispositivi di impedenza): **precisione della misura della posizione**

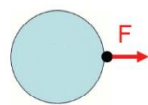
Cybergrasp: Punti Multipli, DOF Molto Alto. **Esoscheletro riflettente la forza che si adatta a un guanto di dati CyberGlove** e aggiunge feedback di forza a ciascun dito. Con il sistema di

feedback di forza CyberGrasp, gli utenti possono percepire la dimensione e la forma degli oggetti 3D generati al computer in un mondo virtuale simulato. Le forze di presa sono prodotte da una rete di tendini instradati fino alle punte delle dita tramite l'esoscheletro. Ci sono cinque attuatori, uno per ogni dito, che possono essere programmati singolarmente per impedire che le dita dell'utente penetrino o schiaccino un oggetto solido virtuale.

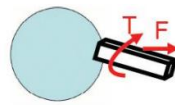
CyberForce: questa unità fornisce feedback di forza alla mano e al braccio dell'utente.

Rendering aptico: il rendering aptico è il processo di calcolo e generazione di forze in risposta all'interazione dell'utente con l'ambiente virtuale. Un dispositivo aptico di solito consente all'utente di muovere un oggetto 3D appartenente all'ambiente virtuale. Pertanto, il dispositivo aptico può essere visto come una copia di un oggetto dell'ambiente virtuale.

L'oggetto virtuale manipolato è chiamato "proxy". Il proxy virtuale nel mondo virtuale segue il movimento del punto di contatto nel mondo reale. Una frequenza di aggiornamento di 1 kHz è necessaria per un'esperienza confortevole per quasi tutte le applicazioni di rendering aptico.



Virtual proxy: point or sphere
input: 3 DOF, position
output: 3 forces



Virtual proxy: 3D object
input: 6 DOF, position+orientation
output: 3 forces + 3 torques

HAPTIC RENDERING TECHNIQUES (IMPEDANCE DEVICES)

1) Direct Rendering

La posizione/orientamento misurati dal sensore aptico vengono applicati direttamente al proxy virtuale con una relazione lineare. La forza restituita all'utente è calcolata come direttamente proporzionale alla profondità di penetrazione del proxy virtuale con gli altri oggetti dell'ambiente grafico.

Esempio: contatto tra il proxy e una sfera

Come verificare se c'è una collisione:

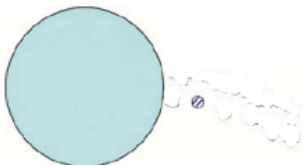


Sphere $(x_s, y_s, z_s), R$

User (x_u, y_u, z_u)

$$d = \sqrt{(x_u - x_s)^2 + (y_u - y_s)^2 + (z_u - z_s)^2}$$

Come calcolare la forza:

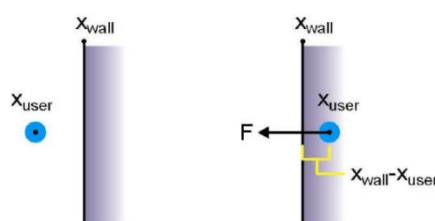


$$F = \begin{cases} 0 & d > R \\ k(R - d)\bar{N} & \text{altrimenti} \end{cases}$$

$$\bar{N} = \frac{1}{d} \begin{bmatrix} x_u - x_s \\ y_u - y_s \\ z_u - z_s \end{bmatrix}$$

Un altro esempio: contatto con un muro

If $x_{user} > x_{wall}$, $F = k(x_{wall} - x_{user})$
stiffness $k > 0$



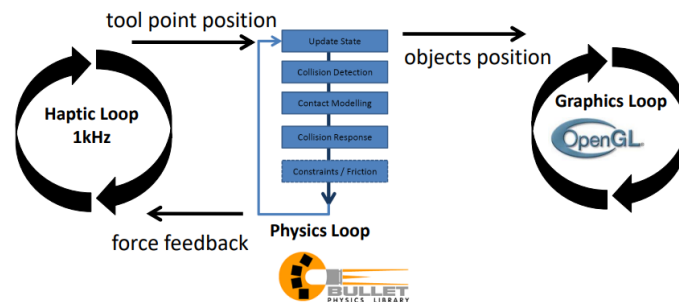
Rendering diretto per oggetti 3D più complessi:

1. Dividere l'oggetto in sotto-volumi
2. Ogni sotto-volume è associato a una porzione della superficie dell'oggetto
3. Il feedback di forza è calcolato come proporzionale alla profondità di penetrazione del proxy virtuale nella sottoparte dell'oggetto. Funziona con oggetti semplici, ma può generare discontinuità nella forza percepita.

2) Virtual Coupling

Basato su motori fisici. Il “proxy virtuale” è accoppiato con il Punto di Contatto tramite un modello a molla e smorzatore. La posizione relativa (così come la velocità lineare relativa) tra il punto di contatto e il proxy virtuale vengono utilizzate per calcolare le forze (F) che vengono applicate al proxy virtuale. Il proxy virtuale non può penetrare altri oggetti. Il feedback di forza (forza opposta $-F$) all'utente viene generato quando si verifica un contatto tra il proxy virtuale e altri oggetti 3D.

$F = k_L d + \delta_L v$, dove k_L, δ_L sono le costanti di rigidità e di smorzamento della molla lineare, d è l'offset delle molle per la distanza lineare, v è la velocità lineare relative dell'oggetto dinamico.



INPUT DEVICES

Utilizzo della visione computerizzata per tracciare l'input della mano. Crea un irresistibile senso di presenza e interazione naturale. Le sfide da risolvere sono non avere il senso del tatto ed è necessaria la linea visiva.



LEAP MOTION

2 telecamere mono a infrarossi (IR) (300 fps). 3 LED IR, che generano un modello 3D di punti di luce IR. Elaborazione dei dati della nuvola di punti su PC. Distanza <1 metro. I vantaggi sono l'alta precisione e l'interazione con due mani. Mentre gli svantaggi sono la bassa portata e il campo visivo ridotto.

LEAP MOTION SDK

L'API Leap Motion restituisce i dati di tracciamento sotto forma di frame. Ogni oggetto Frame contiene elenchi di entità tracciate, come mani, dita e strumenti, oltre a oggetti che rappresentano i gesti riconosciuti e fattori che descrivono il movimento complessivo delle mani nella scena. Un oggetto Mano include posizione e velocità del palmo, vettori direzione e normale, base ortonormale.

Dita: **posizione** e **velocità** della punta, vettori direzione, base ortonormale, **lunghezza** e **larghezza**

Ossa: posizioni delle articolazioni, base ortonormale, lunghezza e larghezza

Braccio: posizioni del polso e del gomito, vettore direzione, base ortonormale, lunghezza e larghezza

I **movimenti** sono movimenti continui della mano, stime di come la **posizione degli oggetti tracciati** (mani, dita e strumenti) **cambia nel tempo**. I **gesti** sono **modelli di movimento** che possono essere utilizzati per **attivare determinate azioni**. L'API Leap Motion include gesti di scorrimento, cerchio e tocco, che vengono emessi come **oggetti evento nei fotogrammi** quando vengono riconosciuti.

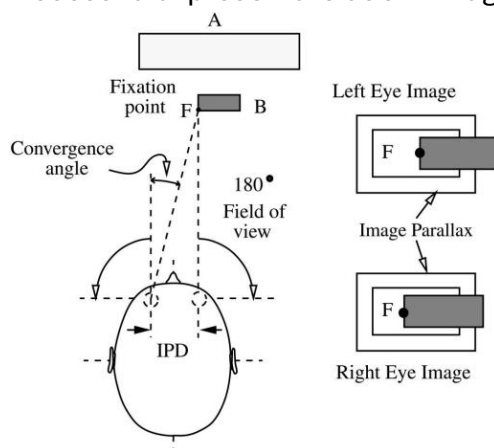
Output Devices

STEREOGRAPHICS CONCEPTS

Un **display grafico** è un'interfaccia informatica che presenta immagini sintetiche del mondo a uno o più utenti che interagiscono con il mondo virtuale. Tali immagini possono essere **monoscopiche o stereoscopiche**.

La **percezione della profondità nella visione monoscopica** (es. grafica 3D standard su un singolo schermo) si basa sulla **prospettiva**, sull'**occlusione** (un oggetto blocca la vista di un altro), sull'**illuminazione**, sulle **ombre** e sulle **texture**. In questo corso siamo interessati ai dispositivi per la visione stereoscopica. Il cervello umano costruisce una visione 3D del mondo. **Le visualizzazioni stereoscopiche "ingannano" il cervello presentando immagini adatte all'occhio destro e sinistro** (visione binoculare). L'entità dello spostamento dipende dalla "distanza interpupillare" (IPD) (che varia per ogni persona da 50 a 75 mm). **Per ottenere un vero effetto 3D è necessario risolvere due problemi: creare immagini** corrette a destra e a sinistra e **presentare** a ciascun occhio un'immagine appropriata.

Necessità di presentare due immagini dello stesso ambiente VR:



Caratteristiche del display visivo:

- Campo visivo (FOV), FOV umano ~200°
- Risoluzione spaziale (dpi)
- Geometria dello schermo (rettangolare, emisferico...)
- Meccanismo di trasferimento della luce (proiezione anteriore/posteriore)
- Frequenza di aggiornamento (Hz)
- Ergonomia

STEREOGRAPHIC DISPLAY TECHNIQUES

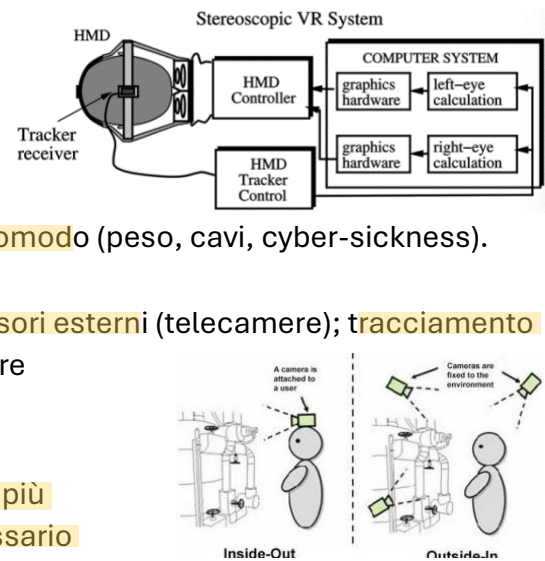
- A. **Le due immagini possono essere presentate contemporaneamente su due display** (HMD) o nello stesso display (occhiali stereo passivi).
 - Display montato sulla testa (HMD)
 - Occhiali stereo passivi (filtri di polarizzazione)
- B. **Le due immagini possono anche essere presentate in sequenza temporale su un unico display (occhiali stereo attivi)**
 - Occhiali stereo attivi (mostrano coppie di immagini stereo in sequenza)

HMD (HEAD MOUNTED DISPLAY)

Schermo con **ottiche speciali davanti a ciascun occhio**, **esperienza immersiva**. Il display occlude il mondo reale, esperienza per un **singolo utente**. Il **tracciamento della testa** consente una visione stereoscopica che segue il movimento della testa dell'utente. Campo visivo (FOV) limitato. **Intrusivo, scomodo** (peso, cavi, cyber-sickness).

Tipi di tracciamento:

- **Tracciamento outside-in:** sono necessari sensori esterni (telecamere); **tracciamento limitato a un'area specifica**; più **preciso**, migliore tracciamento dei controller; **funziona al buio**
- **Tracciamento inside-out:** le **telecamere sono posizionate sul visore** (HMD), funziona in **spazi più grandi**, abilita la realtà aumentata; **non è necessario alcun dispositivo esterno per il tracciamento**; **localizzazione e mappatura simultanea** (SLAM/odometria visiva, classico problema della visione artificiale e robotica)



HMD – PC VR

Name	Head Tracking	Positional Tracking	Display	Resolution (per eye)	Refresh Rate	FOV	
Oculus Rift CV1	Accelerometer, gyroscope, magnetometer	Outside-in	OLED	1080x1200	90 Hz	110°	
HTC Vive	Accelerometer, gyroscope, structured light	Outside-in	OLED	1080x1200	90 Hz	110°	
HTC Vive Pro	Accelerometer, gyroscope, structured light	Outside-in	AMOLED	1440x1600	90 Hz	110°	
Valve Index	Accelerometer, gyroscope, magnetometer	Outside-in	LCD	1440x1600	up to 120 Hz	135°	
PlayStation VR	Accelerometer, gyroscope	Outside-in	OLED	960x1080	up to 120 Hz	100°	
HP Reverb	Accelerometer, gyroscope, magnetometer, proximity sensor	Inside-out	LCD	2160x2160	90 Hz	114°	

Oculus Rift S	Accelerometer, gyroscope, magnetometer	Inside-out	LCD	1280x1440	80 Hz	115°	
---------------	--	------------	-----	-----------	-------	------	---

Esistono HMD che funzionano anche senza un PC:

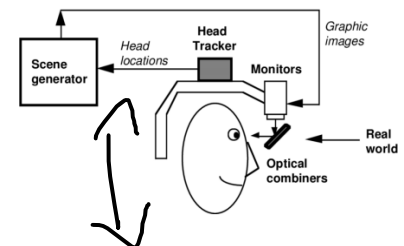
- **Oculus Quest 2:**
 - Processore: Qualcomm Snapdragon XR2
 - Storage: 64GB o 256GB
 - Display Panel: LCD
 - Display Resolution: 1832x1920 per eye
 - Display Refresh: 90Hz
 - Optics: Fresnel
- **Oculus Quest Pro:**
 - Processore: Qualcomm Snapdragon XR2+
 - Storage: 256GB
 - Display Panel: LCD
 - Display Resolution: 1800x1920 per eye
 - Display Refresh: 90Hz, 120Hz
 - Optics: Pancake
 - Eye tracking

Realtà aumentata:

CONFIG + REGIS

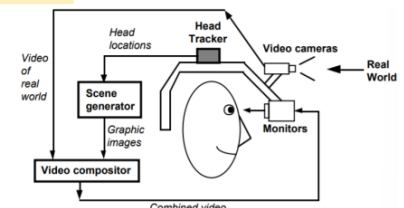
❖ **HMD a visione ottica diretta:** alta risoluzione

dell'immagine fisica; nessun ritardo tra il movimento e l'immagine fisica; ritardo tra l'immagine fisica e quella virtuale; gli oggetti fisici sono messi a fuoco alla loro corretta distanza; è necessaria la registrazione tra l'immagine fisica e quella virtuale; le immagini virtuali appaiono spesso semi-trasparenti a causa della perdita di luce



❖ **Visione basata su video:** il ritardo tra l'immagine fisica e

quella virtuale può essere compensato; la fotocamera può essere utilizzata anche per il tracciamento; il mixer video può aggiungere o sottrarre luce; gli oggetti virtuali possono essere disegnati in nero; gli oggetti fisici possono essere completamente sostituiti; è necessaria la registrazione tra l'immagine fisica e quella virtuale; gli oggetti virtuali possono essere dietro agli oggetti fisici



PASSIVE STEREO GLASSES

Due immagini sovrapposte sullo schermo contemporaneamente. Sono occhiali con filtri economici per separare le immagini destro/sinistro. Non è necessaria la sincronizzazione tra proiettori e occhiali. Gli approcci comuni sono Anaglifo (rosso/blu) o lenti polarizzate. Sono senza cavi (leggeri ed ergonomici, meno invasivi rispetto agli HMD). Funzionano bene con schermi di grandi dimensioni (alta risoluzione). Supportano sistemi di tracciamento della testa. Solitamente usano due proiettori:

- i. Sinistra → filtro verticale davanti alla lente
- ii. Destra → filtro orizzontale davanti alla lente

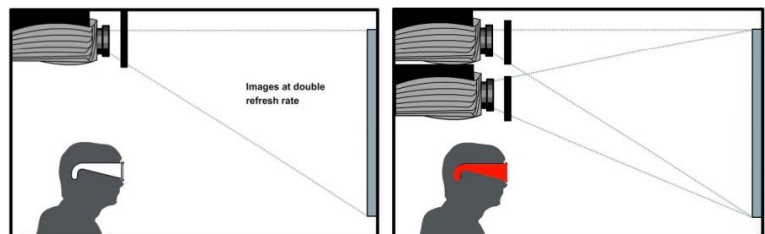
Indossare occhiali con filtri polarizzati: occhio sinistro → filtro verticale, occhio destro → filtro orizzontale.

ACTIVE STEREO GLASSES

Funzionano alternando rapidamente l'immagine per l'occhio sinistro e per l'occhio destro sullo stesso schermo. Richiedono occhiali con otturatore e elettronica personalizzata (monitor/proiettore e occhiali sono sincronizzati). L'immagine per l'occhio sinistro viene visualizzata solo quando la lente sinistra è trasparente (e la destra è opaca) e viceversa. Senza cavi (leggeri ed ergonomici, meno invasivi rispetto agli HMD). Funzionano bene con schermi di grandi dimensioni (alta risoluzione). Supportano sistemi di tracciamento della testa.

PROJECTORS

- Proiettore singolo con un alto refresh di 120Hz → 60Hz per occhio (occhiali stereo attivi)
- Configurazione a doppio proiettore (occhiali stereo passivi): un proiettore per ogni occhio, richiede una calibrazione precisa



LARGE VOLUME DISPLAYS

Permettono a diversi utenti co-locati di visualizzare una vista stereoscopica del mondo virtuale. Possono essere classificati come display di grande volume basati su monitor o display di grande volume basati su proiettore. Consentono maggiore libertà di movimento rispetto ai display personali (HMD). Possono avere un singolo monitor/proiettore o più monitor/proiettori affiancati.

CAVE: ha da 3 a 6 grandi schermi, mette l'utente in una stanza per un'immersione visiva, ampio campo visivo (FOV), solo un utente può essere tracciato a livello di testa

Wall, Domes: i display a mosaico posizionano le immagini affiancate, quindi necessitano di più proiettori; le immagini devono avere una sovrapposizione per garantire la continuità e richiedono una calibrazione; tuttavia, la sovrapposizione di due proiettori comporta una discontinuità di intensità (immagini più luminose nelle aree di sovrapposizione); i proiettori devono modulare le intensità per attenuare la luce sui pixel sovrapposti; costi elevati

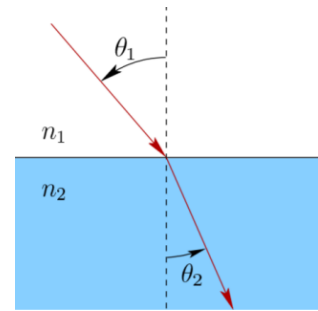
LENSES

Le lenti piegano i raggi di luce in modo che si formi un'immagine a fuoco. Ricorda che la velocità della luce in un mezzo è inferiore alla velocità c nel vuoto. Per un dato materiale, il suo indice di rifrazione è definito come $n = \frac{c}{s}$, in cui s è la velocità della luce nel mezzo. Ad esempio, $n = 2$ significa che la luce impiega il doppio del tempo per attraversare il mezzo rispetto a quanto impiega nel vuoto. Per alcuni esempi comuni, $n = 1.000293$ per l'aria, $n = 1.33$ per l'acqua e $n = 1.523$ per il vetro.

Le lenti funzionano grazie alla legge di Snell, che esprime quanto i raggi di luce si piegano quando entrano o escono da un materiale trasparente $n_1 \sin \theta_1 = n_2 \sin \theta_2$

Di solito, n_1/n_2 e θ_1 sono dati, in modo che $\theta_2 = \sin^{-1} \left(\frac{n_1 \sin \theta_1}{n_2} \right)$

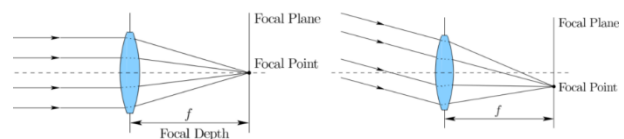
Se $n_1 < n_2$, allora θ_2 è più vicino alla perpendicolare rispetto a θ_1 . Il raggio incidente, il raggio rifratto e la normale alla superficie dell'interfaccia del materiale in cui il raggio è incidente si trovano tutti nello stesso piano.



Caso ideale: una sorgente che si trova a una distanza infinitamente grande.

Lente convessa semplice: la superficie della lente è curvata sfericamente in modo che i raggi di luce paralleli in arrivo convergano a un punto dall'altro lato della lente. Questo punto speciale di convergenza è chiamato punto focale. La sua distanza dal centro della lente è chiamata profondità focale o lunghezza focale.

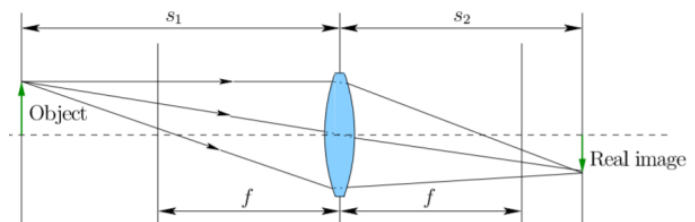
Se i raggi sono paralleli ma non perpendicolari al piano della lente, allora il punto focale si sposta di conseguenza. In questo caso, il punto



focale non si trova sull'asse ottico. Ci sono due gradi di libertà (DOF) delle direzioni dei raggi in arrivo, che portano a un piano focale che contiene tutti i punti focali.

In questo contesto idealizzato, si forma un'immagine reale nel piano dell'immagine, come se fosse uno schermo di proiezione che mostra come appare il mondo di fronte alla lente

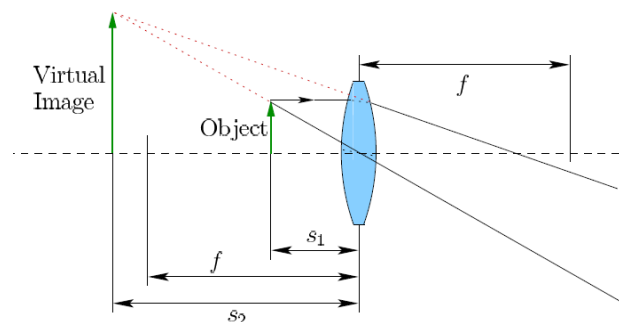
(assumendo che tutto nel mondo sia molto lontano). Nel mondo reale, un oggetto non è infinitamente lontano. Se i raggi non sono paralleli, potrebbe comunque essere possibile metterli a fuoco in un'immagine reale.



Supponiamo di avere una lente con lunghezza focale f . Se la sorgente luminosa è posizionata a una distanza s_1 dalla lente, allora i raggi provenienti da essa saranno a fuoco se e solo se la seguente equazione è soddisfatta (derivata dalla legge di Snell): $\frac{1}{s_1} + \frac{1}{s_2} = \frac{1}{f}$

E se l'oggetto osservato non è completamente piatto e giace in un piano perpendicolare alla lente? In questo caso, non esiste un singolo piano dietro la lente che possa mettere a fuoco l'intero oggetto. Dobbiamo tollerare il fatto che la maggior parte di esso sarà approssimativamente a fuoco. Sfortunatamente, questa è la situazione che si incontra quasi sempre nel mondo reale, compreso il fuoco fornito dai nostri occhi. Quando un oggetto si trova nel punto focale ($s_1 = f$), i raggi in uscita sarebbero paralleli ($s_2 = \infty$) e non si forma alcuna immagine.

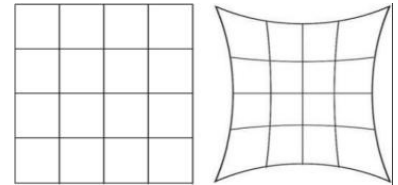
Se l'oggetto è molto vicino alla lente, allora la lente non può costringere i suoi raggi di luce in uscita a convergere in un punto focale. In questo caso, però, appare un'immagine virtuale e la lente funziona come una lente d'ingrandimento.



OPTICAL ABERRATIONS

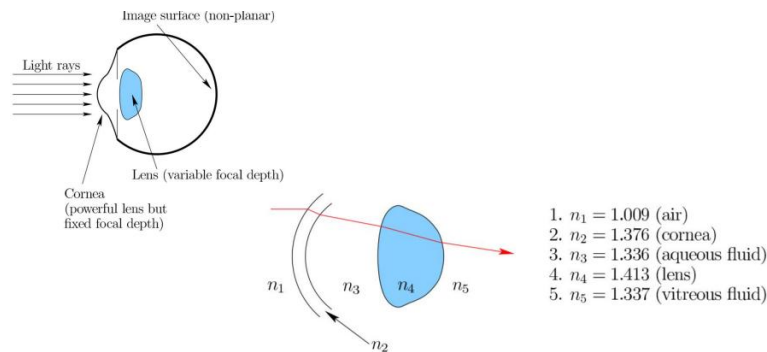
L'aberrazione cromatica è un difetto di una lente che non riesce a mettere a fuoco tutti i colori (lunghezze d'onda) nello stesso punto. È causata dalla dispersione: l'indice di rifrazione degli elementi della lente varia in base alla lunghezza d'onda della luce.

Distorsione: un allungamento o una compressione dell'immagine che diventa progressivamente più grave allontanandosi dall'asse ottico.

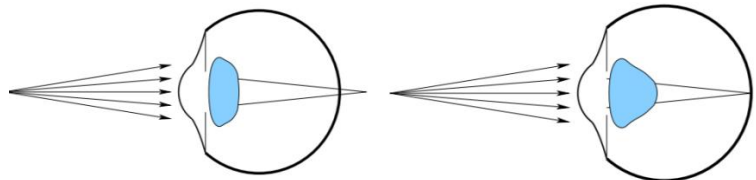


THE HUMAN EYE

L'occhio è considerato parte di un sistema ottico di lenti e immagini. Il piano focale è sostituito da una superficie curva sfericamente chiamata retina. La retina contiene fotorecettori che convertono la luce in impulsi neurali. Gli indici di rifrazione dei materiali lungo il percorso dall'aria esterna alla retina sono diversi.

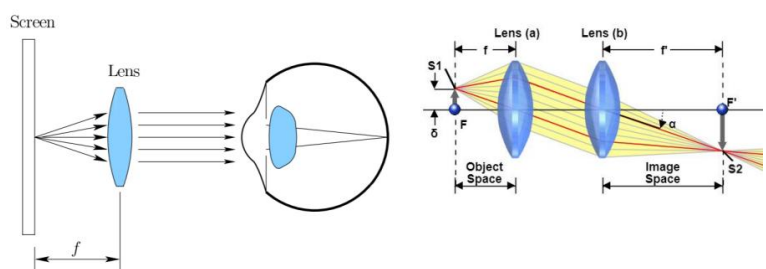


Considerando la legge di Snell, la maggiore curvatura avviene durante la transizione dall'aria alla cornea. Si noti che, una volta che il raggio entra nell'occhio, passa solo attraverso materiali liquidi o solidi. Ci sono effettivamente due lenti convesse: la cornea e il cristallino. Cosa succede quando vogliamo mettere a fuoco un oggetto vicino? Il cristallino cambia forma per mettere a fuoco l'oggetto sulla retina (*accomodazione*). Alla fine, il cervello ruota l'immagine nel verso corretto.



A SIMPLE VR HEADSET MODEL

Supponiamo di costruire un visore VR posizionando uno schermo molto vicino agli occhi. Vogliamo avvicinarlo in modo che riempia la vista dell'utente. Tuttavia, quando un oggetto è posizionato troppo vicino al viso, l'occhio non è in grado di mettere a fuoco correttamente l'oggetto sulla retina. I giovani adulti non riuscirebbero già a metterlo a fuoco se fosse più vicino di 10 cm. Pertanto, è necessaria una lente convessa, di solito posizionata alla distanza della sua profondità focale.



Un dettaglio importante per un visore VR è che ogni lente deve essere centrata perfettamente di fronte alla cornea. Se la distanza tra le due lenti è fissa, allora è impossibile ottenere questo per tutti coloro che utilizzano il visore. La distanza interpupillare, o IPD, è la distanza tra i centri degli occhi umani. La media tra gli esseri umani è di circa 64 mm, ma varia notevolmente in base a razza, sesso e età (nel caso dei bambini). Per poter centrare le lenti per tutti, la distanza tra i centri delle lenti dovrebbe essere regolabile da circa 55 a 75 mm. Devono essere costruite due matrici di visualizzazione con il corretto offset (utilizzando, ad esempio, gluLookAt). Fondamentalmente, vogliamo due posizioni della telecamera, corrispondenti all'occhio sinistro e all'occhio destro. Potrebbe essere necessaria una proiezione non assiale per rendere le sovrapposizioni grafiche percepibilmente corrette per un visore montato sulla testa. Questo crea un frustum asimmetrico per ciascun occhio.

CORRECTING OPTICAL DISTORSIONS

La quantità di distorsione dipende solo dalla distanza dal centro della lente. La distorsione può essere corretta anche in software. La situazione non è perfetta perché, man mano che l'occhio ruota, la pupilla si sposta. La correzione della distorsione prevede due fasi:

- I. Determinare la funzione di distorsione radiale f per un particolare visore.
- II. Calcolare f^{-1} e applicarlo all'immagine renderizzata prima della distorsione della lente.

OCULUS SDK

I seguenti sono i passaggi per il rendering dell'SDK:

1. Inizializza LibOVR tramite ovr_Initialize.
2. Chiama ovr_Create e controlla il valore di ritorno per verificare se ha avuto successo.
3. Integra il tracciamento della testa nel codice di visualizzazione e movimento della tua applicazione:
 1. Ottieni l'orientamento previsto del visore per il frame.
 2. Applica l'orientamento e la posizione del Rift alla vista della telecamera.
 3. Modifica il movimento e il gameplay per considerare l'orientamento della testa.
4. Inizializza il rendering per l'HMD:
 1. Seleziona i parametri di rendering come risoluzione e campo visivo.
 2. Configura il rendering creando una texture di scambio specifica per D3D/OpenGL.
 3. Aggiungi la logica di temporizzazione dei frame nel ciclo di rendering per ottenere correttamente le pose di rendering degli occhi previste.
 4. Renderizza la vista di ciascun occhio su target di rendering intermedi.
 5. Invia il frame renderizzato al visore.
5. Distruggi le risorse create durante lo spegnimento.

Bounding Volumes

Il tema del rilevamento delle collisioni è un argomento ampio che si occupa del seguente problema: rilevare se due (o più) oggetti si intersecano. Il rilevamento delle collisioni è fondamentale in molte applicazioni diverse, tra cui videogiochi, simulazioni basate su

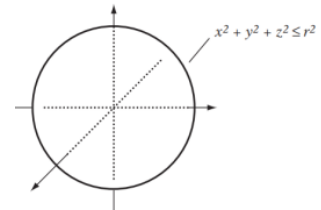
modelli fisici (come l'animazione al computer), robotica, prototipazione virtuale e simulazioni ingegneristiche (per citarne alcune).

La maggior parte dell'hardware attuale utilizza i triangoli come primitiva fondamentale per il rendering. Di conseguenza, una rappresentazione poligonale è una scelta naturale per le scene e gli oggetti della scena, così come per la loro geometria di collisione corrispondente.

Gli oggetti poligonali sono definiti in termini di vertici, spigoli e facce.

Gli oggetti impliciti si riferiscono a sfere, coni, cilindri, ellissoidi, tori e altre primitive geometriche che non sono definite esplicitamente come una rappresentazione poligonale, ma in modo implicito attraverso un'espressione matematica. Gli

oggetti impliciti sono spesso descritti come una funzione che mappa dallo spazio 3D ai numeri reali, $f: \mathbb{R}^3 \rightarrow \mathbb{R}$, dove i punti definiti da $f(x, y, z) < 0$ costituiscono l'interno, $f(x, y, z) = 0$ costituisce il confine, e $f(x, y, z) > 0$ costituisce l'esterno dell'oggetto (Figura).



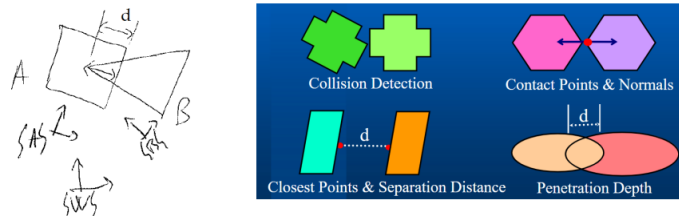
Piuttosto che utilizzare la stessa geometria impiegata per il rendering,

si può sostituire un oggetto con una geometria semplificata di tipo proxy per il rilevamento delle collisioni. Nei videogiochi, ad esempio, è comune affidarsi a forme geometriche semplici come sfere e scatole per rappresentare l'oggetto di gioco, indipendentemente dalla complessità dell'oggetto stesso. Se gli oggetti proxy collidono, si presume che anche gli oggetti reali entrino in collisione. Queste forme geometriche semplici, o volumi di delimitazione, vengono spesso utilizzate per accelerare le interrogazioni di collisione, a prescindere dal tipo di rappresentazione geometrica utilizzata.

TYPES OF QUERIES

Il rilevamento delle collisioni riguarda i problemi di determinare **se, quando e dove** due oggetti entrano in contatto. "Se" implica stabilire un risultato booleano, rispondendo alla domanda se gli oggetti si intersecano o meno. "Quando" deve anche determinare il momento in cui si è verificata la collisione durante un movimento. "Dove" stabilisce in che modo gli oggetti entrano in contatto (chiamato anche "determinazione della collisione").

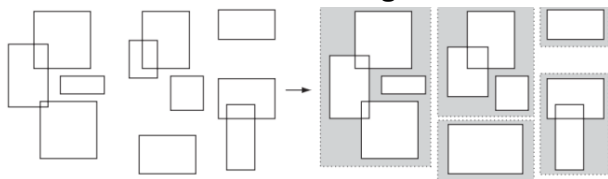
Trovare i punti di intersezione comuni tra gli oggetti (determinazione della collisione) è un problema complesso (a volte si adottano soluzioni approssimative). Se gli oggetti si penetrano, alcune applicazioni richiedono di trovare la profondità di penetrazione, che rappresenta anch'essa un problema difficile. La profondità di penetrazione (d) è solitamente definita in termini di distanza traslazionale minima: la lunghezza del vettore di movimento più breve che separerebbe gli oggetti. Quando la distanza è zero, gli oggetti sono in intersezione.



Un problema più generale è quello di trovare i punti più vicini tra A e B: un punto in A e un punto in B che determinano la distanza di separazione tra gli oggetti. Va notato che i punti più vicini non sono necessariamente unici; ci può essere un numero infinito di punti più vicini.

Poiché ogni oggetto può potenzialmente collidere con qualsiasi altro oggetto, una simulazione con n oggetti richiede $(n-1)+(n-2)+\dots+1 = n(n-1)/2 = O(n^2)$ test a coppie. A causa

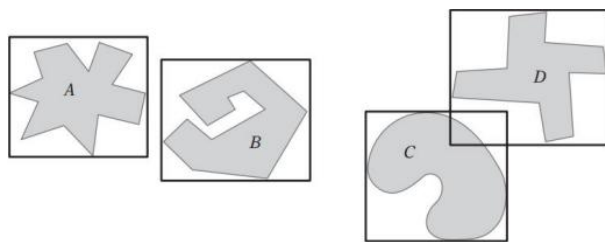
della complessità temporale quadratica, **testare ingenuamente ogni coppia di oggetti per la collisione diventa rapidamente troppo costoso**, anche per valori moderati di n . Ridurre il costo associato al test a coppie influirà solo linearmente sul tempo di esecuzione. **Per accelerare realmente il processo, il numero di coppie testate deve essere ridotto**. Questa riduzione viene effettuata separando la gestione delle collisioni di più oggetti in due fasi: la fase ampia e la fase stretta. La fase **ampia identifica gruppi più piccoli di oggetti che potrebbero collidere**, escludendo rapidamente quelli che sicuramente non lo fanno. La fase stretta riguarda i test a coppie all'interno dei sottogruppi ed è responsabile della **determinazione delle collisioni esatte, se presenti**. La figura sottostante illustra come la fase ampia riduca il carico di lavoro attraverso una strategia di divisione e conquista.



I giochi visivi devono **funzionare a 60 fps**. Questo frame rate lascia **16,7 ms** per preparare ogni fotogramma del gioco. A seconda del tipo di gioco, il rilevamento delle collisioni può rappresentare, ad esempio, dal 10 al 30% di un fotogramma, lasciando a sua volta da 2 a 5 ms per il rilevamento delle collisioni. Per un gioco d'azione a piattaforme che potrebbe avere dozzine di oggetti dipendenti dalle collisioni attivi in un dato momento, potrebbero esserci solo circa 50 a 250 μ s disponibili per gestire la collisione di ciascun oggetto.

BOUNDING VOLUMES

Testare direttamente la geometria di due oggetti per rilevare una collisione è spesso molto costoso, specialmente quando gli oggetti consistono in centinaia o addirittura migliaia di poligoni. Per ridurre questo costo, di solito si testano i volumi di ingombro degli oggetti per verificare eventuali sovrapposizioni prima di eseguire il test di intersezione geometrica. Un volume di ingombro (Bounding Volume, BV) è un singolo volume semplice che racchiude uno o più oggetti di natura più complessa. L'idea è che i volumi più semplici (come scatole o sfere) abbiano test di sovrapposizione più economici rispetto agli oggetti complessi che racchiudono. **Utilizzare volumi di ingombro permette di effettuare rapidamente test di rifiuto di sovrapposizione, poiché si testa la geometria complessa solo quando la prima query di sovrapposizione sui volumi di ingombro dà un risultato positivo**. Naturalmente, quando gli oggetti si sovrappongono davvero, questo test aggiuntivo comporta un aumento del tempo di calcolo. Tuttavia, nella maggior parte delle situazioni, pochi oggetti sono tipicamente abbastanza vicini da far sovrapporre i loro volumi di ingombro. **Pertanto, l'uso dei volumi di ingombro porta generalmente a un notevole guadagno in termini di prestazioni e l'eliminazione di oggetti complessi da ulteriori test giustifica pienamente il piccolo costo aggiuntivo del test sui volumi di ingombro**. Se il numero di poligoni da testare può essere, per esempio, dimezzato, il carico di lavoro si ridurrà del 75%.

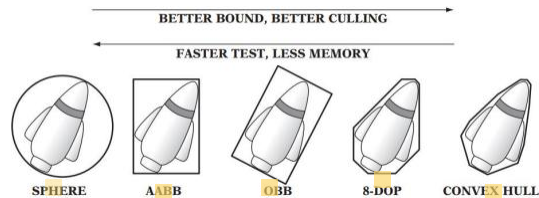


Le **proprietà desiderabili** per i volumi di ingombro includono:

- **Test di intersezione poco costosi**

- Aderenza stretta (ovvero, che si adattino bene all'oggetto)
- Computazione economica
- Facili da ruotare e trasformare
- Utilizzo ridotto di memoria

I volumi di ingombro vengono generalmente calcolati in una fase di pre-elaborazione piuttosto che durante l'esecuzione in tempo reale. I cinque tipi di volumi di ingombro più comuni sono:

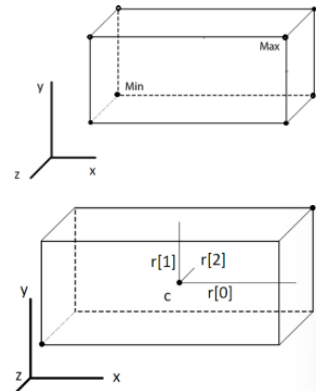


AXIS-ALIGNED BOUNDING BOXES (AABBs)

La scatola rettangolare a sei facce (in 3D, a quattro facce in 2D) è caratterizzata dall'avere le sue facce orientate in modo tale che le normali delle facce siano sempre parallele agli assi del sistema di coordinate dato.

```
struct AABB {
    Point min;
    Point max;
};
```

```
struct AABB {
    Point c; // center point of AABB
    float r[3]; // radius of halfwidth extents (rx, ry, rz)
};
```



AABB-AABB INTERSECTION

Due AABB (Axis-Aligned Bounding Boxes) si sovrappongono solo se si sovrappongono su tutti e tre gli assi. Per la rappresentazione min-max:

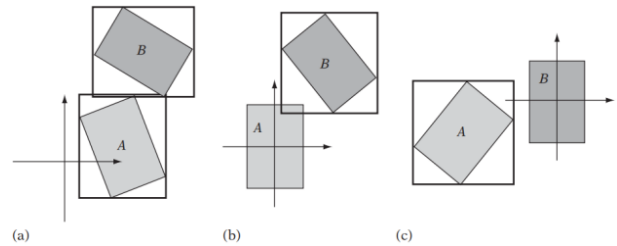
```
int TestAABBAABB(AABB a, AABB b)
{
    // Exit with no intersection if separated along an axis
    if (a.max[0] < b.min[0] || a.min[0] > b.max[0]) return 0;
    if (a.max[1] < b.min[1] || a.min[1] > b.max[1]) return 0;
    if (a.max[2] < b.min[2] || a.min[2] > b.max[2]) return 0;
    // Overlapping on all axes means AABBs are intersecting
    return 1;
}
```

-se ci si trova a sinistra o a destra dell'asse x
- se ci si trova sotto o sopra asse y
-o davanti o dietro asse z
return 1 se ci sono sovrapposizioni

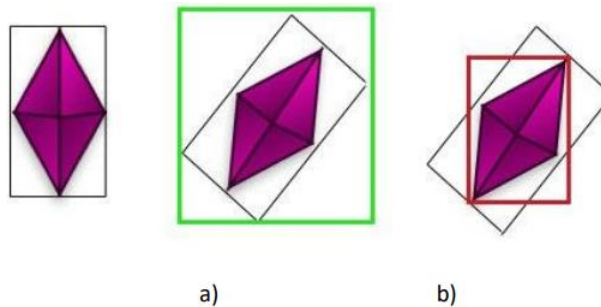
COMPUTING AND UPDATING AABBs

L'AABB di un oggetto può essere calcolato scorrendo tutti i punti della geometria dell'oggetto e memorizzando i valori minimi e massimi di x, y, z. I punti finali minimi e massimi (x, y, z) definiscono l'AABB. I volumi di bounding sono solitamente specificati nello spazio locale del modello degli oggetti che racchiudono. Per eseguire una query di sovrapposizione tra due volumi di bounding, i volumi devono essere trasformati in un sistema di coordinate comune. La scelta è tra trasformare entrambi i volumi di bounding nello spazio mondo o trasformare

un volume di bounding nello spazio locale dell'altro. Un vantaggio della trasformazione nello spazio locale è che riduce il lavoro di trasformazione a metà rispetto alla trasformazione nello spazio mondo. Spesso porta anche a un volume di bounding più stretto rispetto alla trasformazione nello spazio mondo. La figura illustra il concetto. Gli AABB ricalcolati degli oggetti A e B si sovrappongono nello spazio mondo (a). Tuttavia, nello spazio dell'oggetto B, si scopre che gli oggetti sono separati (c).



Aggiornare l'AABB dopo che è stata applicata una trasformazione all'oggetto può essere ottenuto in due modi:



- AABB ricomputato da AABB ruotato. Avvolgere l'AABB ruotato stesso in un nuovo AABB. Questo produce un AABB approssimato piuttosto che stretto, poiché l'AABB risultante è più grande di quello con cui si è iniziato. È importante che l'AABB approssimato venga calcolato da una rotazione dell'AABB originale nello spazio locale. In caso contrario, il ricompito ripetuto dall'AABB ruotato del passo temporale precedente farebbe crescere indefinitamente l'AABB.
- Scorrere ogni vertice trasformato dell'oggetto, calcolare la sua posizione nel mondo e trovare le coordinate del vertice minimo/massimo, quindi ricostruire l'AABB.

AABB RECOMPUTED FROM ROTATED AABB (CASE “a” ABOVE)

Considera un'assegnazione di una bounding box A influenzata da una matrice di rotazione M, risultando in una bounding box orientata A' con una certa orientazione. Supponiamo che A sia data usando la rappresentazione min-max. La bounding box allineata agli assi B che racchiude A' è specificata dagli intervalli di estensione formati dalla proiezione degli otto vertici ruotati di A' sugli assi delle coordinate del mondo. Per esempio, per le estensioni x di B, contribuiscono solo le componenti x dei vettori colonna di M. Pertanto, trovare le estensioni corrisponde a trovare i vertici che producono i prodotti minimi e massimi con le righe di M. Ogni vertice di B è una combinazione di tre valori minimi o massimi trasformati di A. Il valore di estensione minimo è la somma dei termini più piccoli, e l'estensione massima è la somma dei termini più grandi. La traslazione non influisce sul calcolo della dimensione della nuova bounding box e può essere semplicemente aggiunta. Ad esempio, l'estensione massima lungo l'asse x può essere calcolata come:

$$\begin{aligned}
 B.\text{max}[0] = & \max(m[0][0] * A.\text{min}[0], m[0][0] * A.\text{max}[0]) \\
 & + \max(m[0][1] * A.\text{min}[1], m[0][1] * A.\text{max}[1]) \\
 & + \max(m[0][2] * A.\text{min}[2], m[0][2] * A.\text{max}[2]) + t[0];
 \end{aligned}$$

Calcolare una bounding box che racchiuda una AABB ruotata utilizzando la rappresentazione min-max può quindi essere implementato nel seguente modo:

```

// Transform AABB a by the matrix m and translation t
// find maximum extents, and store result into AABB b.
void UpdateAABB(AABB a, float m[3][3], float t[3], AABB &b)
{
    // For all three axes
    for (int i = 0; i < 3; i++) {
        // Start by adding in translation
        b.min[i] = t[i];
        b.max[i] = t[i];
        // Form extent by summing smaller and larger terms respectively

        for (int j = 0; j < 3; j++) {
            float e = m[i][j] * a.min[j];
            float f = m[i][j] * a.max[j];
            if (e < f) {
                b.min[i] += e;
                b.max[i] += f;
            } else {
                b.min[i] += f;
                b.max[i] += e;
            }
        }
    }
}

```

SPHERES

Come le AABB, le sfere hanno un test di intersezione poco costoso. Le sfere hanno anche il vantaggio di essere invarianti rispetto alla rotazione, il che significa che sono facili da trasformare: devono semplicemente essere traslate nella loro nuova posizione.

```

struct Sphere {
    Point c; // Sphere center
    float r; // Sphere radius
};

```

SPHERE-SPHERE INTERSECTION

La distanza euclidea tra i centri delle sfere viene calcolata e confrontata con la somma dei raggi delle sfere. Per evitare un'operazione di radice quadrata spesso costosa, si confrontano le distanze al quadrato.

```

int TestSphereSphere(Sphere a, Sphere b)
{
    // Calculate squared distance between centers
    Vector d;
    d = a.c - b.c; float dist2 = dot(d,d) -> prodotto scalare del vettore distanza
    float radiusSum = a.r + b.r;
    return dist2 <= radiusSum * radiusSum; se dist2 è < le sfere si intersecano
}

```

si calcola la distanza al quadrato per evitare di fare le radici quadrate

Calcolare una sfera di delimitazione ottimale non è semplice come calcolare una scatola di delimitazione allineata agli assi.

1. Una sfera di delimitazione approssimativa semplice può essere ottenuta calcolando prima l'AABB di tutti i punti. Il punto medio dell'AABB viene quindi selezionato come centro della sfera, e il raggio della sfera è impostato sulla distanza dal punto più lontano da questo centro.

- Un approccio alternativo per calcolare una sfera di delimitazione approssimativa semplice è descritto in Ritter, Jack. "An Efficient Bounding Sphere," in Andrew Glassner (ed.), Graphics Gems, Academic Press, pp. 301–303, 1990. Questo algoritmo cerca di trovare una buona sfera di delimitazione quasi ottimale e poi in pochi passaggi la migliora fino a delimitare tutti i punti. L'algoritmo si sviluppa in due passaggi. Nel primo passaggio, vengono trovati sei punti estremali (non necessariamente unici) lungo gli assi del sistema di coordinate. Tra questi sei punti, viene selezionata la coppia di punti più lontani. (Nota che questi due punti non corrispondono necessariamente ai punti che definiscono il lato più lungo dell'AABB del gruppo di punti.) Il centro della sfera viene ora selezionato come punto medio tra questi due punti, e il raggio è impostato sulla metà della distanza tra di essi. Nel secondo passaggio, tutti i punti vengono esaminati di nuovo. Per tutti i punti al di fuori della sfera corrente, la sfera viene aggiornata per essere quella che racchiude la sfera vecchia e il punto esterno.

```
void RitterSphere(Sphere &s, Point pt[], int numPts)
{
    // Get sphere encompassing two approximately most distant points
    SphereFromDistantPoint(s, pt, numPts);    andrà a disegnare una sfera andando a
    // Grow sphere to include all points        racchiudere i punti più distanti
    for (int i = 0; i < numPts; i++)
        SphereOfSphereAndPt(s, pt[i]);      si va a controllare che ogni punto dell'array è
                                                nella sfera, se il punto non dovesse essere
                                                dentro la sfera si allargherà
}
```

ORIENTED BOUNDING BOXES (OBBs)

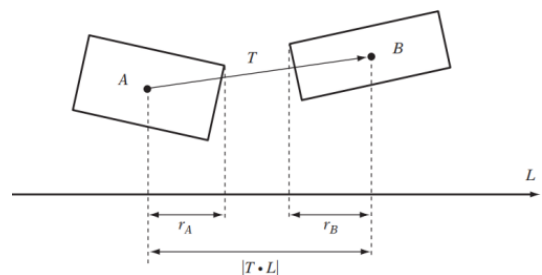
Una scatola di delimitazione orientata (OBB) è un blocco rettangolare, simile a un AABB, ma con un'orientazione arbitraria. Rappresentazione: un punto centrale più una matrice di orientamento e tre lunghezze dei semi-lati.

```
struct OBB {
    Point c; // OBB center point
    Vector u[3]; // Local x-, y-, and z-axes
    Vector e; // Positive halfwidth extents of OBB along axis
};
```

OBB-OBB INTERSECTION

A differenza dei precedenti test di intersezione dei volumi di delimitazione, il test per la sovrapposizione tra due scatole di delimitazione orientate è sorprendentemente complicato.

Un test esatto per l'intersezione OBB-OBB può essere implementato in termini di quello che è conosciuto come il test degli assi separatori. Due OBB sono separate se, rispetto a un certo asse L , la somma dei loro raggi proiettati è inferiore alla distanza tra le proiezioni dei loro punti centrali (come illustrato in figura). Cioè, se $|T \cdot L| > r_A + r_B$.



Per le OBB è possibile dimostrare che al massimo 15 di questi assi separatori devono essere testati per determinare correttamente lo stato di sovrapposizione delle OBB. Questi assi corrispondono ai tre assi coordinati di A, ai tre assi coordinati di B e ai nove assi perpendicolari a un asse di ciascuna. Se le scatole non sovrappongono su nessuno dei 15

assi, non stanno intersecando. Se nessun asse fornisce questo risultato anticipato, ne consegue che le scatole devono essere sovrapposte.

```
int TestOBB(OBB &a, OBB &b)
```

UPDATING AND COMPUTING A TIGHT OBB

Calcolare scatole di delimitazione orientate a misura è un problema difficile. Un modo per effettuare un adattamento brute-force è parametrizzare l'orientamento dell'OBB in qualche modo. Lo spazio delle orientazioni viene campionato a intervalli regolari sulla parametrizzazione e viene mantenuto il miglior OBB tra tutte le rotazioni campionate. Aggiornare un OBB mentre l'oggetto si muove è facile perché l'OBB ruota con l'oggetto; pertanto, è sufficiente applicare all'OBB la trasformazione dell'oggetto senza ricostruire l'OBB. Al contrario, con le AABB dobbiamo ricostruire la struttura dati dopo ogni trasformazione dell'oggetto.

Primitive Tests

Dopo che un sistema di alto livello ha escluso il maggior numero possibile di oggetti dai test di collisione successivi, tutti i sistemi di collisione devono eseguire test di basso livello tra primitive o volumi di delimitazione per determinare lo stato di intersezione.

CLOSEST POINT COMPUTATION

Le query sui punti più vicini sono tra le più potenti per quanto riguarda le collisioni. Date le posizioni dei punti più vicini tra due oggetti, è possibile ottenere la distanza tra di essi.

CLOSEST POINT ON PLANE TO POINT

Un piano nello spazio 3D può essere considerato come una superficie piatta che si estende indefinitamente in tutte le direzioni. Può essere descritto da una normale e da un punto sul piano. Quando un piano è precomputato, è spesso utile che la normale del piano sia un vettore unitario. Avere una normale unitari semplifica la maggior parte delle operazioni che coinvolgono il piano.

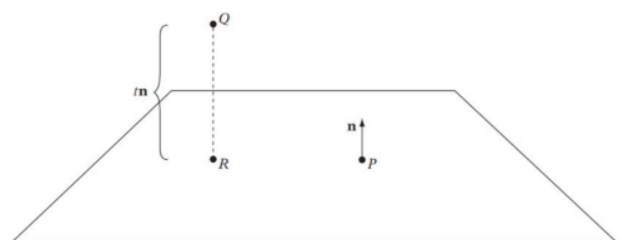
```
struct Plane {
    Vector n; // Plane normal. Points x on the plane satisfy Dot(n,x) = d
    float d; // d = dot(n,p) for a given point p on the plane
};
```

Data una superficie π , definita da un punto P e una normale n , tutti i punti X sul piano soddisfano l'equazione $n \cdot (X - P) = 0$ (cioè, il vettore da P a X è perpendicolare a n). Questa espressione può quindi essere scritta come $n \cdot X = d$. Quando n è unitario, $-d$ equivale alla distanza SEGNALE del piano dall'origine.

Ora, sia Q un punto arbitrario nello spazio. Il punto più vicino R sul piano a Q è la proiezione ortogonale di Q sul piano, ottenuta spostando Q perpendicolarmente (rispetto a n) verso il piano. Cioè, $R = Q - tn$ per qualche valore di t , come illustrato nella figura.

$$n \cdot ((Q - tn) - P) = 0 \Leftrightarrow (\text{inserendo } R \text{ per } X \text{ nell'equazione del piano})$$

$$n \cdot Q - t(n \cdot n) - n \cdot P = 0 \Leftrightarrow (\text{espandendo il prodotto scalare})$$



$n \cdot (Q - P) = t(n \cdot n) \Leftrightarrow$ (raggruppando i termini simili e spostando l'espressione di t a destra)

$t = n \cdot (Q - P) / (n \cdot n)$ (dividendo entrambi i lati per $n \cdot n$)

Sostituendo questa espressione per t in $R = Q - tn$ si ottiene il punto di proiezione R come

$R = Q - (n \cdot (Q - P) / (n \cdot n))n$

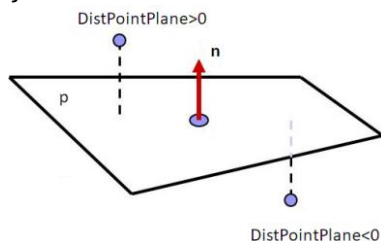
Se t è positivo, Q è davanti al piano (e se è negativo, Q è dietro il piano). Quando il piano è dato nella forma a quattro componenti $n \cdot X = d$, l'espressione corrispondente per t è

$t = ((n \cdot Q) - d) / (n \cdot n)$

Il codice per calcolare il punto più vicino sul piano a un punto e la distanza segnata diventa:

```
Point ClosestPtPointPlane(Point q, Plane p)  ti restituisce il punto più vicino al piano
{
    float t = (Dot(p.n, q) - p.d) / Dot(p.n, p.n);    - Dot(p.n, q) punto q proiettato sulla normale
                                                         del piano
    return q - t * p.n;                                - p.d distanza del piano dall'origine
                                                         - Dot(p.n, p.n) rapporto di scala perchè dot mi
                                                         amplifica e mi fa spostare troppo nello spazio
} punto più vicino sul piano

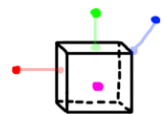
float DistPointPlane(Point q, Plane p)
{
    // return Dot(q, p.n) - p.d; if plane equation normalized (||p.n||==1)
    Return (Dot(p.n, q) - p.d) / sqrt(Dot(p.n, p.n));  mi dirà se la distanza è positiva o negativa ->
                                                         mi dirà se il punto è davanti o dietro al piano
}
```



CLOSET POINT ON AABB TO POINT

Sia B un box delimitante allineato agli assi e P un punto arbitrario nello spazio. Il punto Q su (o dentro) B più vicino a P si ottiene bloccando P ai limiti di B su base componente per componente.

```
void ClosetPtPointAABB(Point p, AABB b, Point &q)  trovare il punto dell'AABB più vicino a p
{
    // For each coordinate axis, if the point coordinate value is
    // outside box, clamp it to the box, else keep it as is
    for (int i = 0; i < 3; i++) {
        float v = p[i];
        if (v < b.min[i]) v = b.min[i]; // v = max(v, b.min[i])
        if (v > b.max[i]) v = b.max[i]; // v = min(v, b.max[i])
        q[i] = v;
    }
} - trovo il punto minimo (sulla la scatola)
  - trovo il punto massimo (sulla la scatola)
```

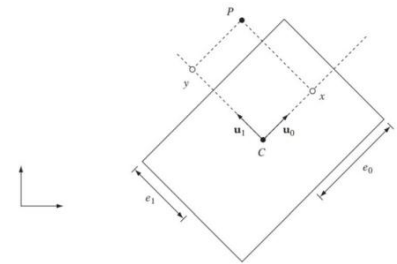


Distanza del punto dall'AABB: per semplificare il calcolo e per evitare una costosa chiamata alla radice quadrata, la distanza quadrata viene calcolata come segue:

```
float SqDistPointAABB(Point p, AABB b)
{
    // mi calcola la distanza fra punto e scatola
    float sqDist = 0.0f;
    for (int i = 0; i < 3; i++) {
        // For each axis count any excess distance outside box extents
        float v = p[i];
        if (v < b.min[i]) sqDist += (b.min[i] - v) * (b.min[i] - v);
        if (v > b.max[i]) sqDist += (v - b.max[i]) * (v - b.max[i]);
    }
    return sqDist;
}
```

CLOSEST POINT ON OBB TO POINT

Sia B un OBB dato da un punto centrale C. Un punto P nello spazio mondiale si relaziona al suo punto corrispondente Q = (x,y,z) nel sistema di coordinate dell'OBB B come
 $P = C + xu_0 + yu_1 + zu_2$.



Per calcolare il punto R su (o dentro) B più vicino a P, si può applicare lo stesso approccio usato per un AABB, esprimendo P nel sistema di coordinate dell'OBB come Q, bloccando Q ai limiti e0, e1 ed e2, e riesprimendo Q nelle coordinate mondiali. Il codice per questo è il seguente:

```
void ClosestPtPointOBB(Point p, OBB b, Point &q) // mi restituisce il punto
{
    // dell'OBB piu vicino a p
    Vector d = p - b.c;
    // Start result at center of box; make steps from there
    q = b.c;
    // For each OBB axis...
    for (int i = 0; i < 3; i++) {
        // ...project d onto that axis to get the distance
        // along the axis of d from the box center
        float dist = Dot(d, b.u[i]);
        // If distance farther than the box extents, clamp to the box
        if (dist > b.e[i]) dist = b.e[i];
        if (dist < -b.e[i]) dist = -b.e[i];
        // Step that distance along the axis to get world coordinate
        q += dist * b.u[i];
    }
}
```

Se è necessaria solo la distanza al quadrato e non il punto più vicino, questo codice può essere ulteriormente semplificato. Proiettando il vettore v dal centro di B a P su ciascuno dei tre assi dell'OBB, si ottiene la distanza d da P al centro della scatola lungo quell'asse. Poiché gli assi sono ortogonali, qualsiasi eccesso che d supera il limite della scatola per un dato asse può essere calcolato, elevato al quadrato e aggiunto alla distanza totale al quadrato di P in modo indipendente dagli altri due assi.

```

float SqDistPointOBB(Point p, OBB b)
{
    Vector v = p - b.c; // centro della scatola
    float sqDist = 0.0f;
    for (int i = 0; i < 3; i++) {
        // Project vector from box center to p on each axis, getting the distance
        // of p along that axis, and count any excess distance outside box extents
        float d = Dot(v, b.u[i]), excess = 0.0f; // prodotto scalare del vettore proiettato sull'asse
        if (d < -b.e[i]) // excess serve per accumulare l'eventuale distanza fuori da 0
            excess = d + b.e[i];
        else if (d > b.e[i])
            excess = d - b.e[i];
        sqDist += excess * excess;
    }
    return sqDist;
}

```

TESTING PRIMITIVES

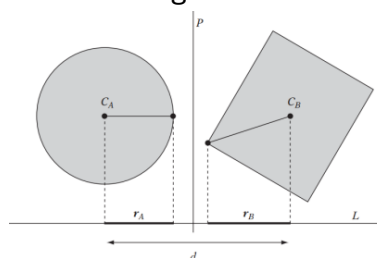
Il test delle primitive è meno generale rispetto al calcolo della distanza tra di esse. In generale, un test indicherà solo se le primitive si stanno intersecando, senza determinare dove o come avviene l'intersezione. Pertanto, questi test di intersezione sono spesso molto più veloci rispetto ai test che restituiscono informazioni aggiuntive.

Uno strumento estremamente utile per implementare vari test di intersezione è il test dell'asse di separazione. Questo test afferma che, date due insiemi convessi A e B, o i due insiemi si intersecano, o esiste un piano di separazione P tale che A si trova da un lato di P e B dall'altro.

Data un piano P che separa A e B, un asse di separazione è una linea L perpendicolare a P. Si chiama asse di separazione perché le proiezioni ortogonali di A e B su L risultano in due intervalli non sovrapposti.

Per eseguire un test dell'asse di separazione, è utile notare che molte primitive di collisione - come segmenti, AABBs, OBBs e sfere - sono simmetriche nel senso che hanno un punto centrale C, che proietta sempre al centro dell'intervallo di proiezione della loro proiezione su un asse. Dato un potenziale asse di separazione L, un test di separazione efficiente di due oggetti simmetrici A e B consiste quindi in:

1. calcolare la larghezza metà, o i raggi, dei loro intervalli di proiezione e confrontare la loro somma con la distanza tra le proiezioni centrali.
2. se la somma è inferiore alla distanza tra le proiezioni centrali, gli oggetti devono essere disgiunti. Un esempio è fornito nella figura.



si prende un asse di riferimento L e si proiettano i due oggetti, si calcolano i raggi delle due proiezioni e si fa la somma. Si calcola infine la distanza fra i centri di queste due proiezioni, se la somma dei raggi dovesse essere minore della distanza dei centri allora significa che le proiezioni non si sovrappongono

Innanzitutto, per ogni oggetto, si ottiene un punto di supporto lungo L; cioè, un punto più distante dal centro dell'oggetto in entrambe le direzioni di L (poiché le proiezioni sono simmetriche, la direzione non ha importanza; ci sarà un punto equidistante in entrambe le direzioni). I due raggi degli oggetti, r_a e r_b , vengono quindi ottenuti calcolando la distanza tra le

proiezioni su L dei centri degli oggetti e i loro rispettivi punti più distanti. Viene anche calcolata la distanza d tra le proiezioni dei centri. Date queste quantità calcolate, gli oggetti sono ora separati se $r_a + r_b < d$.

Il test dell'asse di separazione si applica anche quando un oggetto non è simmetrico - come nel caso di un involucro convesso arbitrario, ad esempio - ma il test deve essere modificato per proiettare i vertici dell'involucro sull'asse al fine di trovare gli intervalli di proiezione. Per i poliedri convessi, è possibile ridurre drasticamente il numero di assi da testare. Per testare la separabilità di due oggetti poliedrici, devono essere testati i seguenti assi:

- Assi paralleli alle normali delle facce dell'oggetto A
- Assi paralleli alle normali delle facce dell'oggetto B
- Assi paralleli ai vettori risultanti dai prodotti incrociati di tutti i lati in A con tutti i lati in B

Non appena viene trovato un asse di separazione, il test può immediatamente uscire con “nessuna intersezione”. Se tutti gli assi vengono testati senza trovare un asse di separazione, gli oggetti devono essere in intersezione. Quando due poliedri collidono, il test dell'asse di separazione può anche assistere nel calcolo delle informazioni di contatto. Invece di uscire precocemente quando viene rilevata una sovrapposizione su un asse, vengono testati tutti gli assi per la sovrapposizione. Dopo che tutti gli assi sono stati testati, l'asse con la sovrapposizione (normalizzata) minima può essere utilizzato come normale di contatto, e la sovrapposizione può essere utilizzata per stimare la profondità di penetrazione.

TESTING SPHERE AGAINST PLANE

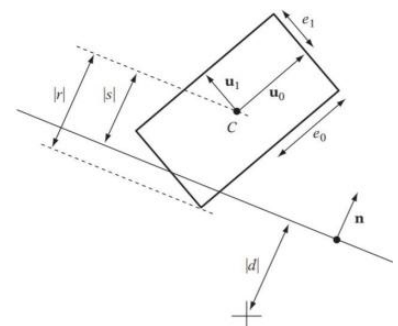
Sia una sfera S specificata da una posizione centrale C e un raggio r , e sia un piano π specificato da $(n \cdot X) = d$, dove n è un vettore unitario; cioè, $\|n\| = 1$. Per determinare se la sfera è intersecata dal piano, l'equazione del piano può essere valutata per il centro della sfera. Poiché n è unitario, il valore risultante corrisponde alla distanza segnata del centro della sfera dal piano. Se il valore assoluto della distanza è all'interno del raggio della sfera, il piano interseca la sfera:

```
int TestSpherePlane(Sphere s, Plane p) verifica se una sfera interseca un piano
{
    // For a normalized plane ( $|p.n| = 1$ ), evaluating the plane equation
    // for a point gives the signed distance of the point to the plane
    float dist = Dot(p.n, s.c) - p.d; prodotto scalare del centro della sfera e la normale del piano meno
    // If sphere center within +/- radius from plane, plane intersects sphere
    Return fabs(dist) <= s.r; fabs è il modulo -> se questa distanza è inferiore al
    raggio vuol dire che c'è intersezione
}
```

TESTING BOX AGAINST PLANE

Sia un piano P dato da $(n \cdot X) = d$. Per verificare se una scatola B interseca P , si può utilizzare il test degli assi separatori. Qui, è necessario testare solo l'asse parallelo alla normale del piano n .

Consideriamo prima il caso in cui B sia un OBB, rappresentato dalla consueta rappresentazione di un centro C ; assi locali u_0 , u_1 e u_2 ; e tre scalari e_0 , e_1 ed e_2 (che rendono i lati dell'OBB larghi



$2 e_i$ per $0 \leq i \leq 2$). I punti R nell'OBB sono dati da $R = C \pm a_0 u_0 \pm a_1 u_1 \pm a_2 u_2$, dove $|a_i| \leq e_i$. Allo stesso modo, gli otto vertici V_i , $0 \leq i \leq 7$, dell'OBB sono dati da

$V_i = C \pm e_0 u_0 \pm e_1 u_1 \pm e_2 u_2$. Sia una retta L parallela a n che passa per il centro della scatola, $L(t) = C + t n$.

Il centro della scatola C proietta su L a $t = 0$. Poiché l'OBB è simmetrico rispetto al suo centro, la proiezione su L risulta in un intervallo di proiezione simmetrico $[C - r n, C + r n]$, centrato in C, con una metà larghezza (o raggio) di r. Verificare se B interseca P ora equivale a calcolare il raggio r dell'intervallo di proiezione e controllare se la distanza del punto centrale di B dal piano P è minore di r.

Poiché i punti dell'OBB più lontani dal suo centro sono i vertici dell'OBB, il raggio di proiezione massimo complessivo su qualsiasi vettore n sarà realizzato da uno dei vertici. Pertanto, è sufficiente considerare solo questi nel calcolare il raggio r. I raggi di proiezione r_i degli otto vertici sono dati da $r_i = (V_i - C) \cdot n = (C \pm e_0 u_0 \pm e_1 u_1 \pm e_2 u_2 - C) \cdot n = (\pm e_0 u_0 \pm e_1 u_1 \pm e_2 u_2) \cdot n$.

Il raggio massimo positivo r si ottiene quando tutti i termini coinvolti sono positivi:

$$r = e_0 |u_0 \cdot n| + e_1 |u_1 \cdot n| + e_2 |u_2 \cdot n|.$$

La distanza firmata s di C da P si ottiene valutando l'equazione del piano per C, dando $s = n \cdot C - d$.

```
int TestOBBPlane(OBB b, Plane p) {    oBB interseca un piano
    // Compute the projection interval radius of b onto L(t) = b.c + t * p.n
    float r = b.e[0]*fabs(Dot(p.n, b.u[0])) +      // il prodotto scalare mi dirà solamente la
    b.e[1]*fabs(Dot(p.n, b.u[1])) +                // direzione della scatola e si andrà a
    b.e[2]*fabs(Dot(p.n, b.u[2]));                 // moltiplicare con b.e per ottenere la
    // Compute distance of box center from plane    // grandezza della scatola -> raggio massimo
    float s = Dot(p.n, b.c) - p.d;
    // Intersection occurs when distance s falls within [-r,+r] interval
    return fabs(s) <= r;}
```

Per una scatola allineata agli assi B, gli assi locali u_0 , u_1 e u_2 sono noti in anticipo, e quindi il codice può essere semplificato di conseguenza.

```
int TestAABBPlane(AABB b, Plane p)
{
    // These two lines not necessary with a (center, extents) AABB representation
    Point c = (b.max + b.min) * 0.5f; // Compute AABB center
    Vector e = b.max - c; // Compute positive extents
    // Compute the projection interval radius of b onto L(t) = b.c + t * p.n
    float r = e[0]*fabs(p.n[0]) + e[1]*fabs(p.n[1]) + e[2]*fabs(p.n[2]);
    // Compute distance of box center from plane
    float s = Dot(p.n, c) - p.d;
    // Intersection occurs when distance s falls within [-r,+r] interval
    return fabs(s) <= r;
}
```

TESTING SPHERE AGAINST AABB

Verificare se una sfera interseca un'asse allineata alla scatola (AABB) si fa meglio calcolando la distanza tra il centro della sfera e il punto più vicino sull'AABB, e confrontando questa distanza con il raggio della sfera. Se la distanza è inferiore al raggio, la sfera e l'AABB devono intersecarsi. Per evitare operazioni di radice quadrata costose, sia la distanza che il raggio possono essere elevati al quadrato prima del confronto, senza modificare il risultato del test.

```

int TestSphereAABB(Sphere s, AABB b, Point &p)
{
    // Find point p on AABB closest to sphere center
    ClosestPtPointAABB(s.c, b, p);
    // Sphere and AABB intersect if the (squared) distance from sphere
    // center to point p is less than the (squared) sphere radius
    Vector v = p - s.c;
    return Dot(v, v) <= s.r * s.r;
}

```

TESTING SPHERE AGAINST OBB

Verificare una sfera contro una scatola di delimitazione orientata è quasi identico a verificare contro una scatola di delimitazione allineata agli assi.

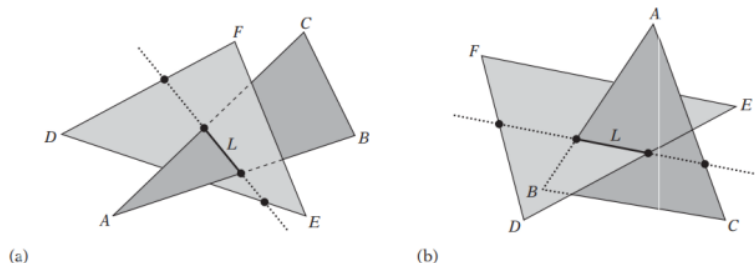
```

int TestSphereOBB(Sphere s, OBB b, Point &p)
{
    // Find point p on OBB closest to sphere center
    ClosestPtPointOBB(s.c, b, p);
    // Sphere and OBB intersect if the (squared) distance from sphere
    // center to point p is less than the (squared) sphere radius
    Vector v = p - s.c;
    return Dot(v, v) <= s.r * s.r;    verificchi il prodotto scalare e il raggio al quadrato
}

```

TESTING TRIANGLE AGAINST TRIANGLE

In generale, quando due triangoli si intersecano, o due spigoli di un triangolo attraversano l'interno dell'altro, oppure uno spigolo di ciascun triangolo attraversa l'interno dell'altro triangolo.



Möller, Tomas. "Un Test Veloce di Intersezione Triangolo-Triangolo," Journal of Graphics Tools, vol. 2, no. 2, pp. 25–30, 1997.

Pseudo codice:

1. Calcola l'equazione del piano del triangolo 1. Esci senza intersezione se i vertici del triangolo 2 sono tutti dallo stesso lato di questo piano.
2. Calcola l'equazione del piano del triangolo 2. Esci senza intersezione se i vertici del triangolo 1 sono tutti dallo stesso lato di questo piano.
3. Calcola la linea L di intersezione tra i due piani, $L(t) = P + td$, dove $d = n_1 \times n_2$ è il prodotto vettoriale delle normali dei due triangoli n_1 e n_2 .
4. Determina quale asse di coordinata principale è più parallelo con la linea di intersezione L.
5. Calcola gli intervalli di intersezione scalari per ciascun triangolo con L, proiettati su questo asse principale.
6. I triangoli si intersecano se gli intervalli di intersezione si sovrappongono; altrimenti, non si intersecano.

I test che coinvolgono linee, raggi e segmenti sono frequentemente utilizzati, ad esempio, per simulare proiettili sparati o per verificare la linea di vista. I test sulle linee sono talvolta usati al posto di query più complicate. Ad esempio, il contatto della mano o del piede di un personaggio giocante con l'ambiente, o il contatto di una ruota di un veicolo con il suolo, possono spesso essere modellati in modo efficiente con un semplice test lineare.

INTERSECTING SEGMENT AGAINST PLANE

Sia dato un piano P definito da $(n \cdot X) = d$ e un segmento descritto dall'equazione parametrica $S(t) = A + t(B - A)$ per $0 \leq t \leq 1$. Il valore di t dell'intersezione del segmento con il piano si ottiene sostituendo l'equazione parametrica nell'equazione del piano e risolvendo per t :

$$(n \cdot (A + t(B - A))) = d \text{ (sostituendo } S(t) = A + t(B - A) \text{ per } X \text{ nell'equazione } (n \cdot X) = d)$$

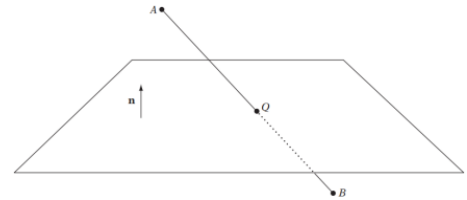
$$n \cdot A + t n \cdot (B - A) = d \text{ (espandendo il prodotto scalare)}$$

$$t n \cdot (B - A) = d - n \cdot A \text{ (spostando il termine scalare a destra)}$$

$$t = (d - n \cdot A) / (n \cdot (B - A)) \text{ (dividendo entrambi i lati per } n \cdot (B - A) \text{ per isolare)}$$

```
int IntersectSegmentPlane(Point a, Point b, Plane p, float &t, Point &q)
{
    // Compute the t value for the directed line ab intersecting the plane
    Vector ab = b - a;
    t = (p.d - Dot(p.n, a)) / Dot(p.n, ab); se Dot(p.n, ab) = 0 allora il segmento è parallelo al piano
    // If t in [0..1] compute and return intersection point
    if (t >= 0.0f && t <= 1.0f) {
        q = a + t * ab;
        return 1;
    }
    // Else no intersection
    return 0;
}
```

t -> posizione di intersezione (numero)
q -> coordinata di intersezione (x,y,z)



INTERSECTING RAY AGAINST SPHERE

Sia data una retta R definita da $R(t) = P + td$, con $t \geq 0$, dove P è l'origine della retta e d è un vettore direzione normalizzato, $\|d\| = 1$. Il confine della sfera è definito da $(X - C) \cdot (X - C) = r^2$, dove C è il centro della sfera e r il suo raggio. Per trovare il valore di t in cui la retta interseca la superficie della sfera, si sostituisce $R(t)$ per X , ottenendo $(P + td - C) \cdot (P + td - C) = r^2$.

Sia $m = P - C$, quindi: $(m + td) \cdot (m + td) = r^2 \Leftrightarrow$ (sostituendo $m = P - C$)

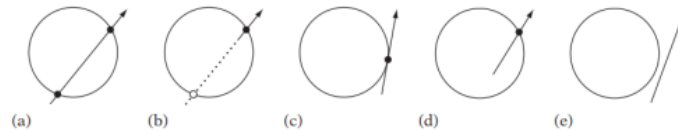
$$(d \cdot d)t^2 + 2(m \cdot d)t + (m \cdot m) = r^2 \Leftrightarrow \text{(espandendo il prodotto scalare)}$$

$$t^2 + 2(m \cdot d)t + (m \cdot m) - r^2 = 0 \text{ (semplificando } d \cdot d = 1; \text{ forma canonica per l'equazione quadratica)}$$

Questa è un'equazione quadratica in t . Per la formula quadratica, le soluzioni sono date da $t = -b \pm \sqrt{b^2 - c}$.

Qui, $b = m \cdot d$ e $c = (m \cdot m) - r^2$. Risolvendo l'equazione quadratica si hanno tre possibili esiti, classificati dal discriminante $d = b^2 - c$. Se $d < 0$, non ci sono radici reali, il che corrisponde a una mancata intersezione della retta con la sfera. Se $d = 0$, c'è una radice reale (doppia), corrispondente a una intersezione tangenziale della retta con la sfera. Se $d > 0$, ci sono due radici reali e la retta interseca la sfera due volte: una entrando e una uscendo dal confine della sfera. In quest'ultimo caso, il valore di t dell'intersezione minore è quello rilevante, dato

da $t = -b - \sqrt{b^2 - c}$. Tuttavia, è importante distinguere il caso di falsa intersezione in cui la retta parte dall'esterno della sfera e punta lontano da essa, risultando in un valore di intersezione $t < 0$. Questo caso è illustrato in figura, insieme a tutte le altre relazioni di intersezione retta-sfera.

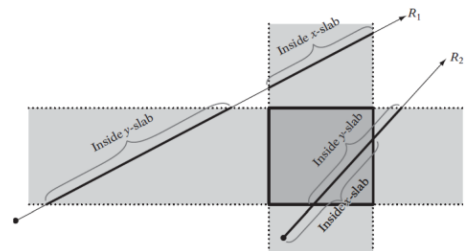


```
// Intersects ray  $r = p + td$ ,  $|d| = 1$ , with sphere  $s$  and, if intersecting,
// returns  $t$  value of intersection and intersection point  $q$ 
int IntersectRaySphere(Point p, Vector d, Sphere s, float &t, Point &q)
{
    Vector m = p - s.c;           // vettore d -> direzione raggio
    float b = Dot(m, d);           // in che direzione si muove il raggio
    float c = Dot(m, m) - s.r * s.r; // determina origine del raggio
    // Exit if r's origin outside s ( $c > 0$ ) and r pointing away from s ( $b > 0$ )
    if ( $c > 0.0f$  &&  $b > 0.0f$ ) return 0;
    float discr = b*b - c;         // calcolo delta
    // A negative discriminant corresponds to ray missing sphere
    if ( $discr < 0.0f$ ) return 0;
    // Ray found to intersect sphere, compute smallest  $t$  value of intersection
     $t = -b - \sqrt{discr}$ ;           // un raggio attraversa la sfera in due punti uno di entrata e uno di
    // uscita e la radice mi permette di calcolare il valore di  $t$  più piccolo
    // If  $t$  is negative, ray started inside sphere so clamp  $t$  to zero
    if ( $t < 0.0f$ )  $t = 0.0f$ ;
     $q = p + t * d$ ;
    return 1;}

```

INTERSECTING RAYAGAINST AABB

Un test per verificare l'intersezione di un raggio con un AABB deve solo calcolare gli intervalli di intersezione del raggio con i piani delle "lastre" (una lastra è definita come lo spazio infinito tra due piani paralleli) e svolgere alcune operazioni di confronto per mantenere l'intersezione logica di tutti gli intervalli di intersezione lungo il raggio. Un



esempio di intersezione e non intersezione di un raggio con un box 2D, formato dall'intersezione di una lastra sull'asse x e una sull'asse y , è illustrato in figura.

Tutto ciò che è necessario è tenere traccia del punto di ingresso più lontano in una lastra e dell'uscita più vicina da una lastra. Se l'ingresso più lontano diventa mai più distante dell'uscita più vicina, il raggio non può intersecare il volume d'intersezione delle lastre e il test può terminare anticipatamente (con un risultato di non intersezione).

Gli intervalli di intersezione del raggio con le lastre sono ottenuti inserendo l'equazione parametrica del raggio, $R(t) = P + t * d$, nelle equazioni dei piani delle lastre, $X \cdot n_i = l_i$, e risolvendo per t , ottenendo $t = (l_i - P \cdot n_i) / (d \cdot n_i)$. Per un AABB, due componenti della normale sono pari a zero e, dato $P = (p_x, p_y, p_z)$ e $d = (d_x, d_y, d_z)$, l'espressione si semplifica, ad esempio, in $t = (l_i - p_x) / d_x$ per un piano perpendicolare all'asse x . Per evitare una divisione per zero quando il raggio è parallelo a una lastra, questo caso è meglio gestirlo separatamente verificando se l'origine del raggio è contenuta nella lastra.

```

// Intersect ray  $R(t) = p + t \cdot d$  against AABB a. When intersecting,
// return intersection distance tmin and point q of intersection
int IntersectRayAABB(Point p, Vector d, AABB a, float &tmin, Point &q)
{
    tmin -> primo punto di intersezione
    tmin = 0.0f;
    float tmax = 1000000;
    // set to max distance ray can travel (for segment)
    // For all three slabs
    for (int i = 0; i < 3; i++) {
        if (fabs(d[i]) < EPSILON) { costante piccolissima diversa da zero per evitare errori
            // Ray is parallel to slab. No hit if origin not within slab
            if (p[i] < a.min[i] || p[i] > a.max[i]) return 0;
        } else {
            // Compute intersection t value of ray with near and far plane of slab
            float ood = 1.0f / d[i];
            float t1 = (a.min[i] - p[i]) * ood;
            float t2 = (a.max[i] - p[i]) * ood;
            float tswap;
            // Make t1 be intersection with near plane, t2 with far plane
            if (t1 > t2) {
                tswap=t2;
                t2=t1;
                t1=tswap;
            }
            // Compute the intersection of slab intersection intervals
            if (t1 > tmin) tmin = t1;
            if (t2 < tmax) tmax = t2;
            // Exit with no collision as soon as slab intersection becomes empty
            if (tmin > tmax) return 0;
        }
    }
    // Ray intersects all 3 slabs. Return point (q) and intersection t value (tmin)
    q = p + d * tmin;
    return 1;
}

```

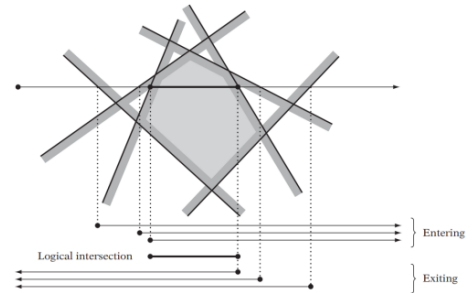
INTERSECTING SEGMENT AGAINST CONVEX POLYHEDRON

Per intersecare un segmento $S(t)=A+t(B-A)$, con $0 \leq t \leq 1$, contro un poliedro convesso, risulta conveniente rappresentare il poliedro come l'intersezione di un insieme di semispazi. In questo formato, il segmento interseca il poliedro se esiste un valore di t , compreso tra 0 e 1, per il quale $S(t)$ si trova all'interno di tutti i semispazi. Un modo efficiente per determinare se questo è il caso è semplicemente ritagliare il segmento contro ciascun semispazio, eliminando la parte del segmento che si trova al di fuori del semispazio. Se durante questo processo di ritaglio il segmento diventa di lunghezza zero, il segmento non può intersecare il poliedro e il processo può terminare con il risultato di "nessuna intersezione". Dopo che il segmento è stato intersecato contro tutti i semispazi, ciò che rimane del segmento deve essere l'intersezione tra il segmento e il poliedro. Il processo di ritaglio può essere implementato tenendo traccia del segmento definito sull'intervallo $t_{first} \leq t \leq t_{last}$, dove t_{first} e t_{last} sono inizialmente impostati su 0 e 1 rispettivamente, ma vengono aggiustati man mano che il segmento viene intersecato con i piani che definiscono i semispazi (con la normale rivolta verso l'esterno). Ora, ricorda che per un segmento $S(t)=A+t(B-A)$, con $0 \leq t \leq 1$, e un piano

$n \cdot X = d$, l'intersezione del segmento con il piano è data da $t = (d - n \cdot A) / (n \cdot (B - A))$.

Il piano fronteggia il segmento se il denominatore $n \cdot (B - A)$ è negativo, e se il denominatore è positivo, il piano è rivolto dall'altra parte rispetto al segmento. Quando il denominatore è zero, il segmento è parallelo al piano. Quando il piano fronteggia il segmento, il valore di t corrisponde al punto in cui il segmento entra nel semispazio. Di conseguenza, se t è maggiore di t_{first} , il segmento deve essere ritagliato impostando t_{first} al valore corrente di t . Quando il piano è rivolto dalla parte opposta rispetto al segmento, t corrisponde al punto in cui il segmento esce dal semispazio. Pertanto, se t è minore di t_{last} , t_{last} deve essere impostato al valore corrente di t . Se in qualsiasi momento $t_{last} < t_{first}$, il segmento è scomparso e non c'è intersezione.

Il caso in cui il segmento è parallelo al piano deve essere gestito come un caso speciale. Se il segmento si trova al di fuori del semispazio, parallelo al piano definente, non può intersecare il poliedro e il test può terminare immediatamente con "nessuna intersezione". Se il segmento si trova all'interno del semispazio, il piano può semplicemente essere ignorato e il processo può continuare con il piano successivo. Per determinare se il segmento si trova all'interno o all'esterno del semispazio, uno qualsiasi dei due estremi del segmento può essere testato contro il piano inserendolo nell'equazione del piano: il segno del risultato determinerà su quale lato del piano si trova il punto. La figura fornisce un'illustrazione in 2D di come un raggio viene ritagliato contro diversi semispazi e di come l'intersezione logica del raggio ritagliato contro ciascun semispazio forma la parte del raggio che si sovrappone al volume di intersezione.



```
int IntersectSegmentPolyhedron(Point a, Point b, Plane p[], int n, float &tfirst,
float &tlast)
    n -> numero di piani
{
    // Compute direction vector for the segment
    Vector d = b - a;
    // Set initial interval to being the whole segment.
    tfirst = 0.0f;
    tlast = 1.0f;
    // Intersect segment against each plane
    for (int i = 0; i < n; i++) {
        float denom = Dot(p[i].n, d); // ti dice come si muove il segmento sulla normale (entra esce parallelo)
        float dist = p[i].d - Dot(p[i].n, a);
        // Test if segment runs parallel to the plane
        if (denom < EPSILON) {
            // If so, return "no intersection" if segment lies outside plane
            if (dist < 0.0f) return 0;
        } else {
            // Compute parameterized t value for intersection with current plane
            float t = dist / denom;
            if (denom < 0.0f) {
                // When entering halfspace, update tfirst if t is larger
                if (t > tfirst) tfirst = t;
            } else {
                // When exiting halfspace, update tlast if t is smaller
                if (t < tlast) tlast = t;
            }
        }
    }
}
```

```

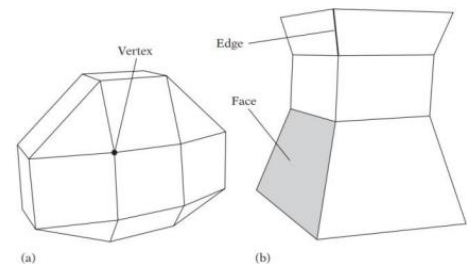
    // Exit with "no intersection" if intersection becomes empty
    if (tfirst > tlast) return 0;
}
}
// A nonzero logical intersection, so the segment intersects the polyhedron
return 1;
}

```

CONVEXITY-BASED METHODS

Gli oggetti convessi hanno alcune proprietà che li rendono particolarmente adatti per i test di rilevamento delle collisioni. Una proprietà importante è l'esistenza di un piano separatore per oggetti convessi che non si intersecano. I poliedri non convessi possono essere scomposti in pezzi convessi, permettendo l'utilizzo di algoritmi efficienti per il rilevamento delle collisioni. Un poliedro è una regione limitata e connessa dello spazio tridimensionale con la forma di un solido multifaccettato.

Il confine di un poliedro è costituito da un certo numero di facce poligonali (piatte) collegate in modo che ciascun lato del poligono faccia parte esattamente di due facce (Figura). Un poliedro P è convesso se e solo se, per tutte le facce di P, tutti i vertici di P si trovano (strettamente) sullo stesso lato di quella faccia.



algoritmo di rilevamento basato sui confini:
 - uno spigolo della scatola p tocca una faccia della scatola q
 - uno spigolo della scatola q buca una faccia della scatola p

BOUNDARY-BASED COLLISION DETECTION

Siano dati due poliedri convessi P e Q in una forma che permetta un facile accesso ai loro vertici e facce (e quindi ai loro spigoli). Assumendo che i poliedri si trovino nello stesso sistema di riferimento (ottenuto, ad esempio, trasformando Q nello spazio di P), la loro intersezione può essere determinata come segue:

1. Intersecare ciascun spigolo di P con il poliedro convesso Q. Se uno spigolo si trova parzialmente o completamente all'interno di Q, fermarsi e segnalare che i poliedri si intersecano.
2. Inversamente, intersecare ciascun spigolo di Q con il poliedro convesso P. Se uno spigolo si trova parzialmente o completamente all'interno di P, fermarsi e segnalare che P e Q si intersecano.
3. Altrimenti, restituire "nessuna intersezione".

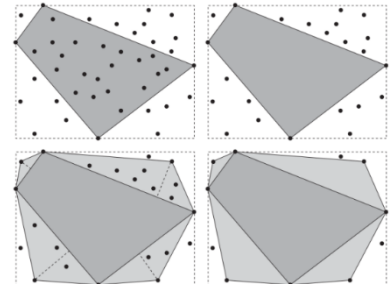
COMPUTING CONVEX HULLS

L'involuppo convesso di un oggetto SSS, denotato come $CH(S)CH(S)CH(S)$, è il più piccolo poliedro che contiene completamente SSS.

Algoritmo Quickhull

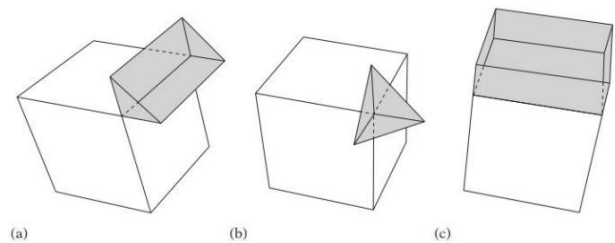
1. Viene calcolato il bounding **box** dell'insieme di punti, e vengono individuati i quattro punti estremi del set (che si trovano su ciascuno dei quattro lati del bounding box). Poiché questi quattro punti sono estremi, devono trovarsi sull'involuppo convesso e formano quindi una prima "approssimazione" dell'involuppo convesso sotto forma di quadrilatero.

2. Qualsiasi punto che si trova all'interno di questo quadrilatero chiaramente non può appartenere all'involuppo convesso dell'insieme di punti e può essere eliminato dalle considerazioni successive.
3. Per ogni lato del quadrilatero, viene individuato il punto al di fuori del lato che è più distante da esso (se presente). Poiché ciascun punto è estremo nella direzione perpendicolare al lato, esso deve trovarsi sull'involuppo.
4. Insieme ai due estremi del lato, questi nuovi punti formano una regione triangolare. Come prima, i punti che si trovano all'interno di questa regione non possono far parte dell'involuppo e possono essere scartati.
5. La procedura viene ripetuta per ogni nuovo lato aggiunto all'involuppo, terminando la ricorsione quando non ci sono più punti all'esterno dei lati.



VORONOI REGIONS

Dato un poliedro PPP, una caratteristica di P è uno dei suoi vertici, spigoli o facce. La regione di Voronoi di una caratteristica F di P è l'insieme di punti nello spazio più vicini (o altrettanto vicini) a F rispetto a qualsiasi altra caratteristica di P. La distanza di un punto p da



una caratteristica F è la distanza di p dal suo punto più vicino q sulla caratteristica F. Tre tipi di regioni di Voronoi relative a una caratteristica sono illustrati in figura.

Dato un poliedro convesso P tutti i punti nello spazio esterno a P possono essere classificati come appartenenti alla regione di Voronoi di un vertice, uno spigolo o una faccia di P, con il confine tra due regioni di Voronoi contigue considerato parte di una sola delle regioni. Poiché le regioni di Voronoi creano una suddivisione dello spazio esterno a un poliedro, esse possono essere utilizzate, ad esempio, per determinare il punto più vicino su un oggetto poliedrico convesso a un punto Q nello spazio. Questa determinazione viene effettuata spostandosi da una regione all'altra fino a quando Q non si trova all'interno della regione. Il punto più vicino sull'oggetto a Q è quindi la proiezione di Q sulla caratteristica associata alla regione considerata.

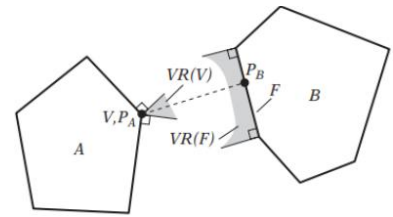
THE V-CIP ALGORITHM

Una coppia di caratteristiche, una da ciascuno dei due poliedri disgiunti, si dice essere le caratteristiche più vicine se contengono una coppia di punti più vicini per i poliedri.

L'algoritmo **V-Clip** opera su una coppia di poliedri convessi ed è basato sul seguente teorema, che definisce i punti più vicini tra la coppia in termini delle caratteristiche più vicine dei poliedri.

Teorema: Siano F_A e F_B una coppia di caratteristiche di due poliedri convessi disgiunti A e B, e siano $VR(F_A)$ e $VR(F_B)$ le rispettive regioni di Voronoi. Siano $P_A \in F_A$ e $P_B \in F_B$ una coppia di punti più vicini tra le caratteristiche. Allora, se $P_A \in VR(F_B)$ e $P_B \in VR(F_A)$ F_A e F_B sono una coppia di caratteristiche globalmente più vicine e P_A e P_B sono una coppia di punti più vicini

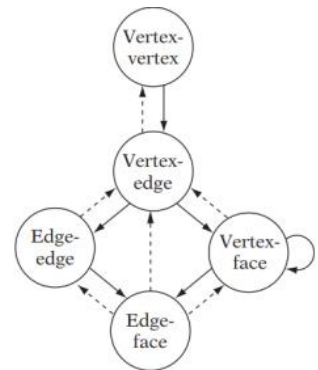
globalmente tra A e B (anche se non necessariamente unica). La figura illustra una coppia di poliedri (2D) che soddisfano le condizioni di questo teorema.



V-Clip inizia con due caratteristiche, una da ciascun poliedro in ingresso. A ogni iterazione dell'algoritmo, le caratteristiche vengono testate per vedere se soddisfano le condizioni del teorema per essere la coppia più vicina di caratteristiche. Se sì, l'algoritmo termina, restituendo un risultato di non intersezione insieme alla coppia di caratteristiche più vicine. Se le condizioni non sono soddisfatte, V-Clip aggiorna una delle caratteristiche a una caratteristica vicina, dove i vicini di una caratteristica sono definiti come segue:

- I vicini di un *vertice* sono gli spigoli incidenti sul vertice.
- I vicini di una *faccia* sono gli spigoli che delimitano la faccia.
- I vicini di uno *spigolo* sono i due vertici e le due facce incidenti sullo spigolo.

La figura mostra un diagramma delle possibili coppie di caratteristiche e delle transizioni legali tra di esse, secondo le definizioni di vicinato elencate sopra. Le frecce solide nel diagramma indicano che la distanza intercaratteristica diminuisce rigorosamente con una transizione. Le frecce tratteggiate indicano che non avviene alcun cambiamento nella distanza. Poiché il diagramma non contiene un ciclo di transizione costituito solo da frecce tratteggiate, è possibile eseguire solo un numero finito di transizioni prima che la distanza tra le caratteristiche diminuisca rigorosamente. Inoltre, non ci sono transizioni che aumentano la distanza intercaratteristica. Pertanto, la convergenza è garantita e l'algoritmo termina eventualmente con la coppia di caratteristiche più vicine (e quindi, implicitamente, con la coppia di punti più vicini) o viene segnalata l'intersezione. Il modo in cui le caratteristiche vengono aggiornate con l'algoritmo V-Clip permette di gestire efficacemente la penetrazione tra i poliedri. Se una caratteristica di un poliedro si trova all'interno di un altro poliedro, una serie di aggiornamenti porterà eventualmente alla condizione in cui uno spigolo di un poliedro avrà i suoi estremi su lati opposti di una faccia dell'altro poliedro. A questo punto, viene segnalata la penetrazione.

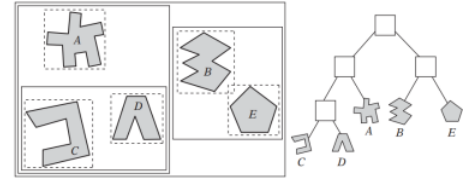


Bounding Volume Hierarchies

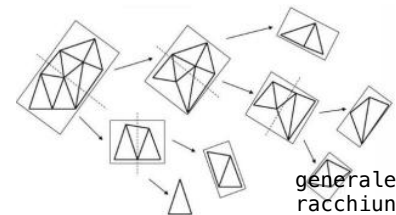
Racchiudere oggetti all'interno di volumi di delimitazione e svolgere test su questi volumi prima di testare la geometria degli oggetti stessi può portare a significativi miglioramenti delle prestazioni. Tuttavia, anche se i test vengono semplificati, il numero di test a coppie rimane lo stesso. Organizzando i volumi di delimitazione in una gerarchia ad albero chiamata *bounding volume hierarchy* (BVH), la complessità temporale può essere ridotta in modo logaritmico rispetto al numero di test eseguiti. Il set originale di volumi di delimitazione forma i nodi foglia dell'albero che costituisce questa gerarchia. Questi nodi vengono poi raggruppati in piccoli insiemi e racchiusi in volumi di delimitazione più grandi. A loro volta, questi gruppi vengono ulteriormente raggruppati e racchiusi in volumi di delimitazione ancora più grandi, in modo ricorsivo, fino a ottenere una struttura ad albero con un singolo volume di delimitazione al vertice dell'albero.

Una volta che la gerarchia è costruita, durante i test di collisione non è necessario esaminare i figli se il volume del nodo genitore non viene intersecato. Due o più volumi in una BVH possono coprire lo stesso spazio. È importante notare che il volume di delimitazione di un nodo genitore non deve necessariamente racchiudere i volumi di delimitazione dei suoi nodi figli.

Le *bounding volume hierarchies* possono essere utilizzate per accelerare sia la fase di analisi generale (*broad-phase*) che la fase di analisi dettagliata (*narrow-phase*). Come metodo di *broad-phase*, le BVH vengono costruite per contenere gli oggetti in una scena e per eliminare gruppi di oggetti distanti (quando il volume di delimitazione dei gruppi non interseca il volume di delimitazione dell'oggetto di riferimento), come illustrato nella figura.



Come metodo di *narrow-phase*, le BVH vengono costruite sulle parti di un oggetto (spesso complesso), consentendo che i test a coppie tra primitive siano limitati alle parti degli oggetti i cui volumi di delimitazione si sovrappongono, come illustrato nella figura.



generale: le BVH racchiudono gli oggetti della scena per scartare rapidamente i gruppi di elementi distanti
 dettagliata: le BVH vengono costruite sulle singole parti dell'oggetto complesso quindi i test si limitano a quelle aree dove i volumi si sovrappongono

Caratteristiche desiderate per una *Bounding Volume Hierarchy* (BVH):

- I nodi contenuti in un dato sottoalbero dovrebbero essere vicini tra loro.
- Ogni nodo nella gerarchia dovrebbe avere un volume minimo.
- Il volume di sovrapposizione dei nodi fratelli dovrebbe essere minimo.
- La gerarchia dovrebbe essere bilanciata sia nella sua struttura nodale che nel contenuto. Il bilanciamento consente di potare quanta più parte possibile della gerarchia quando un ramo non viene attraversato.
- La gerarchia può essere generata automaticamente senza l'intervento dell'utente.

Una formula dei costi che esprime il costo totale di intersezione tra due gerarchie è:

$$T = N_V C_V + N_P C_P + N_U C_U + C_O$$

Dove:

- N_V è il numero di coppie di volumi di bounding (*Bounding Volumes*, BV) testate per sovrapposizione,
- C_V è il costo di testare una coppia di BV per sovrapposizione,
- N_P è il numero di coppie di primitivi testate,
- C_P è il costo di testare una coppia di primitivi,
- N_U è il numero di nodi che devono essere aggiornati,
- C_U è il costo di aggiornare ciascun nodo (ad esempio la trasformazione delle coordinate di un AABB nel sistema di riferimento di un altro AABB),
- C_O è il costo di un'elaborazione una tantum (come una trasformazione di coordinate tra oggetti).

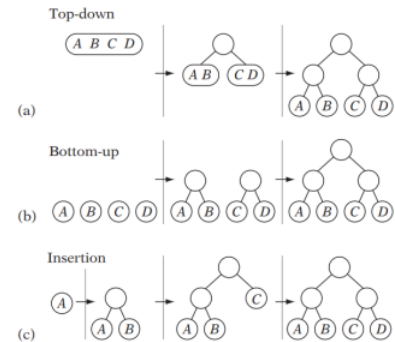
Gli alberi binari sono di gran lunga la rappresentazione gerarchica più comune. Le foglie possono contenere interi oggetti o singoli poligoni. Per scene dinamiche, una BVH può essere ricostruita ad ogni frame (o dopo k frame), oppure i volumi di bounding dei nodi interni possono essere riadattati senza cambiare la gerarchia. Purtroppo, il riadattamento può degradare la qualità dell'albero nel tempo.

BUILDING STRATEGIES FOR HIERARCHY CONSTRUCTION

Ci sono tre categorie principali di metodi di costruzione degli alberi: metodi dall'alto verso il basso, metodi dal basso verso l'alto e metodi di inserimento. I metodi dall'alto verso il basso (o divisivi) procedono partizionando l'insieme di input in due (o più) sottoinsiemi, racchiudendoli nel volume di bounding scelto, e poi ricorsivamente sui sottoinsiemi racchiusi. Grazie alla facilità con cui possono essere implementati, i metodi dall'alto verso il basso sono di gran lunga i più popolari.

I metodi dal basso verso l'alto (o agglomerativi) iniziano con l'insieme di input come foglie dell'albero e poi raggruppano due o più di esse per formare un nuovo nodo (interno), procedendo allo stesso modo fino a quando tutto è stato raggruppato sotto un singolo nodo (la radice dell'albero). Anche se i metodi dal basso verso l'alto tendono a produrre alberi migliori rispetto agli altri metodi, sono anche più difficili da implementare.

I metodi di inserimento costruiscono la gerarchia in modo incrementale inserendo gli oggetti uno alla volta nell'albero. La posizione di inserimento è scelta in modo da minimizzare una qualche misura di costo per l'albero risultante. I metodi di inserimento sono considerati metodi on-line, mentre sia i metodi dall'alto verso il basso che quelli dal basso verso l'alto sono considerati metodi off-line in quanto richiedono che tutte le primitive siano disponibili prima dell'inizio della costruzione. Un vantaggio dei metodi on-line è che consentono aggiornamenti da eseguire durante il runtime.



TOP-DOWN CONSTRUCTION

procedura ricorsiva. Inizia racchiudendo l'insieme di input di primitive (o oggetti) in un volume di bounding. Queste primitive vengono quindi partizionate in due sottoinsiemi. La procedura viene ora chiamata ricorsivamente per formare subgerarchie per i due sottoinsiemi, che vengono poi collegati come figli al volume genitore. La ricorsione si ferma quando l'insieme di input è composto da una singola primitiva, a questo punto la procedura restituisce semplicemente dopo aver creato il volume di bounding per la primitiva. Il seguente frammento di codice illustra come può essere implementata la costruzione dall'alto verso il basso.

```
// Construct a top-down tree. Rearranges object[] array during construction
void TopDownBVTree(Node **tree, Object object[ ], int numObjects)
{
    // nodo con doppio riferimento per non perdere persistenza
    assert(numObjects > 0);
    const int MIN_OBJECTS_PER_LEAF = 1;
    Node *pNode = new Node;
    *tree = pNode;
    // Compute a bounding volume for object[0], ..., object[numObjects - 1]
    pNode->BV = ComputeBoundingVolume(&object[0], numObjects);
    if (numObjects <= MIN_OBJECTS_PER_LEAF) {
        pNode->type = LEAF;
        pNode->numObjects = numObjects;
        pNode->object = &object[0]; // Pointer to first object in leaf
    } else {
        pNode->type = NODE;
        // Based on some partitioning strategy, arrange objects into
        // two partitions: object[0..k-1], and object[k..numObjects-1]
```

```

int k = PartitionObjects(&object[0], numObjects); k punto di divisione
// Recursively construct left and right subtree from subarrays and
// point the left and right fields of the current node at the subtrees
TopDownBVTree(&(pNode->left), &object[0], k);
TopDownBVTree(&(pNode->right), &object[k], numObjects - k);
}
}

```

Per semplificare la partizione, l'insieme viene solitamente diviso in sottoinsiemi utilizzando un iperpiano di divisione. Poiché non è possibile garantire la selezione di un piano di divisione che non attraversi nessuna primitiva, è necessario affrontare qualsiasi primitiva che attraversi il piano durante la partizione dell'insieme. Una soluzione consiste nel suddividere la primitiva in due, assegnando le parti ai rispettivi sottoinsiemi. La suddivisione delle primitive consente ai volumi di bounding figli di essere più piccoli, minimizzando la loro sovrapposizione, possibilmente eliminandola completamente. Uno svantaggio della suddivisione delle primitive che attraversano il piano è che qualsiasi primitiva suddivisa può nuovamente essere soggetta a suddivisione, dando potenzialmente origine a un enorme aumento di primitive. Una soluzione forse più comune è non suddividere la primitiva, ma lasciare che la posizione del suo baricentro rispetto al piano di divisione determini in quale sottoinsieme vada. Utilizzare il baricentro per determinare a quale sottoinsieme assegnare la primitiva cerca di minimizzare la sovrapposizione tra i volumi di bounding fraterni. La scelta di un piano di partizione viene solitamente suddivisa in due fasi. Prima viene selezionato un asse e poi una posizione lungo questo asse. Tra un numero infinito di possibili assi, in qualche modo deve essere selezionato un singolo asse come asse di partizione. Teoricamente, è possibile applicare un metodo di ottimizzazione iterativa (ad esempio, hill climbing) per individuare il miglior asse possibile. Praticamente, tale approccio è solitamente troppo costoso anche per una fase di preprocessing. Di conseguenza, la ricerca deve essere limitata a un numero ristretto di assi da cui viene scelto il migliore. Le scelte comuni di assi

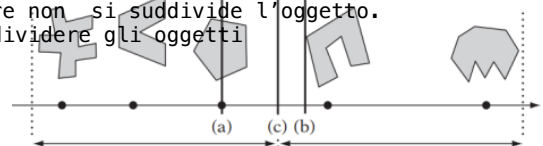
da includere sono:

- in caso di divisione nascono due problemi:
o si suddivide l'oggetto ma comporta una suddivisione
ricorsiva successivamente oppure non si suddivide l'oggetto.
Esistono svariati assi per suddividere gli oggetti
1. Assi locali x, y e z.
 2. Assi del volume di bounding genitore. Anche

se la gerarchia è costruita da, ad esempio, sfere (che non hanno assi associati

apparenti), un OBB temporaneo potrebbe comunque essere calcolato per il set di dati del genitore, da cui verrebbero estratti i candidati agli assi di suddivisione.

3. Asse attraverso i due punti più distanti.
4. Asse lungo il quale la varianza è maggiore. La suddivisione lungo la dimensione in cui i dati di input hanno la maggiore dispersione serve anche a minimizzare il volume dei volumi figli.



L'impossibilità di ottimizzare su tutti gli assi possibili si applica anche alla scelta del punto di suddivisione. Poiché ci sono infiniti punti di suddivisione lungo l'asse, anche in questo caso la selezione deve essere ristretta a un piccolo insieme di scelte, come:

- Mediana delle coordinate del baricentro (mediana dell'oggetto).
- Media delle coordinate del baricentro (media dell'oggetto).
- Mediana delle estensioni della proiezione del volume di bounding (mediana spaziale).

Possono essere adottate diverse strategie di partizione:

- Minimizzare la somma dei volumi (o delle superfici) dei volumi figli. La probabilità di un'intersezione tra un volume di bounding e uno dei due volumi figli può essere considerata proporzionale al loro volume. Pertanto, minimizzare la somma dei volumi riduce efficacemente la probabilità di intersezione.
- Minimizzare il volume (o la superficie) massima dei volumi figli. Mentre la strategia precedente può portare a un volume molto più grande dell'altro, questo approccio cerca di rendere i volumi più simili in dimensione rendendo il volume più grande il più piccolo possibile. Questo porta a una query più bilanciata, migliorando il comportamento nel caso peggiore.
- Minimizzare il volume (o la superficie) dell'intersezione dei volumi figli. Questa strategia aiuta a ridurre la probabilità che entrambi i figli siano sovrapposti e attraversati. A seconda del volume di bounding utilizzato, l'intersezione può essere complessa da costruire e persino da approssimare.
- Massimizzare la separazione dei volumi figli. Separare i figli, anche quando non si sovrappongono, può ulteriormente diminuire la probabilità che entrambi i figli vengano attraversati.
- Dividere le primitive equamente tra i volumi figli. Questa strategia è l'algoritmo median-cut menzionato all'inizio della sezione.
- Combinazioni delle strategie precedenti.

BOTTOM-UP CONSTRUCTION

A differenza dei metodi top-down, i metodi bottom-up sono più complicati da implementare e hanno un tempo di costruzione più lento, ma di solito producono gli alberi migliori. Per costruire una gerarchia ad albero dal basso verso l'alto, il primo passo è racchiudere ogni primitiva all'interno di un volume di bounding. Questi volumi formano i nodi foglia dell'albero. Dall'insieme risultante di volumi, vengono selezionati due (o più) nodi foglia in base a un criterio di fusione (chiamato anche regola di clustering). Un criterio di fusione significativo è selezionare la coppia in modo da minimizzare il volume del loro volume di bounding. Quando si raggruppano due oggetti sotto un nodo comune, la coppia di oggetti che risulta nel volume di bounding più piccolo corrisponde molto probabilmente alla coppia di oggetti più vicini tra loro. Pertanto, il criterio di fusione viene spesso semplificato abbinando il nodo di query al suo vicino più vicino. Questi due nodi vengono quindi racchiusi all'interno di un volume di bounding, che sostituisce i nodi originali nell'insieme. Questa procedura di abbinamento si ripete fino a quando l'insieme consiste in un unico volume di bounding che rappresenta il nodo radice dell'albero costruito.

```
Node *BottomUpBVTree(Object object[], int numObjects)
{
    assert(numObjects != 0);
    int i, j;
    // Allocate temporary memory for holding node pointers to
    // the current set of active nodes (initially the leaves)
    NodePtr *pNodes = new NodePtr[numObjects];
    // Form the leaf nodes for the given input objects
    for (i = 0; i < numObjects; i++) {
        pNodes[i] = new Node;
        pNodes[i]->type = LEAF;
    }
}
```

```

    pNodes[i]->object = &object[i];
}
// Merge pairs together until just the root object left
while (numObjects > 1) {
    // Find indices of the two "nearest" nodes, based on some criterion
    FindNodesToMerge(&pNodes[0], numObjects, &i, &j);
    // Group nodes i and j together under a new internal node
    Node *pPair = new Node;
    pPair->type = NODE;
    pPair->left = pNodes[i];
    pPair->right = pNodes[j];
    // Compute a bounding volume for the two nodes
    pPair->BV = ComputeBoundingVolume(pNodes[i], pNodes[j]);
    // Remove the two nodes from the active set and add in the new node.
    // Done by putting new node at index min and copying last entry to max
    int min = i, max = j;
    if (i > j) {min = j; max = i; }
    pNodes[min] = pPair;
    pNodes[max] = pNodes[numObjects - 1];
    numObjects--;
}
// return root of tree
Node *pRoot = pNodes[0];
delete pNodes;
return pRoot;
}

```

HIERARCHY TRAVERSAL

Per determinare lo stato di sovrapposizione di due gerarchie di volumi di bounding, è necessario stabilire una regola su come discendere negli alberi quando i loro volumi di livello superiore si sovrappongono. Ad esempio:

- *Discendere A prima di B* (o viceversa). Discendere completamente nelle foglie di A prima di iniziare a scendere in B può essere molto svantaggioso. Nell'esempio, se un piccolo oggetto si trova da qualche parte nel mezzo di un grande oggetto, le foglie della gerarchia dell'oggetto piccolo sovrapporranno tutte il volume superiore della gerarchia dell'oggetto grande. Questa regola di discesa è la scelta peggiore possibile, poiché la gerarchia B verrà ricorsivamente esaminata tante volte quante sono le foglie nella gerarchia A. La gerarchia per A è chiaramente controproducente in questo caso, risultando in un lavoro maggiore rispetto a non avere affatto una gerarchia!
- *Discendere il volume più grande*. Determinando dinamicamente quale gerarchia è attualmente la più grande e discendendo in essa, si eludono i problemi dei due metodi precedenti. Questa è una delle regole di discesa più efficaci poiché fornisce la maggiore riduzione del volume totale per i successivi test di volumi di bounding.
- *Discendere A e B alternativamente*. Le gerarchie possono anche essere esplorate in modo alternato, in cui prima si discende in A, poi in B, poi di nuovo in A, e così via.
- *Combinazioni delle regole precedenti o altre regole complesse basate sulla storia della traversata*.

Molte regole di discesa possono essere gestite da una semplice procedura che ricorre sulle due gerarchie. Prima, se i loro volumi di bounding superiori non si sovrappongono, la procedura restituisce semplicemente. Se non è così, se entrambi i nodi forniti sono nodi

foglia, viene chiamata la routine di basso livello per gestire la geometria contenuta in collisione. Altrimenti, la regola di discesa viene valutata e il codice viene chiamato ricorsivamente per i nodi figlio della gerarchia selezionata dalla regola da esplorare. Tradotto direttamente in codice, questo diventa:

```
// Generic recursive BVH traversal code.
// Assumes that leaves too have BVs
void BVHCollision(CollisionResult *r, BVTree a, BVTree b)
{
    if (!BVOverlap(a, b)) return;
    if (IsLeaf(a) && IsLeaf(b)) {
        // At leaf nodes. Perform collision tests on leaf node contents
        CollidePrimitives(r, a, b);
    } else {
        if (DescendA(a, b)) {
            BVHCollision(a->left, b);
            BVHCollision(a->right, b);
        } else {
            BVHCollision(a, b->left);
            BVHCollision(a, b->right);
        }
    }
}
```

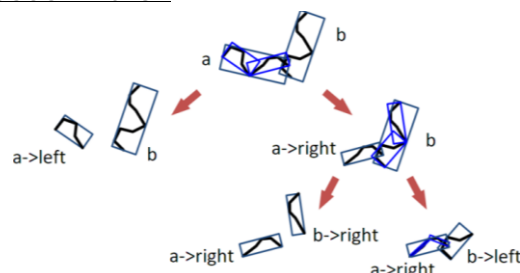
Nel codice, la funzione BVOverlap() determina la sovrapposizione tra due volumi di bounding. IsLeaf() restituisce true se il suo argomento è un nodo foglia e non un nodo interno. CollidePrimitives() gestisce le collisioni tra tutte le primitive contenute, accumulando eventuali collisioni segnalate nella struttura CollisionResult fornita. DescendA() implementa la regola di discesa e restituisce true se la gerarchia dell'oggetto A deve essere esplorata o false se deve essere esplorata la gerarchia dell'oggetto B.

```
// Descend A descent rule
bool DescendA(BVTree a, BVTree b)
{
    return !IsLeaf(a);
}

// Descend B descent rule
bool DescendA(BVTree a, BVTree b)
{
    return IsLeaf(b);
}

// Descend larger descent rule
bool DescendA(BVTree a, BVTree b)
{
    return IsLeaf(b) || (!IsLeaf(a) && (SizeOfBV(a) >= SizeOfBV(b)));
}
```

Esempio di “Descend A descent rule”



SAMPLE BOUNDING VOLUME HIERARCHIES

OBV Trees

- **Vantaggi:**
 - Migliore adattamento rispetto ad AABB e sphere
 - Invarianza rotazionale nello spazio del corpo per corpi rigidi
- **Svantaggi:**
 - Test di intersezione più costosi
 - Più difficile generare l'albero

AABB Trees

- **Vantaggi:**
 - Struttura facile da calcolare
 - Test di intersezione semplici
 - Spesso un adattamento più preciso rispetto all'albero delle sfere
- **Svantaggi:**
 - Se l'oggetto ruota, è necessario ricalcolare l'AABB ad ogni fase
 - La geometria dell'albero varia con la rotazione dell'oggetto

Sphere Tree

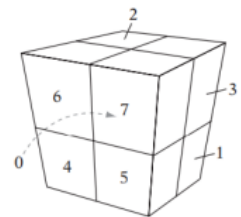
- **Vantaggi:**
 - I nodi sono invarianti rispetto alla rotazione
 - Uso relativamente basso di memoria
- **Svantaggi:**
 - Adattamento scarso per oggetti lunghi/sottili/piatti

Spatial Data Structures

Le tecniche di partizionamento spaziale forniscono un'elaborazione di fase ampia suddividendo lo spazio in regioni e verificando se gli oggetti sovrappongono la stessa regione dello spazio. Poiché gli oggetti possono intersecarsi solo se sovrappongono la stessa regione dello spazio, il numero di test coppia per coppia è drasticamente ridotto.

OCTREES

Un octree è una partizione gerarchica allineata agli assi di un volume dello spazio 3D. Come suggerisce il nome, ogni nodo padre nell'octree ha otto nodi figli. Inoltre, ogni nodo ha anche un volume finito associato. Il volume del nodo radice è generalmente considerato il più piccolo cubo allineato agli assi che racchiude completamente il mondo. Questo volume viene quindi suddiviso in otto cubi più piccoli di dimensioni uguali, dividendo simultaneamente il cubo a metà lungo ciascuno degli assi x, y e z. Questi subcubi formano i nodi figli del nodo radice. I nodi figli sono, a loro volta, ricorsivamente suddivisi nella stessa maniera. I criteri tipici per fermare la creazione ricorsiva dell'octree includono il raggiungimento di una profondità massima dell'albero o il fatto che i cubi diventino più piccoli di una certa dimensione minima preimpostata. In virtù della costruzione, il volume di spazio associato a ciascun nodo dell'albero contiene completamente tutti i nodi discendenti. Anche se i confini geometrici di un nodo possono, ad esempio, essere trasmessi e affinati durante la



struttura di dati ad albero per dividere lo spazio tridimensionale in parti più piccole. Il nodo genitore si suddivide in 8 nodi figli i cui lati sono paralleli agli assi xyz. La suddivisione può essere ricorsiva anche per i nodi figli ottenendo altri cubi. Non si può suddividere all'infinito ma fino a che non si raggiunge una profondità massima oppure fin quando i cubi non raggiungono una dimensione minima stabilita oppure quando un cubo conterrà un solo oggetto oppure nessuno. Per navigare velocemente dentro l'albero ogni nodo contiene la sua posizione nello spazio e la sua grandezza. Dentro ci possono essere oggetti statici e dinamici.

- statico -> non si muove, in fase di divisione può essere tagliato

- dinamico -> entità in movimento e viene inserito nel più piccolo spazio che può contenere l'oggetto, non si taglia

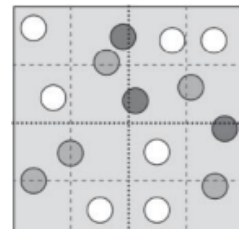
traversata dell'octree, la struttura del nodo è spesso migliorata per contenere sia la posizione del nodo che la sua estensione.

```
struct Node {
    Point center; // Center point of octree node (not strictly needed)
    float halfWidth; // Half the width of the node volume (not strictly needed)
    Node *pChild[8]; // Pointers to the eight children nodes
    Object *pObjList; // Linked list of objects contained at this node
};
```

OCTREE OBJECT ASSIGNMENT

Gli octree possono essere costruiti per contenere dati statici o dinamici. Nel primo caso, i dati possono essere le primitive che formano l'ambiente del mondo. Nel secondo caso, i dati sono le entità in movimento nel mondo. Nello scenario statico, è semplice formare l'octree utilizzando una costruzione dall'alto verso il basso. Inizialmente, tutte le primitive nel mondo sono associate al volume radice. Man mano che il volume radice viene suddiviso, il set di primitive viene riassegnato alle celle figlie con cui sovrappone, duplicando le primitive quando sovrappongono più di una cella. La procedura viene ripetuta ricorsivamente, con il criterio di fermo di non suddividere celle che contengono meno di un certo numero fisso di primitive o quando le primitive sono assegnate a tutti i figli (eliminando ulteriori suddivisioni). Determinare a quali volumi figli assegnare una primitiva può anche essere fatto dividendo prima le primitive secondo il piano di partizione dell'asse x dell'octree, dividendo eventuali primitive che sovrappongono in due parti, una per ciascun lato. Entrambi i set vengono quindi suddivisi similmente per il piano di partizione dell'asse y, e i quattro set risultanti vengono suddivisi ancora una volta dal piano dell'asse z. Gli otto set finali vengono quindi assegnati al corrispondente nodo figlio. Se una primitiva deve essere divisa da un dato piano di partizione è determinato testando se ha vertici su entrambi i lati del piano. Un approccio alternativo per l'assegnazione è non suddividere le primitive. Invece, per ogni volume figlio e per ogni primitiva viene eseguito un test di sovrapposizione completo tra AABB e primitiva per vedere se la primitiva dovrebbe essere assegnata a quel volume.

La duplicazione delle primitive tra gli ottanti sovrapposti funziona bene per un ambiente statico. Tuttavia, questo approccio è inefficiente quando l'octree contiene oggetti dinamici, poiché l'albero deve essere aggiornato man mano che gli oggetti si muovono. Per lo scenario dinamico, un approccio migliore è limitare il posizionamento degli oggetti alla cella octree più bassa che contiene l'oggetto al suo interno. Uno svantaggio di limitare gli oggetti a trovarsi all'interno di una singola cella octree è che gli oggetti possono trovarsi più in alto nell'albero di quanto giustifichi la loro dimensione. Specificamente, qualsiasi oggetto che sovrappone a un piano di partizione rimarrà "bloccato" a quel livello, indipendentemente dalla dimensione relativa tra l'oggetto e il volume del nodo.



Da qui in poi, si presume che l'octree sia di tipo dinamico. Dato la precedente definizione di nodo octree, la preallocazione di un octree basato su puntatori fino a una profondità specificata può essere fatta come:

```
Node *BuildOctree(Point center, float halfWidth, int stopDepth) profondità massima che può avere l'albero
{
    if (stopDepth < 0) return NULL;
    else {
        Node *pNode = new Node; // Construct and fill in 'root' of this subtree
```

```

pNode->center = center;
pNode->halfWidth = halfWidth;
pNode->pObjList = NULL;
// Recursively construct the eight children of the subtree
Point offset;          mi servono a determinare il centro
float step = halfWidth * 0.5f;  degli 8 figli
for (int i = 0; i < 8; i++) {
    offset.x = ((i & 1) ? step : -step);    il ciclo usa l'operatore di bitwise
    offset.y = ((i & 2) ? step : -step);    sull'indice per decidere la posizione
    offset.z = ((i & 4) ? step : -step);    dei figli: sopra sotto avanti indietro
    pNode->pChild[i] = BuildOctree(center + offset, step, stopDepth - 1);
    sinistra o destra
}
return pNode;
}
}

```

Poiché gli oggetti sono stati limitati a trovarsi in una sola cella, la soluzione più semplice con una lista collegata per gestire più oggetti in una cella è quella di incorporare il puntatore "next" della lista collegata all'interno dell'oggetto. Per il codice di esempio fornito qui, si presume che l'oggetto sia strutturato come segue:

```

struct Object {
    Point center; // Center point for object
    float radius; // Radius of object bounding sphere
    ...
    Object *pNextObject; // Pointer to next object when linked into list
};

```

Il codice per inserire un oggetto diventa ora: pTree nodo corrente (dove andrò a inserire l'oggetto)

```

void InsertObject(Node *pTree, Object *pObject)
{
    index: indice del sottonodo dove si trova l'oggetto
    int index = 0, straddle = 0; straddle variabile booleana se l'oggetto si trova tra piu nodi figli (difficile
    // Compute the octant number [0..7] the object sphere center is in da suddividere)
    // If straddling any of the dividing x, y, or z planes, exit directly
    for (int i = 0; i < 3; i++) {
        float delta = pObject->center[i] - pTree->center[i];
        if (Abs(delta) <= pObject->radius) {
            straddle = 1;
            break;
        }
        if (delta > 0.0f) index |= (1 << i); // ZYX inizio a strutturare l'indice
    }
    if (!straddle && pTree->pChild[index]) { se l'oggetto non attraversa alcun piano ed esiste un figlio in
        // Fully contained in existing child node; insert in that subtree quella posizione l'algoritmo richiama se stesso cercando di
        InsertObject(pTree->pChild[index], pObject); scendere verso il nodo foglia
    } else {
        // Straddling, or no child node to descend into, so link object into list
        pObject->pNextObject = pTree->pObjList;
        pTree->pObjList = pObject;
    }
}
}

```

Il test di collisione di un octree dinamico può essere eseguito come una procedura ricorsiva dall'alto verso il basso, in cui gli oggetti di ciascun nodo non vuoto vengono controllati contro tutti gli oggetti nel sottoalbero sotto il nodo.

```

void TestAllCollisions(Node *pTree)
{
    // Keep track of all ancestor object lists in a stack
    const int MAX_DEPTH = 40;
    static Node *ancestorStack[MAX_DEPTH];
    static int depth = 0; // 'Depth == 0' is invariant over calls
    // Check collision between all objects on this level and all ancestor
    // objects. The current level is included as its own ancestor
    ancestorStack[depth++] = pTree;
    for (int n = 0; n < depth; n++) {
        Object *pA, *pB;
        for (pA = ancestorStack[n]->pObjList; pA; pA = pA->pNextObject) {
            for (pB = pTree->pObjList; pB; pB = pB->pNextObject) {
                // Avoid testing both A->B and B->A
                if (pA == pB) break;
                TestCollision(pA, pB); // test collision between pA and pB in some manner
            }
        }
    }
    for (int i = 0; i < 8; i++) // Recursively visit all existing children
        if (pTree->pChild[i]) se nodo esiste
            TestAllCollisions(pTree->pChild[i]);
    depth--; // Remove current node from ancestor stack before returning
}

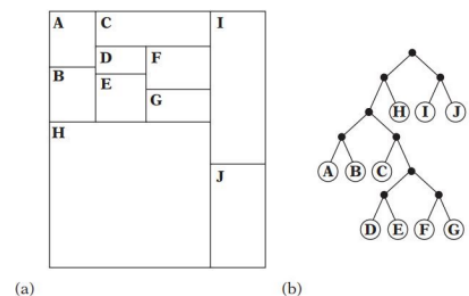
```

va a confrontare pA che sono oggetti del nodo antenato e pB che sono gli oggetti del nodo attuale

K-D TREES

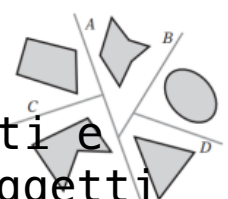
Una generalizzazione degli octree può essere trovata nell'albero k-dimensionale, o albero k-d. Invece di suddividere simultaneamente lo spazio in tre dimensioni (octree), l'albero k-d suddivide lo spazio lungo una dimensione alla volta.

Tradizionalmente, gli alberi k-d vengono suddivisi lungo x, poi y, poi z, poi x di nuovo, e così via, in modo ciclico. Tuttavia, spesso l'asse di suddivisione viene selezionato liberamente tra le k dimensioni. Poiché la suddivisione può essere posizionata arbitrariamente lungo l'asse, questo comporta che sia l'asse che il valore di suddivisione vengano memorizzati nei nodi dell'albero k-d. La figura mostra una decomposizione spaziale k-d in 2D e il corrispondente layout dell'albero k-d.



BINARY SPACE PARTITION (BSP) TREE

Tra i numerosi metodi di partizionamento spaziale disponibili, l'albero BSP è il più versatile. Un albero di partizionamento dello spazio binario (o BSP tree in breve) è una struttura ad albero binario che partiziona ricorsivamente lo spazio in coppie di sottospazi rispetto a piani di suddivisione di posizione e orientamento arbitrari. Le due partizioni, o semispazi, in cui lo spazio è suddiviso, sono comunemente chiamate semispazio positivo e semispazio negativo. Il semispazio positivo si trova davanti all'iperpiano di suddivisione e quello negativo dietro di esso. I piani di suddivisione vengono anche chiamati piani di partizione o piani di suddivisione. La figura mostra come uno spazio può essere partizionato spazialmente per accelerare le interrogazioni di collisione



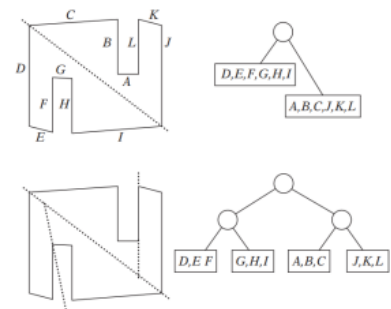
bsp divide lo spazio virtuale in due parti e ad ogni passaggio riduce il numero di oggetti da controllare, non controlla tutti gli oggetti ma solo la piccola parte e basta

Grazie alla gerarchia formata dall'albero BSP, con n oggetti nell'albero, solo nell'ordine di $O(\log n)$ oggetti vengono generalmente testati durante una query, poiché ci si può aspettare che metà degli oggetti rimanenti venga eliminata da ciascun test successivo contro un piano di partizione. Gli alberi BSP che utilizzano solo piani di suddivisione allineati agli assi sono comunemente chiamati alberi k-d.

LEAF-STORING BSP TREES

Il termine **albero BSP con archiviazione nelle foglie** (o **leaf-storing**) si riferisce a qualsiasi albero BSP in cui la geometria è archiviata nelle foglie dell'albero, piuttosto che nei nodi interni. Un nodo interno dell'albero contiene solo il piano di suddivisione e i riferimenti ai sottoalberi figli del nodo. Le interrogazioni di collisione sugli alberi con archiviazione nelle foglie eseguono i test su tutti i poligoni contenuti nelle foglie che vengono sovrapposte dall'oggetto della query. I test sull'albero possono interrompersi anticipatamente se viene rilevata una collisione. I piani di suddivisione degli alberi con archiviazione nelle foglie possono essere generali o auto-partizionanti. In entrambi i casi, eventuali facce coplanari con il piano di suddivisione selezionato non vengono archiviate nel nodo, ma inviate al sottoalbero del lato anteriore o posteriore. Nel caso di auto-partizionamento, tutte le facce coplanari con un piano di suddivisione selezionato dovrebbero essere contrassegnate per escluderle dalla selezione di futuri piani di suddivisione. Se non vengono escluse, teoricamente un piano di suddivisione potrebbe essere rilesionato più avanti nell'albero, il che non ha senso.

La costruzione di un albero con archiviazione nelle foglie è mostrata in figura. Usando il partizionamento arbitrario, il piano di suddivisione selezionato inizialmente divide il set di input a metà. Queste due metà vengono quindi divise nuovamente, e a quel punto la costruzione viene fermata, poiché è stata ottenuta una partizione ragionevole. Gli alberi BSP con archiviazione nelle foglie (insieme agli alberi k-d e agli octree) sono strutture dati comuni per la geometria di collisione, in particolare quando la geometria di collisione è rappresentata da una mesh poligonale.



BUILDING THE BSP TREE

La costruzione di un albero BSP prevede tre passaggi:

1. Selezione di un piano di partizione.
2. Partizionamento della geometria in ingresso in due insiemi rispetto al piano: la geometria davanti al piano e la geometria dietro di esso. La geometria che attraversa il piano viene divisa lungo il piano prima della partizione.
3. Creazione dell'albero collegando con un nuovo nodo dell'albero i due sottoalberi creati chiamando ricorsivamente l'algoritmo di costruzione con i due insiemi partizionati ottenuti nel passaggio precedente.

```
// Constructs BSP tree from an input vector of polygons. Pass 'depth' as 0
```

```

BSPNode *BSPTree::BuildBSPTree(std::vector &polygons, int depth)
{
    // Return NULL tree if there are no polygons
    if (polygons.empty()) return NULL;
    // Get number of polygons in the input vector
    size_t numPolygons = polygons.size();
    // If criterion is matched, create a leaf node from remaining polygons
    if (depth >= BSPTree::MAX_DEPTH || numPolygons <= MIN_LEAF_SIZE)
        return new BSPNode(polygons);
    // Select best possible partitioning plane based on the input geometry
    Plane splitPlane = PickSplittingPlane(polygons);
    std::vector frontList, backList;
    // Test each polygon against dividing plane, adding them to lists
    for (int i = 0; i < numPolygons; i++) {
        Polygon *poly = polygons[i], *frontPart, *backPart;
        switch (ClassifyPolygonToPlane(poly, splitPlane)) {
            case POLYGON_COPLANAR_WITH_PLANE:
            case POLYGON_IN_FRONT_OF_PLANE:
                frontList.push_back(poly);
                break;
            case POLYGON_BEHIND_PLANE:
                backList.push_back(poly);
                break;
            case POLYGON_STRADDLING_PLANE:
                // Split polygon to plane and send a part to each side
                SplitPolygon(*poly, splitPlane, &frontPart, &backPart);
                frontList.push_back(frontPart);
                backList.push_back(backPart);
                break;
        }
    }
    // Recursively build child subtrees and return new tree root
    if (frontList.size() == 0 || backList.size() == 0) return new BSPNode(polygons);
    BSPNode *frontTree = BuildBSPTree(frontList, depth + 1);
    BSPNode *backTree = BuildBSPTree(backList, depth + 1);
    return new BSPNode(frontTree, backTree, splitPlane);
}

```

Per la costruzione di alberi BSP a foglia o alberi solidi, i poligoni che si trovano sul piano di partizione vengono inviati su uno dei due lati del piano (in questo caso, sul lato anteriore del piano).

La costruzione di un albero BSP a foglia viene tipicamente interrotta quando:

- La foglia contiene meno di un numero prefissato di poligoni.
- È stata raggiunta una profondità massima predefinita.
- Non è possibile trovare un buon piano di divisione.

SELECTING DIVIDING PLANES

La selezione di un buon piano di partizione è un problema complesso. L'ordine in cui i piani di divisione vengono selezionati durante la costruzione dell'albero influisce notevolmente sulla dimensione e la forma dell'albero risultante, e quindi sull'efficacia dell'albero nelle interrogazioni. A causa dell'enorme numero di possibili ordini dei piani, calcolare un albero BSP ottimale è un problema intrattabile. Fortunatamente, nella pratica è possibile ottenere soluzioni buone o quasi ottimali provando un certo numero di piani di divisione candidati a

ciascun livello dell'albero e procedendo con quello che ottiene il punteggio migliore secondo una funzione di valutazione.

Poiché esistono infiniti piani di divisione arbitrari, un approccio per limitare il problema consiste nel ridurre il set di possibili piani a un numero finito, affidandosi all'autopartizione. In questo caso, la strategia consiste nel limitare la ricerca ai piani di supporto delle facce nella geometria di input. Tuttavia, l'autopartizione da sola non è sempre l'opzione ideale. In alcuni casi - come per un albero a foglia - affidarsi esclusivamente all'autopartizione può portare a più divisioni di quanto desiderato.

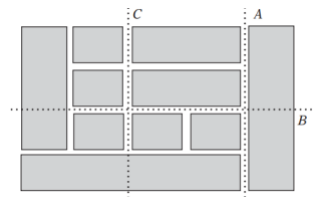
Per illustrare il punto, la figura mostra una configurazione geometrica in cui tutti i possibili piani di divisione autopartizionati finiscono per dividere diverse facce.



La figura mostra invece come, utilizzando piani di divisione generali, i gruppi di poligoni possano essere separati tra loro senza causare alcuna divisione.



Molte strategie generali sono state proposte per valutare i piani di divisione. Due strategie particolarmente rilevanti per il rilevamento delle collisioni consistono nello scegliere piani che minimizzano la divisione della geometria e cercare di bilanciare la geometria in modo equo su entrambi i lati del piano di divisione. La prima strategia, quella del *least-crossed* (minimizzazione delle intersezioni), tenta di ridurre al minimo la duplicazione della geometria dovuta alla divisione. D'altra parte, i tagli bilanciati - sebbene producano alberi strutturalmente ben organizzati - rischiano di causare un'eccessiva quantità di geometria divisa, poiché tendono a tagliare esattamente nel mezzo di una scena e dei suoi oggetti. Per questo motivo, nella pratica si utilizza una combinazione lineare ponderata tra le due strategie. La riduzione delle divisioni è spesso ponderata molto più del bilanciamento. La figura illustra una divisione progettata per minimizzare i poligoni attraversati, una divisione per bilanciare il numero di poligoni su entrambi i lati del piano di divisione, e una divisione progettata come compromesso tra la minimizzazione delle intersezioni e il bilanciamento dei poligoni.



Il seguente frammento di codice mostra una selezione di piano di divisione molto basilare utilizzando l'autopartizione. Qui, tutti i piani di supporto del set di poligoni vengono considerati per la divisione. Viene utilizzata una combinazione lineare ponderata tra gli obiettivi in conflitto: cercare di ridurre le divisioni e cercare di migliorare il bilanciamento.

Plane BSPTree::PickSplittingPlane(std::vector &polygons)

```
{
    int best_poly_i = 0;
    const float K = 0.2f; // Blend factor
    Plane bestPlane;
    float bestScore = FLT_MAX;
```

```
// Try the plane of each polygon as a dividing plane
for (int i = 0; i < polygons.size(); i++) {
    int numInFront = 0, numBehind = 0, numStraddling = 0;
    Plane plane = GetPlaneFromPolygon(polygons[i]);
    // Test against all other polygons
```

più k è piccolo più si dà importanza al bilanciamento piuttosto che al taglio

```

for (int j = 0; j < polygons.size(); j++) {
    if (i == j) continue; // Ignore testing against self
    // Keep standing count of the various poly-plane relationships
    switch (ClassifyPolygonToPlane(polygons[j], plane)) {
        case POLYGON_COPLANAR_WITH_PLANE:
            /* Coplanar polygons treated as being in front of plane */
            case POLYGON_IN_FRONT_OF_PLANE:
                numInFront++;
                break;
            case POLYGON_BEHIND_PLANE:
                numBehind++;
                break;
            case POLYGON_STRADDLING_PLANE:
                numStraddling++;
                break; }
    }
    float score = K * numStraddling + (1.0f - K) * abs(numInFront - numBehind); //
    Compute score as a weighted combination
    if (score < bestScore) {
        bestScore = score;
        bestPlane = plane;
        best_poly_i = i;
    }
}
return bestPlane;
}

```

CLASSIFYING POLYGONS WITH RESPECT TO A PLANE

Come parte della costruzione dell'albero BSP, dopo che un piano di divisione è stato selezionato, tutti i poligoni devono essere suddivisi rispetto a questo piano in una delle seguenti quattro categorie:

1. Poligoni che si trovano davanti al piano di divisione.
2. Poligoni che si trovano dietro al piano di divisione.
3. Poligoni che attraversano il piano di divisione.
4. Poligoni coincidenti con il piano di divisione.

Sebbene teoricamente semplice, questa classificazione presenta un problema quando si lavora con l'aritmetica a virgola mobile. Il problema è che i punti calcolati per trovarsi su un determinato piano in genere non finiscono esattamente sul piano, ma a una piccola distanza da esso. Questa leggera deviazione dal piano significa che, quando un poligono viene diviso da un piano di divisione, i pezzi risultanti non si classificano necessariamente come uno davanti al piano di divisione e uno dietro di esso. Per esempio, le imprecisioni nei punti di intersezione calcolati potrebbero far sì che entrambi i pezzi attraversino il piano di divisione, anche se concettualmente l'operazione di divisione ha creato due pezzi che dovrebbero trovarsi su lati diversi del piano.

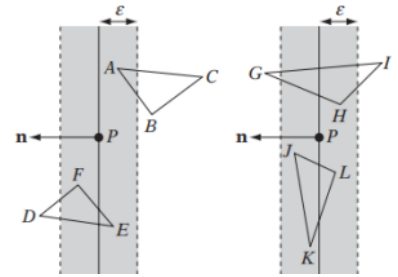
Affrontare il problema dell'aritmetica a virgola mobile è necessario per una classificazione robusta dei poligoni in queste quattro categorie. Una soluzione pratica a questo problema consiste nell'utilizzare piani "spessi", il che significa che tutti i punti entro una piccola distanza dal piano sono considerati come giacenti sul piano. Più specificamente, invece di lavorare con un piano definito come $n \cdot (X - P) = 0$, i punti sono considerati sul piano se

soddisfano $n \cdot (X-P) < \epsilon$ per un certo ϵ . La seguente funzione mostra come può essere implementata la classificazione di un punto rispetto a un piano spesso.

```
int BSPTree::ClassifyPointToPlane(Point p, Plane plane)
{
    // Compute signed distance of point from plane and Classify p
    float dist = Dot(plane.n, p) - plane.d;
    if (dist > PLANE_THICKNESS_EPSILON)
        return POINT_IN_FRONT_OF_PLANE;
    if (dist < -PLANE_THICKNESS_EPSILON)
        return POINT_BEHIND_PLANE;
    return POINT_ON_PLANE;
}
```

Come esempio di classificazione di poligoni rispetto a un piano spesso, considera i quattro triangoli della figura.

- Per il triangolo ABC, il vertice A si trova sul piano, mentre i vertici B e C si trovano dietro il piano. Poiché almeno un vertice è dietro il piano e nessun vertice si trova davanti, ABC è classificato come situato dietro al piano.
- Il triangolo DEF ha un vertice, D, davanti al piano e due vertici, E e F, sul piano. Poiché c'è almeno un vertice davanti al piano e nessun vertice dietro, DEF è classificato come situato davanti al piano. Nota che DEF è considerato davanti al piano, anche se E si trova nel semispazio negativo del piano $n \cdot (X-P) = 0$, poiché E è classificato come sul piano piuttosto che dietro di esso.
- Il triangolo GHI è classificato come attraversante il piano di divisione poiché c'è almeno un vertice davanti al piano e uno dietro al piano (rispettivamente G e I).
- Il triangolo JKL è classificato come complanare al piano in quanto tutti i suoi vertici si trovano sul piano spesso.



Il codice seguente illustra come questo approccio possa essere esteso per classificare un poligono arbitrario rispetto a un piano di divisione.

```
int BSPTree::ClassifyPolygonToPlane(Polygon *poly, Plane plane)
{
    int numInFront = 0, numBehind = 0;
    int numVerts = poly->getNumVertices();
    for (int i = 0; i < numVerts; i++) {
        Point p = (poly->getVertexes())[i];
        switch (ClassifyPointToPlane(p, plane)) {
            case POINT_IN_FRONT_OF_PLANE:
                numInFront++;
                break;
            case POINT_BEHIND_PLANE:
                numBehind++;
                break; }
    }
    if (numBehind != 0 && numInFront != 0)
        return POLYGON_STRADDLING_PLANE;
    if (numInFront != 0)
        return POLYGON_IN_FRONT_OF_PLANE;
    if (numBehind != 0)
        return POLYGON_BEHIND_PLANE;
    return POLYGON_COPLANAR_WITH_PLANE;
}
```

indica che si trova esattamente fra le due parti

La differenza pratica più grande tra gli alberi BSP e altri tipi di alberi 3D (Octrees, k-d trees) è che gli alberi BSP possono essere più ottimizzati ma funzionano solo su geometria statica. Questo perché gli alberi BSP sono generalmente molto lenti da costruire, spesso richiedendo ore o giorni per un tipico livello di gioco urbano statico. Il motivo principale per cui gli alberi BSP richiedono più tempo per essere costruiti è che utilizzano piani di divisione non allineati agli assi. Un'altra differenza è che gli alberi BSP memorizzano la geometria solo nelle foglie, mentre gli Octrees memorizzano la geometria non solo nei nodi foglia, ma anche negli altri nodi.

Physics-Based Animation

Comprendere le leggi del mondo reale e simularle computazionalmente.

La modellazione basata sulla fisica è diventata un approccio importante nell'animazione computerizzata e nella modellazione grafica al computer.

Comporta l'uso di alcune nozioni matematiche:

- Dinamica dei corpi rigidi (Leggi del moto di Newton)
- Metodi numerici per equazioni differenziali
- Rilevamento delle collisioni
- Modellazione dei contatti e dei vincoli

Applicazioni:

- Scienza
- Giochi
- Film
- Industria

In tempo reale; almeno 30 fotogrammi al secondo. *Verosimiglianza*: è sufficiente che sembri naturale, l'approssimazione può funzionare. *Interattività*: differente da un'animazione predefinita, deve interagire con gli utenti e con altri elementi del gioco.

Gli oggetti interagiscono attraverso forze tra oggetti distanti (gravità) o tra oggetti a contatto (attrito, collisione). Fisica nei giochi: analizzare e simulare l'interazione.

Cinematica: descrizione del moto (Posizione, orientamento, velocità, accelerazione).

Dinamica: forze e i loro effetti (Massa, inerzia, collisioni, quantità di moto).

Animazione scriptata:

- ❖ Ideale per sequenze predeterminate.
- ❖ Richiede la prescrizione dell'intera sequenza.
- ❖ Può essere progettata per un effetto drammatico.
- ❖ Richiede animatori esperti per effetti realistici.
- ❖ Le risorse e lo sforzo dell'animatore devono aumentare in proporzione alla complessità.

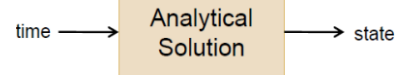
é un movimento creato da un animatore, si fissano i famosi kframe e il pc andrà a riempire i buchi fra questi due kframe.
PRO: risulta essere poco costoso a livello di cpu
CONTRO: se noi mettiamo un muro davanti al personaggio lui lo attraversa e continua a camminare.

Animazione basata sulla fisica:

- ❖ Ideale per applicazioni interattive.
- ❖ Richiede un modello fisico e condizioni iniziali.
- ❖ L'animazione non può essere controllata direttamente.
- ❖ Il realismo è un sottoprodotto della modellazione fisica.
- ❖ Le risorse computazionali devono aumentare in proporzione alla complessità.

vengono applicate le leggi matematiche ad un esoscheletro virtuale quindi ha un realismo estremo
Contro: consuma molta cpu e può causare bug. Se metto un muro l'esoscheletro non lo attraverserà

il movimento viene descritto da una funzione precisa. La funzione riceverà in input il parametro temporale T e restituisce lo stato del sistema. Risulta essere precisa e senza errore, inoltre lo stato del sistema può essere determinato in qualsiasi istante anche in quelli precedenti. Il problema è che queste funzioni precise sono molto difficili da determinare

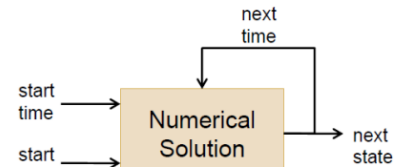


Modelli analitici

- Calcolano lo stato come funzione del tempo.
- Computazionalmente efficienti.
- Molto accurati (nessuna accumulazione di errori).
- Limitati a configurazioni semplici e prevedibili senza interazioni.
- Richiedono una soluzione per ogni classe di problema.

Modelli numerici

- Aggiornano iterativamente lo stato su piccoli intervalli di tempo.
- Complessità polinomiale.
- Errori numerici si accumulano nelle simulazioni nel tempo.
- Possono gestire configurazioni interattive di complessità arbitraria.
- Approccio generico adatto a molti problemi.

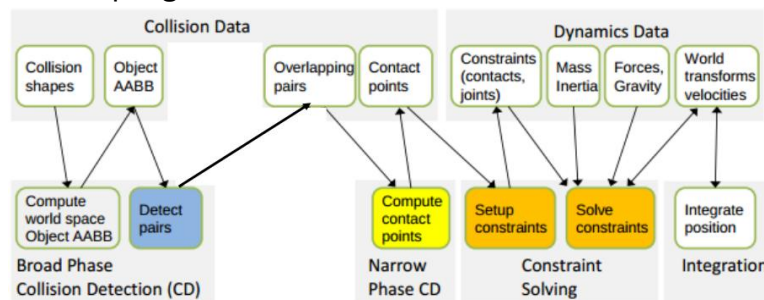


Non si utilizzano delle formule ma si procede per approssimazioni. Per determinare lo stato successivo si utilizza lo stato attuale e l'intervallo di tempo. Per poter aggiornare le variabili ci si basa sulla configurazione dell'istante precedente, quindi verrà fornito lo stato successivo. Richiede un costo computazionale elevato e potrebbe essere soggetto ad errori di approssimazione

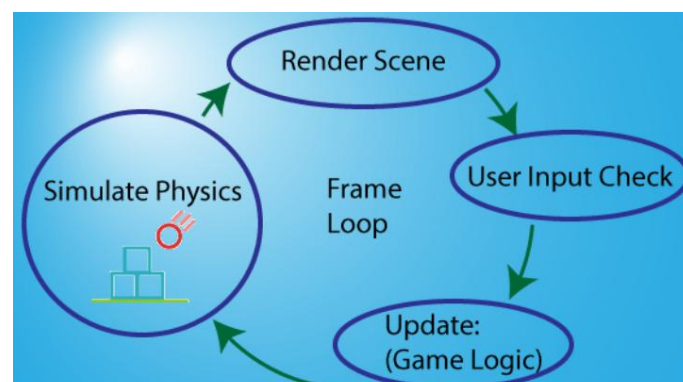
Sfide:

- 1) Discretizzazione
 - Stabilità e convergenza
 - Prevenzione degli errori
- 2) "Fino a che punto spingersi?"
 - Efficienza vs. accuratezza
 - Come fare "abbastanza bene"
- 3) Visivamente convincente
 - Approssimazione ragionevole
- 4) Come controllare le dinamiche per ottenere un effetto cinematografico?
 - Controllo attivo

Pipeline della fisica dei corpi rigidi:



Frame loop:



BASIC PHYSICS

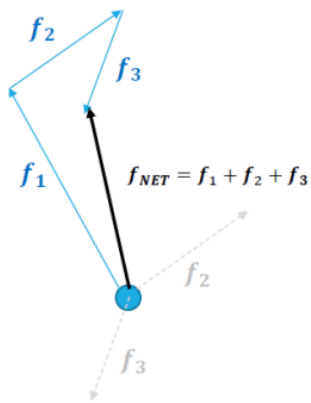
Velocità: variazione istantanea della posizione $v(t) = \frac{dx(t)}{dt} = \dot{x}(t)$

Accelerazione: variazione istantanea della velocità $a(t) = \frac{dv(t)}{dt} = \dot{v}(t) = \ddot{x}(t)$

$x(t)$, $v(t)$ e $a(t)$ sono vettori

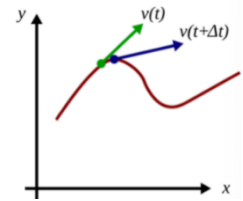
Posizione: traiettoria di un punto

Velocità: tangente alla curva della traiettoria



In un motore fisico per videogiochi, ciò che causa il movimento non sono le velocità o le accelerazioni, ma le **forze**.

La somma vettoriale di tutte le forze che agiscono su un oggetto (forza netta) determina come si muove. Isaac Newton descrisse tre leggi che governano tutto il movimento.



La prima legge di Newton specifica cosa succede quando la forza netta su un oggetto è nulla: tutte le forze individuali si annullano a vicenda, e non ci dovrebbe essere alcun cambiamento nel movimento. *Se $F_{net}=0$, non c'è alcun cambiamento nel movimento.* Questo significa che, fino a quando non viene applicata una forza netta diversa da zero un oggetto a riposo rimane a riposo, un oggetto in movimento continua a muoversi alla stessa velocità nella stessa direzione.

La seconda legge del moto di Newton descrive come si muove un oggetto quando la forza netta non è zero. $F_{net}=m \cdot a$ dove m è la massa e a l'accelerazione. Essa mostra anche che maggiore è la forza applicata, più velocemente l'oggetto accelera, per una quantità di forza equivalente, gli oggetti più leggeri accelerano più rapidamente rispetto agli oggetti più pesanti.

La terza legge del moto di Newton descrive la relazione tra le forze. Tutte le forze tra due oggetti esistono in uguale intensità e in direzione opposta: quando due oggetti entrano in contatto, si esercitano forze uguali e opposte l'uno sull'altro.

Le particelle sono oggetti che hanno massa, posizione e velocità, e rispondono a forze, ma che non hanno estensione spaziale. Le particelle sono di gran lunga gli oggetti più facili da simulare. Una vasta varietà di strutture non rigide può essere costruita collegando particelle con semplici molle smorzate,

$f = m \cdot a$ o $\ddot{x} = f/m$ di secondo ordine, equivalenti a una coppia di ODE di primo ordine accoppiate (equazioni differenziali ordinarie) $\dot{v} = f/m$, $\dot{x} = v$.

La posizione e la velocità possono essere raggruppate in un vettore a 6 dimensioni chiamato fase della particella: $\begin{bmatrix} x \\ v \end{bmatrix} \quad \{x_x, x_y, x_z, v_x, v_y, v_z\}$

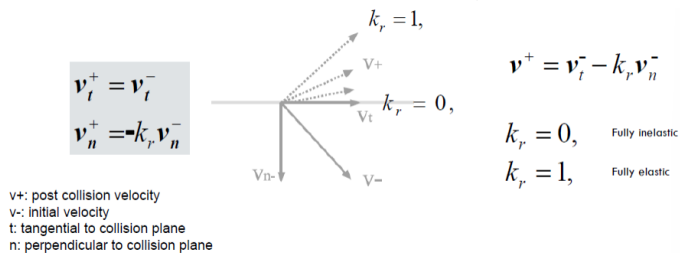
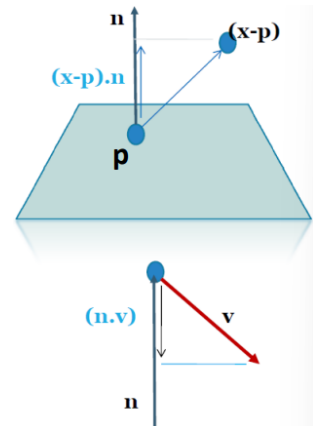
L'equazione del moto nello spazio delle fasi: $\begin{bmatrix} \dot{x} \\ \dot{v} \end{bmatrix} = \begin{bmatrix} v \\ \frac{f}{m} \end{bmatrix}$

Un sistema di n particelle è descritto da n equazioni dello spazio delle fasi, una per ciascuna particella.

Collisioni e contatto tra particelle e piani

Caso più semplice: particelle che collidono con un piano (ad esempio, il terreno o un muro). Dato il normale n e un qualsiasi punto p sul piano. Al tempo t c'è una collisione se $((x-p) \cdot n < \epsilon \ \&\& \ (n \cdot v < 0))$, cioè sta penetrando e va più a fondo. È necessario annullare la forza nel piano e respingere l'oggetto per una risposta di "rimbalzo" basata sulla fisica.

Applica un cambiamento istantaneo di velocità: modello di impulso. La velocità nella direzione tangenziale è conservata e la velocità nella direzione normale è scalata da $-k_r$, il coefficiente di restituzione.



$$\text{Dalla seconda legge di Newton } F = m \cdot a \Leftrightarrow F = \frac{m(v(t+\Delta t) - v(t))}{\Delta t} \Leftrightarrow F = \frac{m \cdot v(t+\Delta t) - m \cdot v(t)}{\Delta t} \Leftrightarrow F = \frac{\Delta p}{\Delta t} \Leftrightarrow F \Delta t = \Delta p$$

Un impulso è un cambiamento di quantità di moto, cioè la forza netta applicata all'oggetto moltiplicata per il tempo di applicazione, l'unità è anche $\text{kg} \cdot \text{m/s}$ (quindi è diversa da una forza)

In Unity3D:

- ForceMode.Impulse cambia la velocità in base a forza / massa (assumendo $\Delta t = 1$)
- ForceMode.Force cambia la velocità in base a forza / massa * Time.fixedDeltaTime

Aggiornamento della particella:

1. Azzerare la forza dalla particella
2. Accumulare le forze (ad esempio forze esterne, gravità, ecc...)
3. Applicare le forze, risolvere per la fase della particella (integrazione numerica)
4. Gestire le collisioni

FORCES

Gravità

Applicando la seconda legge di Newton a un oggetto con massa m sulla superficie della Terra otteniamo: $F_{net} = F_g = m \cdot g$, $g = 9.81 \text{ m/s}^2$.

Attrito

Capacità di resistere al movimento:

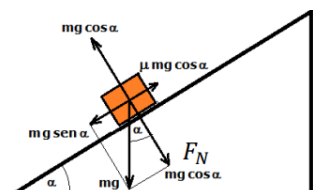
- L'attrito statico impedisce a un oggetto su una superficie di muoversi:

$$F_s = \mu_s F_N = \mu_s mg \cos(\alpha)$$

- L'attrito cinetico rallenta un oggetto in contatto

$$F_k = \mu_k F_N = \mu_k mg \cos(\alpha)$$

Se la forza tangenziale è maggiore dell'attrito statico, l'oggetto si muove. Se l'oggetto si muove mentre è in contatto, l'attrito cinetico viene applicato all'oggetto.



Molla

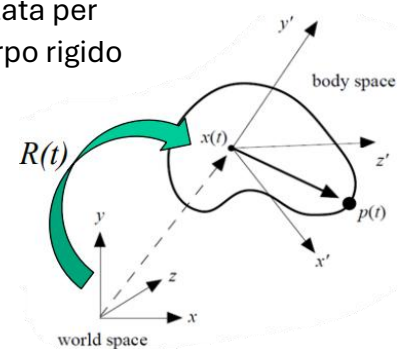
Utilizzata per connettere due o più oggetti. Reagisce secondo la legge di Hook sull'estensione e la compressione, cioè sullo spostamento relativo tra gli oggetti connessi. La lunghezza relativa l rispetto alla lunghezza a riposo l_0 determina la forza da applicare $F_s = -K(l - l_0)$, dove K è la costante della molla (in N/m).

Smorzatore

Rallenta il movimento tra due oggetti A e B connessi da una molla (riduce l'ampiezza dell'oscillazione). La forza di smorzamento dipende dalla velocità relativa tra i due oggetti $F_c = -C(v_A - v_B)$, dove C è il coefficiente di smorzamento e la forza risultante è applicata su A (in direzione opposta su B).

UNCONSTRAINED RIGID BODY DYNAMICS

Simulare corpi rigidi segue esattamente la stessa strategia utilizzata per simulare le particelle. Differenza: il vettore di stato $X(t)$ per un corpo rigido contiene più informazioni. Sia $x(t)$ la posizione del sistema di riferimento locale del corpo rispetto alle coordinate globali. Sia $R(t)$ la matrice di rotazione 3x3 del sistema di riferimento del corpo rispetto al sistema di riferimento globale. Usiamo $x(t)$ e $R(t)$ per trasformare la descrizione del corpo nello spazio del mondo.



Richiediamo che il centro di massa (vedi le prossime diapositive) del corpo sia l'origine (0,0,0) dello spazio del corpo.

Poiché il centro di massa del corpo si trova all'origine dello spazio del corpo, $x(t)$ rappresenta la posizione del centro di massa nello spazio globale al tempo t . Se p_0 è un punto nello spazio del corpo, allora la posizione nello spazio globale $p(t)$ di p_0 è il risultato di una prima rotazione di p_0 attorno all'origine e poi di una traslazione: $p(t) = R(t)p_0 + x(t)$.

Il significato fisico di $R(t)$ è che la prima colonna di $R(t)$ fornisce la direzione in cui l'asse x del corpo rigido è orientato, una volta trasformato nello spazio globale al tempo t . Allo stesso modo, la seconda e la terza colonna di $R(t)$ rappresentano le direzioni degli assi y e z del corpo rigido nello spazio globale al tempo t .

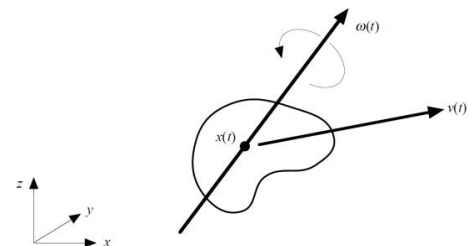
$$R(t) \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix}$$

Abbiamo bisogno di definire come cambiano la posizione e l'orientamento nel tempo.

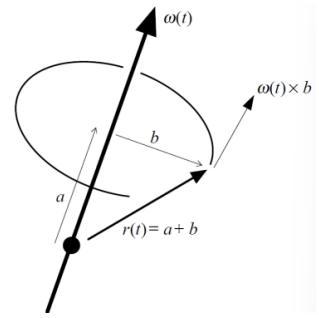
Definiamo la velocità lineare del corpo rigido $v(t)$ come $v(t) = \dot{x}(t) = \frac{d}{dt} x(t)$

Oltre a traslare, un corpo rigido può anche ruotare.

Immagina di congelare la posizione del centro di massa nello spazio. Qualsiasi movimento dei punti del corpo deve quindi essere dovuto alla rotazione del corpo attorno a un asse che passa per il centro di massa. Possiamo descrivere questa rotazione come un vettore $\omega(t)$ nello spazio globale. La direzione di $\omega(t)$ indica la direzione dell'asse attorno al quale il corpo ruota. La magnitudine $|\omega(t)|$ indica la velocità con cui il corpo sta ruotando. La quantità $\omega(t)$ è chiamata velocità angolare del corpo rigido.



Come sono correlati $R(t)$ e $\omega(t)$? Esaminiamo come il cambiamento di un vettore arbitrario in un corpo rigido è correlato alla velocità angolare $\omega(t)$. Consideriamo il corpo che ruota puramente: $v(t)=0$. Supponiamo che il centro di massa del corpo si trovi all'origine del sistema di riferimento globale. Consideriamo un vettore $r(t)$ nello spazio globale, al tempo t , applicato al centro di massa del corpo che ruota insieme al corpo. Scomponiamo $r(t)$ in vettori a e b , dove a è parallelo a $\omega(t)$ e b è perpendicolare a $\omega(t)$.



Poiché l'estremità del vettore $r(t)$ si sta muovendo istantaneamente lungo un cerchio, il cambiamento istantaneo di $r(t)$ è perpendicolare sia a b che a $\omega(t)$. La velocità istantanea di $r(t)$ ha magnitudine $|\omega(t)||b|$. Pertanto: $\dot{r}(t) = \omega(t) \times b$

Ma possiamo anche scrivere: $\dot{r}(t) = \omega(t) \times b = \omega(t) \times b + \omega(t) \times a = \omega(t) \times (b + a)$

E infine: $\dot{r}(t) = \omega(t) \times r(t)$

Al tempo t , sappiamo che la direzione dell'asse x del corpo rigido nello spazio globale è la

prima colonna di $R(t)$, che è $\begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix}$

La derivata della prima colonna di $R(t)$ è semplicemente la velocità di cambiamento di questo vettore: usando la regola del prodotto vettoriale che abbiamo appena scoperto, questo

cambiamento è $\omega(t) \times \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix}$

Pertanto, possiamo scrivere $\dot{R} = \begin{pmatrix} \omega(t) \times \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix} & \omega(t) \times \begin{pmatrix} r_{yx} \\ r_{yy} \\ r_{yz} \end{pmatrix} & \omega(t) \times \begin{pmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{pmatrix} \end{pmatrix}$

Ricorda che, se a e b sono vettori, allora $a \times b$ è il vettore $\begin{pmatrix} a_y b_z - b_y a_z \\ -a_x b_z - b_x a_z \\ a_x b_y - b_x a_y \end{pmatrix}$

Dato il vettore a , definiamo a^* come la matrice antisimmetrica (o simmetrica rispetto allo

scambio di segno): $\begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix}$

Allora: $a * b = \begin{pmatrix} 0 & -a_z & a_y \\ a_z & 0 & -a_x \\ -a_y & a_x & 0 \end{pmatrix} \begin{pmatrix} b_x \\ b_y \\ b_z \end{pmatrix} = \begin{pmatrix} a_y b_z - b_y a_z \\ -a_x b_z + b_x a_z \\ a_x b_y - b_x a_y \end{pmatrix} = a \times b$

Ora possiamo riscrivere la derivata di $R(t)$ come

$$\dot{R}(t) = \begin{pmatrix} \omega(t)^* \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix} & \omega(t)^* \begin{pmatrix} r_{yx} \\ r_{yy} \\ r_{yz} \end{pmatrix} & \omega(t)^* \begin{pmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{pmatrix} \end{pmatrix}$$

In base alle regole della moltiplicazione di matrici, possiamo scomporla in

$$\dot{R}(t) = \omega(t)^* \begin{pmatrix} \begin{pmatrix} r_{xx} \\ r_{xy} \\ r_{xz} \end{pmatrix} & \begin{pmatrix} r_{yx} \\ r_{yy} \\ r_{yz} \end{pmatrix} & \begin{pmatrix} r_{zx} \\ r_{zy} \\ r_{zz} \end{pmatrix} \end{pmatrix}$$

E quindi $\dot{R}(t) = \omega(t)^* R(t)$ or $\omega(t)^* = \dot{R}(t) R(t)^T$

mi permette di
ricavare la
velocità di
rotazione però 60
in forma
matriciale

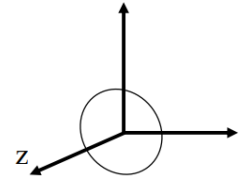
equazione differenz

Esempio: considera la seguente matrice di rotazione elementare attorno all'asse z, come funzione del tempo.

$$R(t) = \begin{bmatrix} \cos \alpha(t) & -\sin \alpha(t) & 0 \\ \sin \alpha(t) & \cos \alpha(t) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\omega(t)^* = \dot{R}(t)R(t)^T = \begin{bmatrix} -\dot{\alpha}(t) \sin \alpha(t) & -\dot{\alpha}(t) \cos \alpha(t) & 0 \\ \dot{\alpha}(t) \cos \alpha(t) & -\dot{\alpha}(t) \sin \alpha(t) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos \alpha(t) & \sin \alpha(t) & 0 \\ -\sin \alpha(t) & \cos \alpha(t) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & -\dot{\alpha}(t) & 0 \\ \dot{\alpha}(t) & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \rightarrow \omega = [0 \quad 0 \quad \dot{\alpha}]^T$$



Consideriamo una particella r_{0i} nello spazio del corpo al tempo t , la posizione della particella nello spazio globale $r_i(t)$ è $r_i(t) = R(t)r_{0i} + x(t)$

La velocità della particella nello spazio globale è $\dot{r}_i(t) = \omega^* R(t)r_{0i} + v(t)$

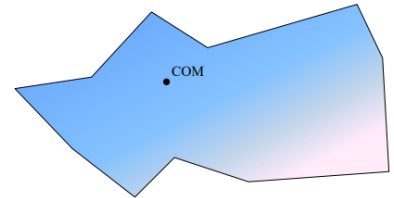
Questo separa la velocità di un punto su un corpo rigido in due componenti: una componente lineare e una componente angolare.

La *massa* è la misura della quantità di materia nel volume di un oggetto $m = \int_V \rho(x, y, z) dV$ dove ρ è la densità in ciascuna posizione nel volume dell'oggetto V . Per un oggetto 3D, la massa è quindi l'integrale sul suo volume lungo le tre dimensioni

$$m = \int \int \int \rho(x, y, z) dx dy dz$$

Per oggetti a densità uniforme (che di solito sono i corpi rigidi), la massa è quindi $m = \rho * V$, dove ρ è la densità dell'oggetto e V il suo volume.

Il centro di massa (COM) è il punto in cui tutta la massa può essere considerata "concentrata", punto di "equilibrio" dell'oggetto. Se la densità è uniforme, il COM è anche il baricentro. In un motore fisico, a ciascun corpo rigido viene assegnata una massa.



Le coordinate del COM: $\left(\frac{1}{m} \int_V \rho(x, y, z) * x dV, \frac{1}{m} \int_V \rho(x, y, z) * y dV, \frac{1}{m} \int_V \rho(x, y, z) * z dV \right)$

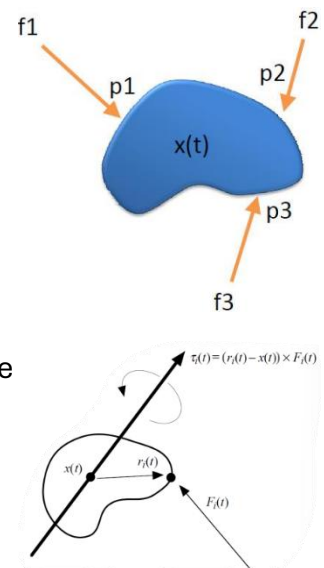
Per un corpo composto da particelle $COM = \frac{1}{m} \sum_{i=1}^n m_i p_i$, dove m_i è la massa di ciascuna particella p_i nel corpo.

Quando una forza agisce su un corpo rigido, quella forza agisce su una particolare particella (posizione) del corpo. Sia $F_i(t)$ la i -esima forza che agisce su un corpo rigido al tempo t . La forza esterna totale $F(t)$ che agisce sul corpo è $F(t) = \sum F_i(t)$

Definiamo $\tau_i(t)$ come l' i -esimo momento torcentale esterno che agisce sul corpo rigido: $\tau_i(t) = (r_i(t) - x(t)) \times F_i(t)$

Il momento torcentale dipende dalla posizione $r_i(t)$ della particella, nello spazio globale, rispetto al centro di massa $x(t)$. Possiamo pensare alla direzione di $\tau_i(t)$ come l'asse attorno al quale il corpo ruoterebbe a causa di $F_i(t)$, se il centro di massa fosse mantenuto saldamente in posizione. Il momento torcentale esterno totale è definito come

$$\tau(t) = \sum \tau_i(t) = \sum (r_i(t) - x(t)) \times F_i(t)$$



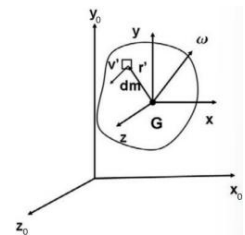
La quantità di moto lineare $P(t)$ è definita come la massa di un oggetto moltiplicata per la sua velocità lineare (cioè, la velocità del centro di massa, o COM): $P(t) = Mv(t)$

Poiché M è una costante $\dot{v}(t) = \frac{\dot{P}(t)}{M}$ e $\dot{P}(t) = F(t)$, la quantità di moto lineare dell'oggetto dipende dal sistema di riferimento in cui viene misurata la velocità.

Il momento angolare $L(t)$ ("momento della quantità di moto" o "momento angolare") permette di esprimere l'effetto della coppia totale $\tau(t)$ sul corpo, ed è definito come $L(t) = I(t)\omega(t)$, dove $I(t)$ è una matrice 3x3, chiamata tensore d'inerzia di un corpo (vedi prossimo). La relazione tra $L(t)$ e la coppia totale $\tau(t)$ è (dimostrazione omessa): $\dot{L}(t) = \tau(t)$. Il momento angolare $L(t)$ di un corpo dipende dal sistema di riferimento rispetto al quale è definito.

$I(t)$ è il tensore d'inerzia di un corpo nel sistema di riferimento del mondo, calcolato al centro di massa e con gli assi paralleli allo spazio del mondo.

$$I = \begin{pmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{yx} & I_{yy} & -I_{yz} \\ -I_{zx} & -I_{zy} & I_{zz} \end{pmatrix}$$



Insieme di integrali di volume:

$$\begin{aligned} I_{xx} &= \int_V \rho(x, y, z) (y^2 + z^2) dV & I_{xy} &= I_{yx} = \int_V \rho(x, y, z) (xy) dV \\ I_{yy} &= \int_V \rho(x, y, z) (x^2 + z^2) dV & I_{xz} &= I_{zx} = \int_V \rho(x, y, z) (zx) dV \\ I_{zz} &= \int_V \rho(x, y, z) (x^2 + y^2) dV & I_{yz} &= I_{zy} = \int_V \rho(x, y, z) (yz) dV \end{aligned}$$

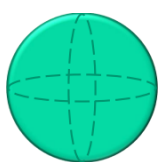
Il tensore d'inerzia $I(t)$ descrive come la massa di un corpo è distribuita rispetto al centro di massa del corpo. Il tensore $I(t)$ dipende dall'orientazione di un corpo, ma non dipende dalla traslazione del corpo.

$$I(t) = \int_V \rho(x, y, z) \begin{bmatrix} y^2 + z^2 & -xy & -xz \\ -xy & z^2 + x^2 & -yz \\ -xz & -yz & x^2 + y^2 \end{bmatrix} dx dy dz$$

$$L(t) = \begin{bmatrix} I_{xx} & -I_{xy} & -I_{xz} \\ -I_{yx} & I_{yy} & -I_{yz} \\ -I_{zx} & -I_{zy} & I_{zz} \end{bmatrix} \begin{pmatrix} \omega_x(t) \\ \omega_y(t) \\ \omega_z(t) \end{pmatrix} = I(t) * \omega(t)$$

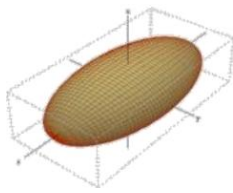
Il tensore d'inerzia è costante nel sistema di riferimento del corpo, ma varia nel sistema di riferimento del mondo. Per qualsiasi oggetto esiste (almeno uno) un insieme di assi principali per cui il tensore d'inerzia è diagonale. Si può dimostrare che gli assi principali sono ortogonali. Gli assi principali sono spesso allineati con gli assi di simmetria dell'oggetto. A ogni frame della simulazione, il tensore d'inerzia nel sistema di riferimento del mondo può essere calcolato tramite $I_{world}(t) = R(t) * I_{body}(t) * R(t)^T$, dove R è la matrice di rotazione che descrive l'orientamento del corpo nel sistema di riferimento del mondo. $I_{body}(t)$ è il tensore d'inerzia calcolato nel sistema di riferimento del corpo (che è costante nel tempo e può essere pre-calcolato).

Esempio: sfera solida di raggio r e massa m



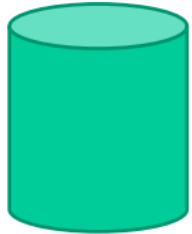
$$I = \begin{bmatrix} \frac{2}{5}mr^2 & 0 & 0 \\ 0 & \frac{2}{5}mr^2 & 0 \\ 0 & 0 & \frac{2}{5}mr^2 \end{bmatrix}$$

Esempio: ellissoide solido con semi-assi a, b, c e massa m



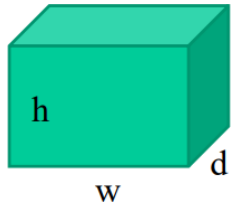
$$I = \begin{bmatrix} \frac{1}{5}m(b^2 + c^2) & 0 & 0 \\ 0 & \frac{1}{5}m(a^2 + c^2) & 0 \\ 0 & 0 & \frac{1}{5}m(a^2 + b^2) \end{bmatrix}$$

Esempio: cilindro solido, raggio r, altezza h e massa m



$$I = \begin{bmatrix} \frac{1}{12}m(3r^2 + h^2) & 0 & 0 \\ 0 & \frac{1}{12}m(3r^2 + h^2) & 0 \\ 0 & 0 & \frac{1}{5}mr^2 \end{bmatrix}$$

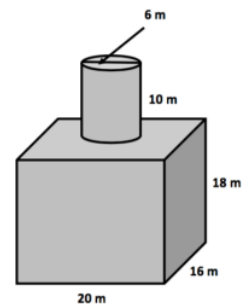
Esempio: scatola solida, larghezza w, altezza h, profondità d e massa m



$$I = \begin{bmatrix} \frac{1}{12}m(h^2 + d^2) & 0 & 0 \\ 0 & \frac{1}{12}m(w^2 + d^2) & 0 \\ 0 & 0 & \frac{1}{12}m(w^2 + h^2) \end{bmatrix}$$

Quando un singolo oggetto rigido è composto da più forme primitive, è possibile calcolare la matrice d'inerzia a partire dalle matrici d'inerzia delle forme primitive.

1. Calcola l'inerzia individuale di ciascuna forma.
2. Usa il teorema degli assi paralleli (lo tralasciamo) per trasformare l'inerzia rispetto a un asse che passa per il centro di massa (COM) dell'oggetto.
3. Somma le matrici d'inerzia.



RIGID BODY EQUATIONS OF MOTION

Per un corpo rigido definiamo il suo vettore di stato $X(t)$ come: $X(t) = \begin{pmatrix} x(t) \\ R(t) \\ P(t) \\ L(t) \end{pmatrix}$

Pertanto, lo stato di un corpo rigido è la sua posizione e orientazione (che descrivono le informazioni spaziali) e la sua quantità di moto lineare e angolare (che descrivono le informazioni sulla velocità). In ogni momento, le quantità ausiliarie $I(t)$, $\omega(t)$ e $v(t)$ sono calcolate tramite $v(t) = \frac{P(t)}{M}$, $I(t) = R(t)I_{body}R(t)^T$ e $\omega(t) = I(t)^{-1}L(t)$.

La derivata del vettore di stato $X(t)$ è $\frac{d}{dt}X(t) = \frac{d}{dt} \begin{pmatrix} x(t) \\ R(t) \\ P(t) \\ L(t) \end{pmatrix} = \begin{pmatrix} v(t) \\ \omega(t)^*R(t) \\ F(t) \\ \tau(t) \end{pmatrix}$

QUATERNIONS VS ROTATION MATRICES

Per la simulazione di corpi rigidi, i quaternioni sono più adatti rispetto alle matrici di rotazione, che soffrono di deriva numerica. Poiché i quaternioni hanno solo quattro parametri, invece di nove, il grado di ridondanza è notevolmente inferiore per i quaternioni rispetto alle matrici di rotazione. Una rotazione di θ radi attorno a un asse unitario u è rappresentata dal quaternione unitario: $[\cos(\frac{\theta}{2}), \sin(\frac{\theta}{2})u]$.

La derivata di un quaternione è $\dot{q}(t) = \frac{1}{2}\omega(t)q(t)$, dove $\omega(t)q(t)$ è un'abbreviazione per la moltiplicazione tra i quaternioni $[0, \omega(t)]$ e $q(t)$.

NUMERICAL INTEGRATION

Ricorda che $FORZA=MASSA \times ACCELERAZIONE$. Se assumiamo che la massa sia costante, allora $F(p_0, t) = m * a(p_0, t)$. Sappiamo che $v'(t) = a(t)$ e $p'_0(t) = v(t)$. Quindi abbiamo $F(p_0, t) = m * p_0''(t)$

Questa è un'equazione differenziale: un ramo ben studiato della matematica, spesso difficile da risolvere nelle applicazioni in tempo reale.

La serie di espansione di Taylor di una funzione può essere applicata su p_0 a $t+\Delta t$:

$$p_0(t + \Delta t) = p_0(t) + \Delta t * p'_0(t) + \frac{\Delta t^2}{2} p''_0(t) + \dots + \frac{\Delta t^n}{n!} p_0^{(n)}(t)$$

Ma ovviamente non conosciamo i valori dell'intera serie infinita; al massimo abbiamo $p_0(t)$ e le prime due derivate.

Approssimazione di primo ordine (nota come metodo di Eulero): assumiamo una velocità costante per il tempo Δt trascorso tra due fotogrammi

$$v(t + \Delta t) = v(t) + \frac{F(t)}{m} \Delta t = v(t) + a(t) \Delta t$$

$$p_0(t + \Delta t) = p_0(t) + v(t) \Delta t$$

$\Delta t = 1$ $\Delta t = 0.5$

Time	Acceleration	Position	Velocity
0	2	0	0
1	2	0	2
2	2	2	4
3	2	6	6
4	2	12	8

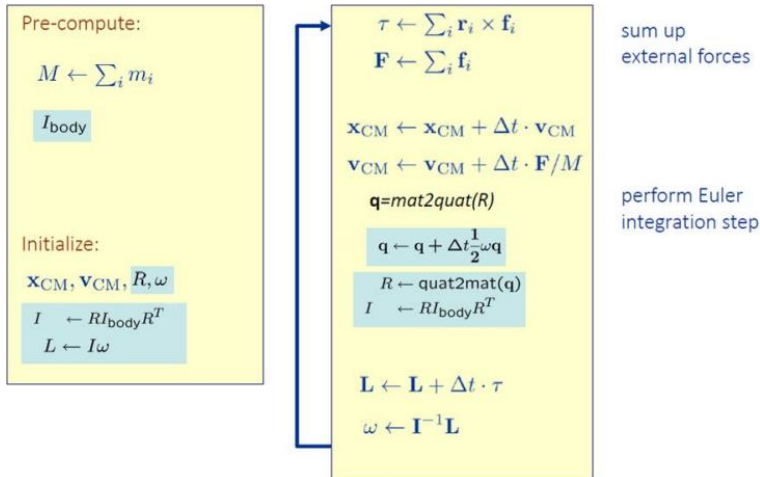
Time	Acceleration	Position	Velocity
0	2	0	0
0.5	2	0	1
1	2	0.5	2
1.5	2	1.5	3
2	2	3	4
2.5	2	5	5
3	2	7.5	6
3.5	2	10.5	7
4	2	14.0	8
4.5	2	18.0	9

Più è piccolo Δt , più precisa è l'approssimazione. Quindi, la soluzione classica è ridurre Δt il più possibile (di solito la frequenza dei fotogrammi del ciclo di gioco è sufficiente). Metodi di integrazione più avanzati (midpoint, Runge-Kutta). *Midpoint*: accuratezza di secondo ordine per la posizione e accuratezza di primo ordine per la velocità

$$v(t + \Delta t) = v(t) + a(t) \Delta t$$

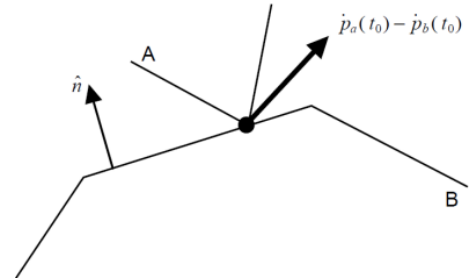
$$p_0(t + \Delta t) = p_0(t) + v(t) \Delta t + a(t) \frac{\Delta t^2}{2}$$

UNCONSTRAINED RIGID BODY PHYSICS: FRAME LOOP



COLLIDING CONTACT

Assumiamo che un sistema di rilevamento delle collisioni identifichi un contatto tra due corpi A e B al tempo t_0 . Utilizziamo un metodo basato sugli impulsi per modellare l'evento di collisione (applichiamo impulsi agli oggetti in collisione, nel punto di collisione). Assumiamo corpi senza attrito (la collisione con attrito è omessa). Per corpi senza attrito, la direzione dell'impulso per il corpo A sarà nella direzione normale: $\mathbf{J} = j \hat{\mathbf{n}}(t_0)$. Il corpo B è soggetto a un impulso uguale ma opposto $-\mathbf{J}$.



Un'altra assunzione: collisioni bordo-bordo o vertice-faccia (altri casi sono considerati degenerati). Vettore normale unitario di contatto:

- ❖ *Caso faccia-vertice*: utilizza la normale della faccia.
- ❖ *Caso bordo-bordo*: utilizza il prodotto vettoriale dei vettori direzione dei due bordi.

Sia $\dot{\mathbf{p}}_a^-(t_0)$ la velocità del vertice di contatto di A prima che venga applicato l'impulso, e sia $\dot{\mathbf{p}}_a^+(t_0)$ la velocità del vertice di contatto dopo l'impulso (siano $\dot{\mathbf{p}}_b^-(t_0)$ e $\dot{\mathbf{p}}_b^+(t_0)$ le velocità del vertice di contatto di B prima e dopo l'impulso).

- Le velocità relative dei punti di contatto nella direzione normale prima e dopo il contatto sono:

$$v_{rel}^- = \hat{\mathbf{n}}(t_0) \cdot (\dot{\mathbf{p}}_a^-(t_0) - \dot{\mathbf{p}}_b^-(t_0))$$

$$v_{rel}^+ = \hat{\mathbf{n}}(t_0) \cdot (\dot{\mathbf{p}}_a^+(t_0) - \dot{\mathbf{p}}_b^+(t_0))$$

La legge empirica per le collisioni senza attrito afferma semplicemente che: $v_{rel}^+ = -\epsilon v_{rel}^-$ $0 \leq \epsilon \leq 1$ è il *coefficiente di restituzione*. Se $\epsilon=1$, allora la collisione è perfettamente "rimbalzante" e non si perde energia cinetica. Dall'altra parte, se $\epsilon=0$, si perde una quantità massima di energia cinetica. Dopo questo tipo di collisione, i due corpi saranno in contatto statico nel punto di contatto p.

Dopo alcune manipolazioni delle equazioni...:

$$j = \frac{-(1 + \epsilon)v_{rel}^-}{\frac{1}{M_a} + \frac{1}{M_b} + \hat{\mathbf{n}}(t_0) \cdot \left(I_a^{-1}(t_0)(\mathbf{r}_a \times \hat{\mathbf{n}}(t_0)) \right) \times \mathbf{r}_a + \hat{\mathbf{n}}(t_0) \cdot \left(I_b^{-1}(t_0)(\mathbf{r}_b \times \hat{\mathbf{n}}(t_0)) \right) \times \mathbf{r}_b}$$

L'impulso può quindi essere utilizzato per calcolare una variazione istantanea delle velocità lineari e angolari come descritto nella diapositiva precedente.

CONSTRAINED RIGID BODY DYNAMICS

In pratica, nessun corpo rigido è libero di muoversi "da solo". Il movimento è vincolato (es: ruote su un'auto, parti del corpo umano, porta che si apre). *Coppia cinematica*: connessione tra due corpi che impone vincoli sul loro movimento relativo.

- Coppia inferiore: vincolo su un punto, una linea o un piano.
 - Coppia revolute, o giunto a cerniera: 1 gradi di libertà rotazionali (DOF).
 - Giunto prismatico, o scorrevole: 1 gradi di libertà traslazionali (DOF).
 - Coppia sferica, o giunto a sfera: 3 gradi di libertà rotazionali (DOF).
 - Coppia planare: 3 gradi di libertà traslazionali (DOF).
- Coppia superiore: vincolo su una curva o superficie.

Si scopre che i vincoli e le collisioni possono essere formulati in modi molto simili (non entriamo nei dettagli). *Impulsi sequenziali* (SI): applicare impulsi a ciascun vincolo in modo iterativo per correggere la velocità. I vincoli di velocità potrebbero produrre una deriva di posizione, rompendo il vincolo.

COMPLETE PHYSICS-BASED FRAME LOOP

- **Fase 1:** integrare le forze applicate, ottenendo velocità provvisorie (dinamica non vincolata).
- **Fase 2:** eseguire il rilevamento delle collisioni per trovare i punti di contatto e le normali di contatto, calcolare le velocità lineari e angolari in quei punti di contatto, utilizzare i coefficienti di restituzione e la conservazione della quantità di moto per determinare gli impulsi da applicare, applicare gli impulsi in modo sequenziale per tutti i vincoli (collisioni e giunti) per correggere la velocità.
- **Fase 3:** utilizzare le nuove velocità per aggiornare le posizioni degli oggetti.

Bullet Physics Library

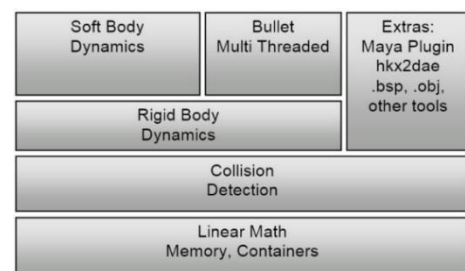
Un motore fisico gestisce la simulazione del mondo (dinamica dei corpi rigidi, risolutori di vincoli, collisioni...) e fornisce al motore grafico la trasformazione del mondo aggiornata per tutti gli oggetti. Bullet Physics è una libreria professionale open source per il rilevamento delle collisioni, la dinamica dei corpi rigidi e dei corpi deformabili (C++). La libreria è gratuita per uso commerciale sotto la licenza ZLib.

Ordine di operazioni di base:

1. Inizializzazione e creazione dei corpi rigidi
2. Operazioni del ciclo fisico (ripeti fino al completamento)
3. Pulizia

Operazioni del ciclo fisico

1. Modificare i parametri fisici
2. Eseguire un passo della simulazione

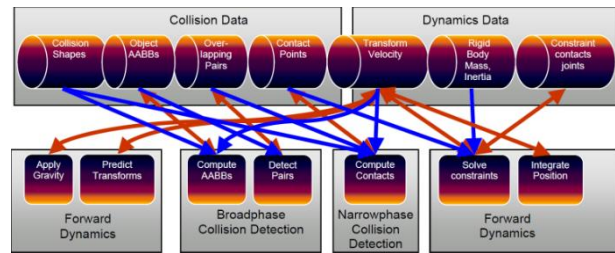


3. Aggiornare la grafica
4. Tornare al Passo 1, fino alla fine del ciclo fisico.

Pipeline della libreria Bullet

L'intera computazione della pipeline fisica e le sue strutture dati sono rappresentate in Bullet da un mondo dinamico. Quando si esegue `stepSimulation` sul mondo dinamico, tutte le fasi sottostanti vengono eseguite.

L'implementazione predefinita del mondo dinamico è `btDiscreteDynamicsWorld`.



Prima viene definito il gestore di simulazione di alto livello `btDiscreteDynamicsWorld`, gestisce gli oggetti fisici e i vincoli e implementa la chiamata di aggiornamento a tutti gli oggetti ad ogni fotogramma. Poi gli oggetti vengono creati e aggiunti al

`DynamicsWorld`. `btRigidBody` avrà bisogno di:

- la massa (>0 per oggetti dinamici, 0 per oggetti statici)
- la forma di collisione `btCollisionShape` (scatola, sfera, ecc.)
- le proprietà del materiale (attrito, restituzione, tensor di inerzia nel sistema di riferimento locale)

Infine, la simulazione viene aggiornata ad ogni fotogramma chiamando `stepSimulation(...)` sul mondo dinamico. `stepSimulation` eseguirà il rilevamento delle collisioni e la simulazione fisica. Aggiorna la trasformazione del mondo per gli oggetti attivi chiamando il `setWorldTransform` di `btMotionState`.

INITIALIZATION

```
// Collision configuration contains default setup for memory, collision setup
btDefaultCollisionConfiguration * collisionConfiguration = new
btDefaultCollisionConfiguration();

// Set up the collision dispatcher
btCollisionDispatcher * dispatcher = new btCollisionDispatcher(collisionConfiguration);

// Set up broad phase method
btBroadphaseInterface * overlappingPairCache = new btDbvtBroadphase();

// Set up the constraint solver
btSequentialImpulseConstraintSolver * solver = new
btSequentialImpulseConstraintSolver();

btDiscreteDynamicWorld * dynamicWorld = new btDiscreteDynamicWorld(dispatcher,
overlappingPairCache, solver, collisionConfiguration);

dynamicsWorld->setGravity(btVector(0,-9.81,0));
```

SIMULATION

```
for (int i=0; i<100; i++) {
    dynamicsWorld->stepsSimulation(1.0f/60.f, 10);

    // print positions of all objects
    for (int j=dynamicsWorld->getNumCollisionObjects()-1; j>=0; j--) {
        btCollisionObject * obj = dynamicsWorld->getCollisionObjectArray() [j];
        btRigidBody * body = btRigidBody::upcast(obj);
        if (body && body->getMotionState()) {
            btTransform trans;
            body->getMotionState()->getWorldTransform(trans);
            printf("World pos = %f,%f,%f\n", float(trans.getOrigin().getX()),
float(trans.getOrigin().getY()), float(trans.getOrigin().getZ()));
        }
    }
}
```

L'upcast è un downcast, poiché btRigidBody deriva da btCollisionObject.

TERMINATION

```
// remove the rigid bodies from the dynamics world and delete them
for (int i=dynamicsWorld->getNumCollisionObjects()-1; i>=0; i--) {
    btCollisionObject * obj = dynamicsWorld->getCollisionObjectArray() [i];
    btRigidBody * body = btRigidBody::upcast(obj);
    if (body && body->getMotionState()) delete body->getMotionState();
    delete obj;
}

// delete collision shapes
for (int j=0; j<collisionShapes.size(); j++) {
    btCollisionShape * shape = collisionShapes[j];
    collisionShapes[j] = 0;
    delete shape;
}

delete dynamicWorld;
delete solver;
delete overlappingPairCache;
delete dispatcher;
delete collisionConfiguration;
```

BULLET PHYSICS LIBRARY

Simulation step

Bullet utilizza un passo di tempo fisso interno (fixedTimeStep) di 60 Hertz. Il "timeStep" può essere una costante o può essere impostato come il tempo trascorso dall'ultima chiamata (per sincronizzarsi con il rendering dei fotogrammi). Bullet suddividerà automaticamente il passo di tempo in un numero di passi di tempo fissi interni, fino a un numero massimo di sottopassi specificato come secondo argomento della funzione stepSimulation.

btDynamicsWorld::stepSimulation(btScalar timeStep, int maxSubSteps=1, btScalar fixedTimeStep=btScalar(1.)/btScalar(60.));

timeStep - the amount of time in seconds to step the simulation by.

maxSubSteps - the maximum number of substeps that Bullet is allowed to take each time you call it.

fixedTimeStep - internal fixed time step

Bullet Detection

Bullet fornisce algoritmi e strutture per il rilevamento delle collisioni:

- *btCollisionObject*: Oggetto con trasformazione nel mondo e forma di collisione.
- *btCollisionShape*: Descrive la forma di collisione di un oggetto di collisione (scatola, sfera, involucro convesso o mesh di triangoli, ecc.), di solito centrato attorno all'origine del sistema di coordinate locali. Una singola forma di collisione può essere condivisa tra più oggetti di collisione.
- *btCollisionWorld*: Memorizza tutti i *btCollisionObject* e fornisce un'interfaccia per eseguire interrogazioni.

La fase preliminare respinge rapidamente le coppie di oggetti che non collidono utilizzando un albero dinamico di volumi delimitanti basato sugli AABB. Può essere sostituito da un altro algoritmo (vedi prossimo punto).

Collision Dispatcher

Un dispatcher delle collisioni itera su ogni coppia di oggetti che potrebbero collidere e richiama l'algoritmo di collisione corrispondente a ciascuna configurazione. Questi algoritmi restituiscono il tempo di impatto, i punti più vicini su ciascun oggetto e la profondità di penetrazione/vettore di distanza.

	BOX	SPHERE	CONVEX, CYLINDER, CONE, CAPSULE	COMPOUND	TRIANGLE MESH
BOX	boxbox	spherebox	gjk	compound	concaveconvex
SPHERE	spherebox	spheresphere	gjk	compound	concaveconvex
CONVEX, CYLINDER, CONE, CAPSULE	gjk	gjk	gjk	compound	concaveconvex
COMPOUND	compound	compound	compound	compound	compound
TRIANGLE MESH	concaveconvex	concaveconvex	concaveconvex	compound	gimpact

Collision Filtering

Bullet utilizza un piccolo margine di collisione per le forme di collisione, al fine di migliorare le prestazioni e l'affidabilità. Impostato su un fattore di 0,04 (cioè, espande la forma di 4 cm se l'unità di misura è il metro). Bullet fornisce tre modi per personalizzare il filtraggio delle coppie di oggetti:

- *Masks*: ID definiti dall'utente (possono essere visti come strati in 2D) che raggruppano insieme oggetti che possono collidere.
- *Callback del filtraggio in fase preliminare*: callback definite dall'utente chiamate nella fase preliminare del rilevamento delle collisioni.
- *Nearcallbacks*: callback definite dall'utente chiamate nella fase finale e ristretta del rilevamento delle collisioni.

La dinamica del corpo rigido è implementata sopra il rilevamento delle collisioni. Aggiunge forza, massa, inerzia, velocità e vincoli. L'oggetto principale del corpo rigido è *btRigidBody*: gli oggetti in movimento hanno massa e inerzia diverse da zero; eredita la trasformazione nel

mondo, l'attrito e la restituzione da *btCollisionObject* (cioè, *btCollisionObject* è la classe padre di *btRigidBody*); aggiunge velocità lineare e angolare.

Bullet ha 3 tipi di corpi rigidi:

- ❖ *Corpi dinamici (in movimento)*: hanno massa positiva e la loro posizione viene aggiornata a ogni frame.
- ❖ *Corpi statici (non in movimento)*: hanno massa zero, non possono muoversi ma possono collidere.
- ❖ *Corpi cinematici*: hanno massa zero, possono essere animati dall'utente (possono spingere corpi dinamici ma non reagire a essi).

La trasformazione del mondo di un corpo rigido in Bullet deve essere relativa al suo centro di massa, e la sua base definisce anche il sistema di coordinate locale per l'inerzia. Bullet richiede inoltre che il sistema di coordinate locale del corpo rigido sia allineato con gli assi principali di inerzia. Se la forma di collisione non è allineata con il centro di massa, può essere spostata in una forma composta. La classe *btCollisionShape* fornisce un metodo per calcolare automaticamente l'inerzia locale in base alla forma e alla massa. L'inerzia può essere modificata se la forma di collisione è diversa dalla forma dell'inerzia.

Basic Data Types and Math Library

Un *btScalar* è un numero in virgola mobile.

- *btVector3*: vettori 3D.
- *btQuaternion* e *btMatrix3x3*: orientamenti e rotazioni 3D (Bullet utilizza un sistema di coordinate destro).
- *btTransform*: combinazione di una posizione e un'orientazione.

Per risparmiare molti calcoli inutili, non vogliamo simulare un oggetto che non si muove; ad esempio, un oggetto fermo a terra o una molla a riposo. Dobbiamo implementare due funzionalità: una per disattivare un oggetto e una per riattivare un oggetto. Il rilevatore di collisioni continua a restituire i contatti con oggetti disattivati, ma l'integrazione numerica viene saltata per gli oggetti disattivati. Un oggetto viene disattivato quando sia le velocità lineari che quelle angolari sono al di sotto di una soglia (valori specifici per il corpo). L'oggetto viene riattivato quando collide con un altro oggetto attivo o quando forze esterne non costanti vengono applicate all'oggetto.

I vincoli dei corpi rigidi sono definiti come *btTypedConstraint*. Bullet include diversi vincoli, come il giunto a cerniera (1 grado di libertà rotazionale) e il giunto sferico (3 gradi di libertà rotazionali). I limiti del vincolo sono definiti per ciascun grado di libertà (DOF). Limite inferiore e limite superiore. 3 configurazioni:

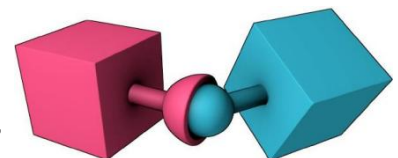
- se limite inferiore = limite superiore, il DOF è bloccato
- se limite inferiore > limite superiore, il DOF è illimitato
- se limite inferiore < limite superiore, il DOF è limitato in quel range

Vincoli

Point-to-point (giunto sferico): limita la traslazione in modo che i punti di pivot locali di 2 corpi rigidi coincidano nello spazio globale. *pivotInA* e *pivotInB* sono nello spazio locale, non nello spazio globale.

```
btPoint2PointConstraint(btRigidBody& rbA, btRigidBody& rbB, const  
btVector3& pivotInA, const btVector3& pivotInB);
```

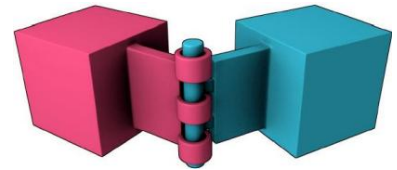
Unisce due corpi in un unico punto,
blocca tutti i movimenti di
spostamento



permette ai corpi di ruotare attorno a un asse (centrale), blocca tutti gli altri movimenti quindi toglie gradi di libertà, l'utente può impostare i limiti di rotazione

Vincolo a cerniera, o giunto rivoluzionario: restringe due ulteriori gradi di libertà angolari, in modo che il corpo possa ruotare solo attorno a un asse, l'asse della cerniera. Questo può essere utile per rappresentare porte o ruote che ruotano attorno a un asse. L'utente può specificare limiti e motore per la cerniera. Ogni corpo ha un punto di pivot e un asse nello spazio locale.

```
btHingeConstraint(btRigidBody& rbA, btRigidBody& rbB, const btVector3& pivotInA, const btVector3& pivotInB, btVector3& axisInA, btVector3& axisInB, bool useReferenceFrameA = false);
```



Il **vincolo a slitta** consente al corpo di ruotare attorno a un asse e di traslare lungo questo asse. *frameInA* e *frameInB* sono trasformazioni dallo spazio del vincolo agli spazi di BodyA e BodyB. L'asse di traslazione è sempre lungo l'asse X negli spazi del vincolo.

```
btSliderConstraint(btRigidBody& rbA, btRigidBody& rbB, const btTransform& frameInA, const btTransform& frameInB, bool useLinearReferenceFrameA);
```

permette due movimenti sullo stesso asse, una rotazione e una traslazione



La classe interfaccia *btMotionState* consente al mondo dinamico di sincronizzare e interpolare le trasformazioni del mondo aggiornate con la grafica. Un corpo rigido può essere creato caricando una mesh triangolare da un file che descrive la forma di un oggetto.

risolve il disaccoppiamento tra il motore fisico e grafica tramite la sincronizzazione, interpolazione e ottimizzazione

Motion Capture Systems – Motion Capture Technologies

Registrare i movimenti di oggetti o esseri umani: animazione, cinesologia, riabilitazione, giochi, robotica, realtà virtuale/aumentata.

OBJECT MOTION CAPTURE

Cattura del movimento degli oggetti in alta definizione: in tempo reale (possibilmente), corpi rigidi, 3 o 6 Gradi di Libertà (DoF)

Per la realtà virtuale e aumentata: tracciamento del visore, visualizzazione di oggetti reali in movimento interazione con oggetti reali (aptica: replica di compiti)

SKELETAL TRACKING

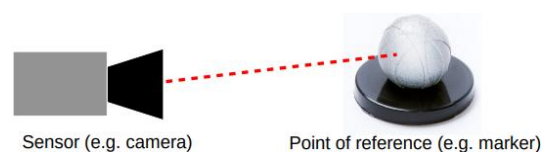
Cattura del movimento di un essere umano come un insieme di “ossa” rigide. Struttura dati ad albero: scheletro (modello cinetico). Movimento a 6 DoF di un “osso” radice (ad es. torace). Angoli delle articolazioni o posizione di ciascun “osso”. Lunghezza di ciascun “osso”: altezza dell'utente, lunghezza del braccio.

Per la grafica computerizzata: animazioni

Per la realtà virtuale e aumentata: tracciamento dell'utente e del visore; riconoscimento della postura e dei gesti; avatar dell'utente → percezione di sé e degli altri utenti

MOTION CAPTURE TECHNOLOGIES

I sistemi di MoCap sono composti da sensori che misurano la posizione/orientamento di alcuni punti di riferimento (PoR). I sensori e/o i PoR possono essere sull'oggetto (o sulla persona) o fissi nell'ambiente: tre categorie.



Category	Sensors	PoR
Inside In	On object/person	On object/person
Inside Out	On object/person	Stationary
Outside In	Stationary	On object/person

Inside In	Inside Out	Outside in
Esoscheletro Suite inerziale Ottico 	Elettromagnetico Ottico 	Ottico: marker passivi, marker attivi, senza marker  sensori posizionati in parti fisse dell'ambiente e osservano l'oggetto muoversi

traccia il movimento
delle parti del corpo
senza guardare ambiente
esterno, dall'interno
verso esterno

il sistema di tracciamento è montato direttamente
sull'oggetto, osserva l'ambiente circostante per
capire dove si trova

INSIDE IN: EXOSKELETONS

L'utente indossa un esoscheletro con sensori che misura gli angoli delle articolazioni nel tempo (meccanico, fibra ottica). Può anche essere un dispositivo aptico. Tempo reale e alta precisione. Problemi: nessuna posizione globale, limita i movimenti, deve essere adattato all'utente.

INSIDE IN: INERTIAL SUITES

I sensori inerziali misurano l'accelerazione (accelerometri) e l'orientamento (giroscopi): 6 DoF. Tracciano la posizione attuale rispetto alla posizione iniziale. Disponibili negli smartphone. Real time. Problemi: posizione globale "relativa", accumulo di errori: drift, possono essere ingombranti.

INSIDE IN: OPTICAL

Telecamere posizionate sul corpo dell'utente che osservano il corpo dell'utente, per lo più sperimentali. Problemi: nessuna posizione globale, bassa precisione.

INSIDE OUT: ELECTRO-MAGNETIC

I sensori sono posizionati sull'utente. Tracciano la posizione (6 DoF) rispetto a una fonte esterna, la stazione di origine emette un campo magnetico. Tempo reale, la precisione varia. Problemi: interferenze da oggetti metallici, possono essere ingombranti, molto costosi.

POLHEMUS FASTRAK

Source: emette tre campi magnetici oscillanti ortogonali

Sensor: contiene tre bobine perpendicolari, le correnti sono indotte dalla fonte

Specifiche: 6 DoF; frequenza di aggiornamento: 120 Hz; precisione: 0,76 mm, 0,15°; latenza: 4 ms; portata: fino a 1,5 m.

INSIDE OUT: OPTICAL

Telecamere sull'utente: ottiche inside-out. Odometria visiva, telecamere tracciano caratteristiche visive. Utilizzate in molti visori VR. Problemi: potrebbero richiedere molta potenza di elaborazione, non scalabile per molti utenti o oggetti, ambiente scuro/uniforme/dinamico.

INSIDE OUT: IR OPTICAL

Aiuta l'approccio ottico utilizzando proiettori IR nella stanza. I proiettori IR inviano "pulsazioni" di luce. Le telecamere misurano la distanza dai proiettori. Approccio time-of-flight (simile al GPS). Problemi: necessita di proiettori IR esterni, non scalabile per molti utenti/oggetti, interferenze da fonti IR

OUTSIDE IN: OPTICAL

Rilevamento di utenti e oggetti utilizzando una o più telecamere. Funziona solo nell'area osservata dalle telecamere. I target (marker) possono essere piccoli ed economici. Alto costo di installazione, ma alta scalabilità.

OUTSIDE IN: OPTICAL (PASSIVE MARKERS)

Marker passivi: semplici sfere. Illuminati da una fonte di luce esterna (di solito IR). Alta precisione. Problema principale: tutti i marker sono sfere. Come identificarli?

OUTSIDE IN: OPTICAL (ACTIVE MARKERS)

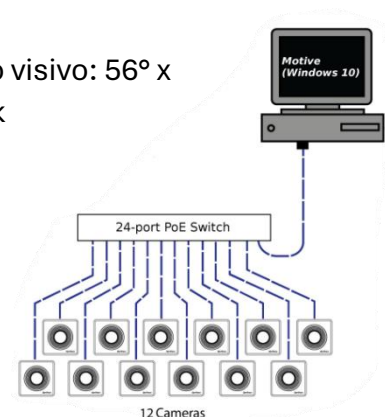
Marker attivi: luci LED. Ogni luce pulsa a una frequenza unica. Identificazione più semplice. Precisione ancora più elevata. Problemi: meno economici e semplici, necessita di una fonte di alimentazione.

OUTSIDE IN: OPTICAL (MARKER-LESS)

Utilizzo di telecamere di profondità o RGB-D. Movimenti completamente liberi. Problemi: interferenze da fonti IR, bassa precisione.

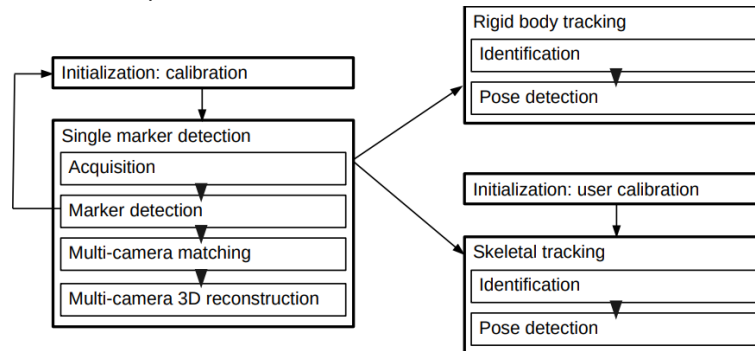
OPTITRACK MOTION CAPTURE SYSTEM

12 Prime 13 camere. Risoluzione: 1280 x 1024 @ 240 FPS. Campo visivo: 56° x 46°. Proiettore: IR 850 nm. Switch Power Over Ethernet. OptiTrack "Motive". Software proprietario



PASSIVE MARKER-BASED MOTION CAPTURE

Rilevamento di marker singoli, tracciamento di corpi rigidi (per oggetti), tracciamento scheletrico (per esseri umani).

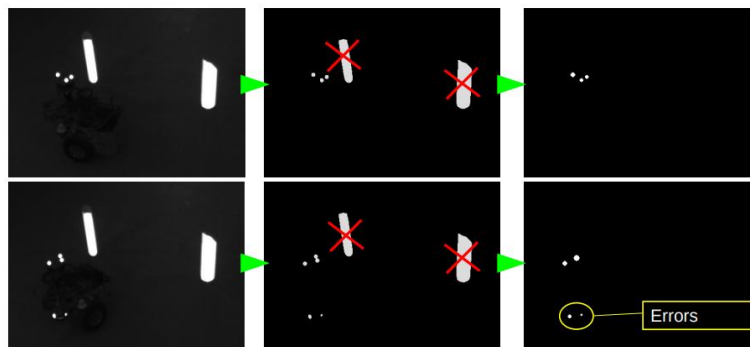


SINGLE MARKER: ACQUISITION

Proiettore IR nell'infrarosso vicino (850 nm), telecamera IR, immagine in scala di grigi per intensità.

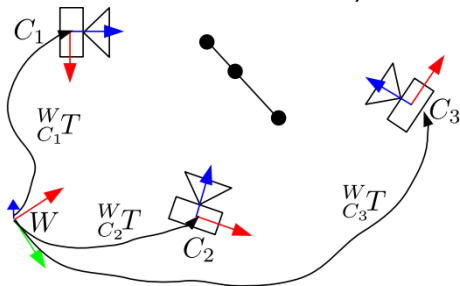
SINGLE MARKER: MARKER DETECTION

Soglia e segmentazione: trovare le macchie bianche. Rimozione dei falsi positivi basata su dimensioni e forma.



INITIALIZATION: CALIBRATION

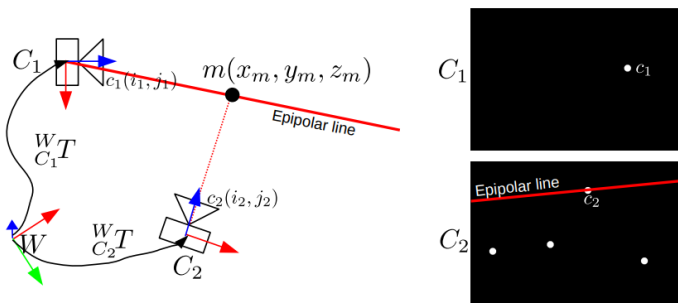
Abbiamo bisogno di trasformazioni tra il mondo e la telecamera ${}^W_{C_k}T$, trovate una sola volta, per inizializzare il sistema. Una configurazione di marker nota viene spostata (ad es. bacchetta di calibrazione)



per la calibrazione si utilizza una bacchetta, ovviamente il sistema conosce la geometria della bacchetta quindi sarà in grado di poter determinare la posizione delle telecamere tramite un algoritmo di ottimizzazione

SINGLE MARKER: MULTI-CAMERA MATCHING

Un punto in un'immagine è una linea nelle altre: linea epipolare. Data una marcatura in un'immagine, cercare un marker corrispondente lungo la linea epipolare in un'altra immagine. Risolvi le ambiguità utilizzando più di due telecamere.



SINGLE MARKER: RECONSTRUCTION

Trovare $m(x_m, y_m, z_m)$ dato $c_k(i_k, j_k)$. Tre incognite, ogni telecamera fornisce due equazioni, sono necessarie almeno due telecamere. Ottimizzazione dei minimi quadrati: minimizzare l'errore $e = \sum_k e_{ik}^2 + e_{jk}^2$. In generale, non esiste una soluzione in forma chiusa

Ck corrisponde ai pixel presenti nell'immagine, solo che per determinare i parametri x_m, y_m, z_m ho bisogno di un'altra telecamera per poter risolvere il sistema

$$\begin{cases} \begin{bmatrix} i_1 + e_{i1} \\ j_1 + e_{j1} \\ 1 \end{bmatrix} \propto K_1 W_{C_1} T^{-1} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix} \\ \begin{bmatrix} i_2 + e_{i2} \\ j_2 + e_{j2} \\ 1 \end{bmatrix} \propto K_2 W_{C_2} T^{-1} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix} \\ \dots \end{cases}$$

Camera matrices K

perché il sistema è sovradeterminato si hanno più equazioni che incognite (aggiungendo seconda telecamera) quindi serve un processo di calcolo iterativo

SINGLE MARKER: RECONSTRUCTION

per poter completare l'equazione viene introdotto il parametro lambda (profondità del punto) per avere un'uguaglianza

$$\lambda \begin{bmatrix} i_1 + e_{i1} \\ j_1 + e_{j1} \\ 1 \end{bmatrix} \propto K_1 W_{C_1} T^{-1} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix}$$

Roto-translation (4x4)
Proportional to

$$\exists \lambda_1 \in \mathbb{R} \quad \lambda_1 \begin{bmatrix} i_1 + e_{i1} \\ j_1 + e_{j1} \\ 1 \end{bmatrix} = K_1 W_{C_1} T^{-1} \begin{bmatrix} x_m \\ y_m \\ z_m \\ 1 \end{bmatrix}$$

coordinate dei pixel + l'errore

vettore delle incognite

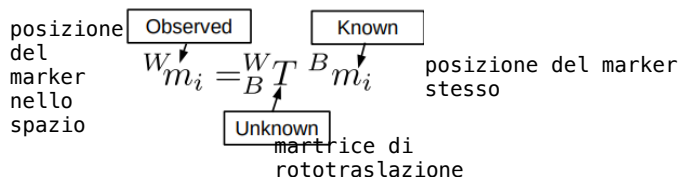
Camera matrix (projection matrix) (3x4)

$$K_1 = \begin{bmatrix} f_x & 0 & c_x & 0 \\ 0 & f_y & c_y & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

f_x Focal length (x)
 f_y Focal length (y)
 c_x Image center (x)
 c_y Image center (y)

RIGID BODY

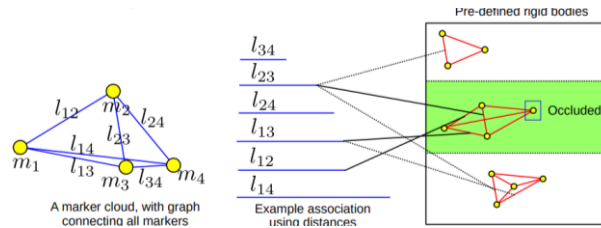
Corpo rigido: almeno tre marker rigidamente vincolati. Di solito, un oggetto con marker. Data una cornice di riferimento B sul corpo rigido. Ogni marker ha un centro in una posizione ${}^B m_i$



RIGID BODY: IDENTIFICATION

Data una nuvola di marker, trovare il corpo rigido. Sottografo comune massimo (NP-completo). Problemi: alcuni marker potrebbero non essere visibili (occlusione), "Distribuire" più corpi rigidi vicini, ambiguità per corpi rigidi simili.

altri possono avere delle configurazioni di marker molto simili fra loro oppure i corpi rigidi sono così vicini che i loro punti si intersecano quindi è difficile distinguerli



RIGID BODY: POSE DETECTION

Dobbiamo trovare ${}^W_B T$, da B a W (world)

$$\begin{bmatrix} {}^W m_i \\ 1 \end{bmatrix} = {}^W_B T \begin{bmatrix} {}^B m_i \\ 1 \end{bmatrix} \Rightarrow {}^W m_i = {}^W_B R {}^B m_i + {}^W t_B$$

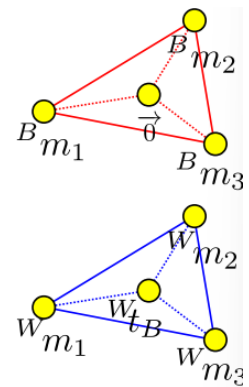
Per trovare ${}^W t_B$, usa il centro del corpo rigido (non dipende dalla rotazione)

${}^W t_B = \frac{1}{n} \sum_i {}^W m_i$. Per trovare la rotazione:

$$\underbrace{\begin{bmatrix} ({}^W m_1 - {}^W t_B) & \dots & ({}^W m_n - {}^W t_B) \end{bmatrix}}_C \simeq {}^W_B R \underbrace{\begin{bmatrix} {}^B m_1 & \dots & {}^B m_n \end{bmatrix}}_A$$

Risolvere $\min_R \|RA - C\|$

Chiamato problema di Procruste ortogonale, abbiamo bisogno di almeno 3 marker non collineari.

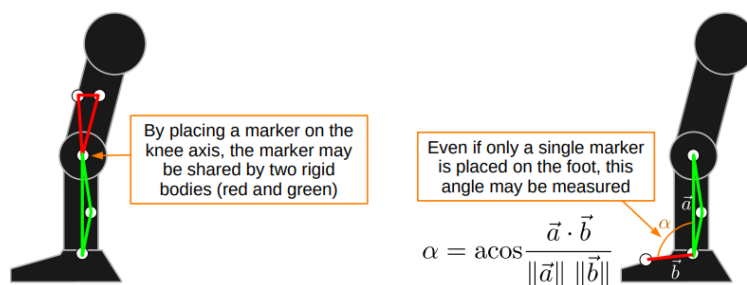


SKELETAL TRACKING

L'utente indossa una tuta per la cattura del movimento. I marker sono attaccati alla tuta.

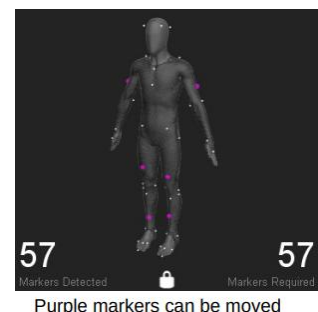
SKELETAL TRACKING: MARKER CONFIGURATIONS

Potremmo creare un corpo rigido per ogni "osso". Problema principale: ridurre al minimo il numero di marker. Diverse configurazioni ottimizzate disponibili a seconda dell'applicazione. Minimizzare il numero di marker: un marker può essere condiviso da più corpi rigidi. Posizionare i marker sugli assi delle articolazioni (minimizzare il movimento). Sfruttare il limitato grado di libertà delle articolazioni umane. Nessun metodo generale: ottimizzazione ad-hoc.



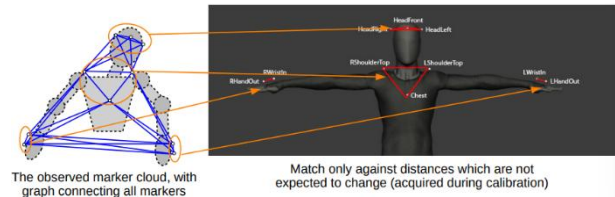
SKELETAL TRACKING: USER CALIBRATION

L'utente assume una postura predefinita. Il software memorizza le posizioni dei marker e le lunghezze delle ossa. Alcuni marker possono essere spostati leggermente per differenziare gli utenti.



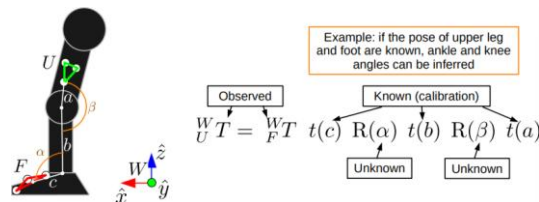
SKELETAL TRACKING: IDENTIFICATION

I marker che formano un corpo rigido possono essere identificati come prima. Per gli altri marker, ottimizzazioni e euristiche ad-hoc. Esempio: sottografo comune massimo con le distanze registrate durante la calibrazione.



SKELETAL TRACKING: POSE DETECTION

Per i marker non presenti nei corpi rigidi, utilizzare vincoli cinematici: alcune articolazioni umane non sono articolazioni a sfera (3 DoF) oppure possono essere approssimate: compromesso. Inferenza (cinematica inversa) per “ossa” senza marker, maggiore imprecisione e ridotta robustezza.



SKELETAL TRACKING: ERRORS

- I. Marker visibili insufficienti
- II. Interferenze da fonti IR, alcuni dispositivi VR utilizzano anch'essi IR, oggetti riflettenti IR
- III. Le distanze relative non sono preservate, i marker si muovono troppo
- IV. Interferenze da marker di altri corpi rigidi o utenti

TEMPORAL TRACKING

Tracciamento di marker, corpi rigidi e scheletro nel tempo. Fornisce una stima iniziale per gli algoritmi di ottimizzazione, risolve alcune ambiguità. "Ripulisce" la traiettoria.

PRACTICAL ADVICE

- i. Evitare fonti IR o materiali riflettenti IR, inclusi altri utenti e corpi rigidi
- ii. Almeno due telecamere per vedere un marker (tre è meglio)
- iii. Almeno tre marker per vedere un corpo rigido
- iv. Evitare sottografi comuni in più corpi rigidi
- v. Evitare marker collineari (3 o più)
- vi. La distanza relativa tra i marker non deve cambiare
- vii. Occlusioni: non tutti i marker potrebbero essere visibili
- viii. Per i movimenti dell'utente che si auto-occludono, sono necessarie più telecamere e/o più marker

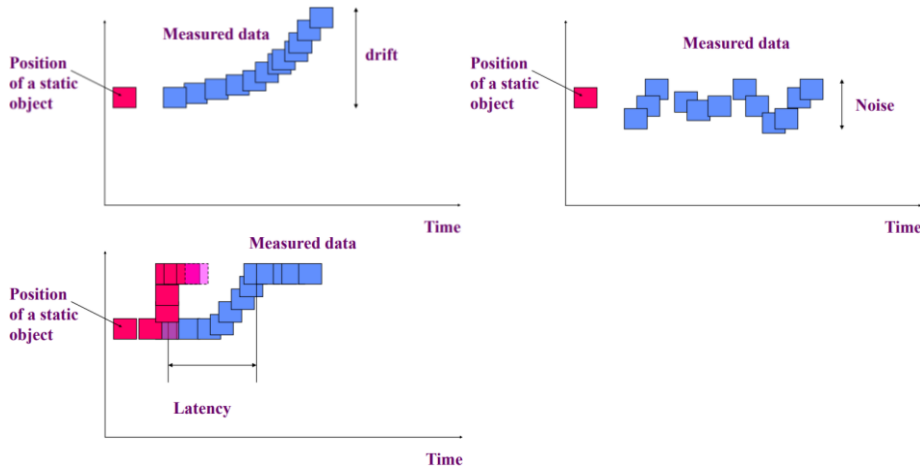
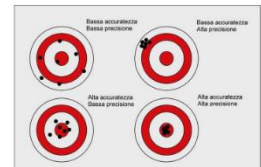
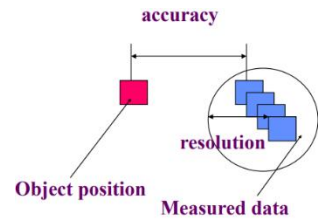
Augmented Reality

CARATTERISTICHE DEI SENSORI

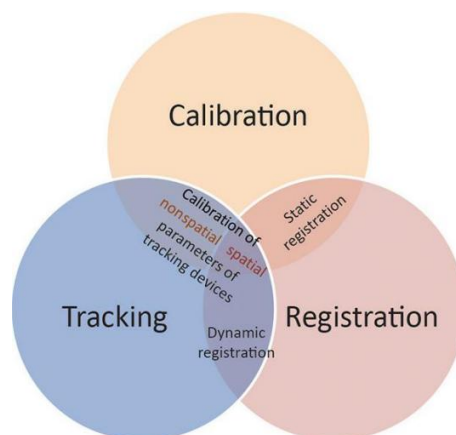
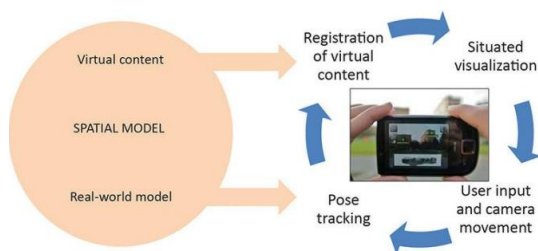
Accuratezza: L'errore tra il valore reale e il valore misurato.

Precisione: La dispersione casuale dei valori misurati attorno ai valori medi misurati.

Risoluzione: La più piccola misura/cambiamento che uno strumento può rilevare o misurare.



AUGMENTED REALITY



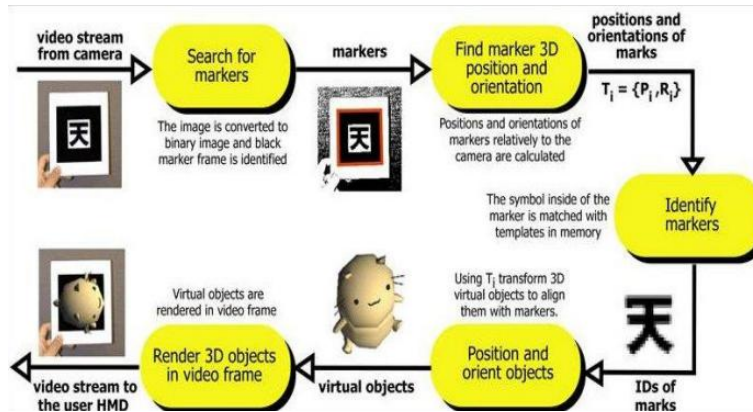
CALIBRATION

1. Calibrare la tua vera fotocamera
2. La matrice estrinseca corrisponde alla matrice modelview di OpenGL
3. La matrice intrinseca deve essere convertita nella matrice di proiezione di OpenGL

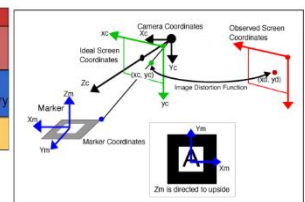
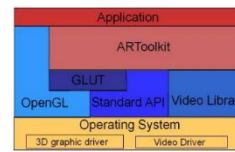
Nel modello di fotocamera pinhole di OpenCV, i parametri intrinseci sono: f_x (lunghezza focale orizzontale), f_y (lunghezza focale verticale), c_x (coord X del centro della fotocamera), c_y (coord Y del centro della fotocamera). Devi trasformarla in una matrice di proiezione di OpenGL, in modo da poter effettuare il rendering sopra l'immagine originale utilizzando la pipeline grafica di OpenGL. Questo problema di calcolo della matrice di proiezione di OpenGL dalla matrice della fotocamera di OpenCV NON è facile, ma può essere risolto in forma chiusa.

ARTOOLKIT

Una libreria software basata su marker per la realtà aumentata.



Tracciamento 6DOF dei marker (dimensione e forma conosciute). Linux, Windows, open-source (<http://www.hitl.washington.edu/artoolkit/>). Fornisce strumenti per la calibrazione della fotocamera. Autonomo (non richiede OpenCV).



ARToolKit (REGISTRATION)

Dopo la sogliatura dell'immagine di input, vengono estratte le regioni il cui contorno può essere approssimato da quattro segmenti di linea. Quando due lati paralleli di un marker quadrato vengono proiettati sull'immagine, le equazioni di quei segmenti di linea nelle coordinate dello schermo della fotocamera sono le seguenti:

$$a_1x + b_1y + c_1 = 0, \quad a_2x + b_2y + c_2 = 0$$

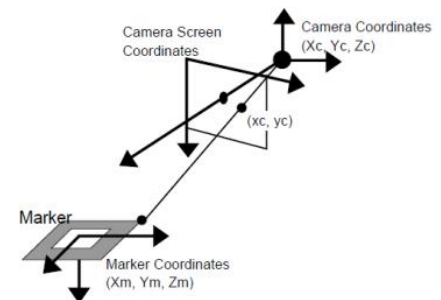
Date la matrice di proiezione prospettica P che si ottiene dalla calibrazione della fotocamera, le equazioni dei piani che includono rispettivamente questi due lati possono essere rappresentate come

$$P = \begin{bmatrix} P_{11} & P_{12} & P_{13} & 0 \\ 0 & P_{22} & P_{23} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad \begin{bmatrix} hx_c \\ hy_c \\ h \end{bmatrix} = P \begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix}$$

$$a_1P_{11}X_c + (a_1P_{12} + b_1P_{22})Y_c + (a_1P_{13} + b_1P_{23} + c_1)Z_c = 0$$

$$a_2P_{11}X_c + (a_2P_{12} + b_2P_{22})Y_c + (a_2P_{13} + b_2P_{23} + c_2)Z_c = 0$$

Dati i vettori normali n1 e n2 di questi piani, il vettore direzionale dei lati paralleli del marker è il prodotto vettoriale dei vettori normali: $n_1 \times n_2$. Dato che due vettori direzionali unitari ottenuti da due coppie di lati paralleli devono essere perpendicolari, la componente di rotazione $V_{3 \times 3}$ della matrice di trasformazione T_{cm} può essere calcolata. Le coordinate dei quattro vertici del marker nel sistema di coordinate del marker (che sono note poiché la dimensione del marker è conosciuta in anticipo) e quelle coordinate nel sistema di coordinate dello schermo della fotocamera possono poi essere utilizzate per calcolare la componente di traslazione W_x, W_y, W_z .



$$\begin{bmatrix} X_c \\ Y_c \\ Z_c \\ 1 \end{bmatrix} = \begin{bmatrix} V_{11} & V_{12} & V_{13} & W_x \\ V_{21} & V_{22} & V_{23} & W_y \\ V_{31} & V_{32} & V_{33} & W_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_m \\ Y_m \\ Z_m \\ 1 \end{bmatrix}$$

$$= \begin{bmatrix} V_{3 \times 3} & W_{3 \times 1} \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X_m \\ Y_m \\ Z_m \\ 1 \end{bmatrix} = T_{cm} \begin{bmatrix} X_m \\ Y_m \\ Z_m \\ 1 \end{bmatrix}$$

MARKER-BASED AR

Pros

Posizionamento molto accurato ($< \text{cm}$, $< \text{gradi}$), rilevamento veloce, economico e facile, ampiamente supportato.

Cons

Necessità di attrezzare l'ambiente.